

全向与定向干扰源的无源测向定位与自主清除模型

摘要

及时排查清除干扰源是无线电频谱管理的重要任务。干扰源的传统精准定位需技术人员徒步排查、定位并清除，耗时费力且存在安全风险，机器狗的发展为该问题提供了新的解决途径。本文针对利用多个检测点示向度对干扰源进行无源定位，并由机器狗在频道、位置与有效接收半径均未知的条件下快速自主完成发现、定位与彻底清除的问题，建立了从几何交会定位到路径调度优化的完整模型体系。

针对问题一：针对示向误差不超过 1° 的凸锥半平面模型，将每个示向度转化为两条半平面约束，利用 Sutherland-Hodgman 顺序裁剪求交，并以旋转卡壳算法在 $O(n \log n)$ 时间内计算定位区域直径。通过内接、外切正 720 边形对目标圆域双侧逼近，直径上下界差在 10^{-13} m 量级。证明直径圆覆盖的充要条件为最小覆盖圆半径等于直径之半；等边三角形反例表明直径圆一般不能覆盖，Jung 定理给出保证半径上界 $D/\sqrt{3}$ 。数值算例定位区域为六边形，直径 44.14 m，直径圆恰好覆盖，验证了算法可靠性。

针对问题二：在首次示向度已知条件下，定义完整可能源集合 U_1 ，构建 1000 m 保证接收区 C_{1000} 与利用首次成功检测信息的条件保证区 C_{det} 。采用一阶误差传播构造面积代理 $A_{\text{proxy}} = \pi F_1^2$ ，通过 Pareto 非支配筛选与字典序选点确定第二检测点。验证场景中选得 $S_2 = (993.74, -143.47)$ m，局部耗时 205.81 s，最坏确定性误差界 60.88 m，保证最远源距 999.91 m。该策略兼顾接收可靠性与定位精度，避免仅优化交会角的片面性。

针对问题三：设计七点扫描骨架（原点加半径 1200 m 正六边形），证明最坏最近点距离 968.90 m < 1000 m，可发现全部全向源。提出保证第二观测点 $S' = S + 400u + 300v$ ，其接收信号有严格证明；以最小覆盖圆半径 ≤ 20 m 为清除判据，并设网格兜底。两阶段批量调度将平均定位清除时间从 507.9 s 降至 418.5 s（降 17.6%）。快速版迭代阶段的 15 局演练平均为 262.2 s，清除失败 0 次。三次正式测试清除干扰源 40 个，平均定位清除时间 289.91 s，程序运行时间 2198–2404 ms，空清除 0 次，20 个频道全部取得确定性结论。时间构成中移动占 80%，优化重点在路径。

针对问题四：引入定向源盲区导致“无信号”三义性，建立方向覆盖证书：扫描集须满足 $G \in \text{conv}\{P_j : \|P_j - G\| \leq 1000\}$ 。采用 31 点等边三角格（边长 990 m）或 22 点认证集，配合连续域凸包认证，保证任意未知朝向至少一个扫描点可见。定位遇无信号时转扇形 20 m 网格覆盖式清除，有限步内必成功。快速版迭代阶段的 15 局演练平均为 499.1 s，清除 198/198；三次正式测试清除 39 个，平均 462.66 s，空清除 2 次均在预算内，程序运行时间 1976–2426 ms。模型以完备性优先，时间约为问题三的 1.7–2 倍，但程序运行时间远低于 1200 s 上限。

关键词： 无源测向定位；凸锥半平面交会；方向覆盖证书；覆盖式清除；路径调度优化

一、问题重述

1.1 问题背景

无线电频谱是重要战略资源，其有序使用关系到通信、导航和广播等业务的正常运行。然而，随着无线设备激增，各类干扰源大量涌现，及时排查清除干扰源成为频谱管理的紧迫任务[?]。传统干扰排查通常经历初步监测、区域排查和人工近场定位，最后一环耗时且可能使人员暴露于危险环境。而搭载测向机、光学定位装置和激光清除装置的机器狗（四足机器人），凭借“感知—决策—移动—清除”的自主闭环，为人工近场定位的问题提供解决方案；但示向误差、未知接收半径、未知频道以及定向源盲区，使系统同时面临几何不确定性和路径决策问题。如何制定策略并设计算法使机器狗在最短时间内完成全部干扰源的检测、定位和清除，是本文研究的核心问题。

1.2 问题重述

问题一：由若干检测点坐标和同一干扰源相对应的示向度，在 $\pm 1^\circ$ 误差下构造交会定位区域，求其直径，并讨论以直径为定位区域直径的圆能否覆盖该区域。

问题二：已知某全向干扰源在第一检测点的示向度，设计第二检测点的选取策略，使示向度再次接收具有确定性保证，并在定位精度与移动检测耗时之间取得权衡，给出候选区域及推荐站址。

问题三：设定目标圆域内有10–16个频道互异的全向干扰源，其数量、位置、频道和精确接收半径均未知。机器狗从原点、频道1出发，自主完成发现、定位、清除；要求设计算法策略，使其在清除全部干扰源的基础上，最小化任务完成时间。同时要求借助模拟器开展演练，完成三次正式测试，记录清除数量、平均定位清除时间和程序运行时间等统计量，并计算被清除干扰源个数的比例以及平均定位清除时间。

问题四：在问题三的基础上加入数量和朝向均未知的定向干扰源。此时一次无信号还可能由定向盲区造成，需重新设计发现证书与清除策略，并完成模拟器测试。

二、问题分析

本文整体采用“几何建模—误差传播—优化选点—覆盖认证—调度清除”的递进思路：先由示向度与误差界建立凸锥半平面交会模型，解决单次定位区域及其直径与覆盖判定；再在首次观测已知条件下，以可能源集合为基础，兼顾接收可靠性与定位精度设计第二检测点；最后面向机器狗自主作业，通过可证明覆盖的扫描骨架、保证信号的取点规则与最小覆盖圆清除判据，实现多源发现、定位、清除与终止认证。各问层层递进，均以几何与优化为核心。

问题一：由于测向误差不超过 1° ，若把示向度视为精确射线，多个方向的交点通常只是一个点，既无法反映真实定位范围，也无法讨论覆盖性，因此建模起点必须是带宽度的方向约束。将真实方位角视为落在以测得方向为中心、张角 2° 的扇形内，每个检测点便给出一个以该点为顶点的凸锥，干扰源须同时位于所有凸锥与半径1800m目标圆域的交集内。求解时，先将每个示向锥化为两条半平面约束，用内接、外切正多边形双侧逼近圆域，再以Sutherland–Hodgman逐半平面裁剪得到凸多边形定位区域；由于凸多边形最远点对必在顶点处取得，可用凸包与旋转卡壳求直径。覆盖性问题不能仅由直径判断，核心是比较最小覆盖圆半径与直径之半是否相等，并用等边三角形反例说明直径圆并非普遍覆盖。

问题二：此问与问题一不同：第一次检测点及示向度已知，第二次检测点待设计，而真实距离、第二次示向度及误差实现均未知，因此选点必须先保证再次收到信号，再追求定位区域最小。第一次读数只给出带误差的示向锥，结合目标距离大于5m、接收半径不超过1500m及目标圆域，可得二维可能源集合 U_1 。保证接收的最保守做法是要求第二检测点到 U_1 任意点距离不超过1000m，得核

心保证区；利用第一次成功接收提供的距离下界，可放宽为条件保证区。精度目标上，由于第二次读数未知且面积非凸，采用一阶误差传播构造面积代理用于排序，第二次观测后再按问题一模型重算实际区域。选点按“可能源集合—保证接收区—面积代理与耗时—Pareto 选点—数值验证”展开，接收可靠是首要约束，不能只追求交会角接近直角。

问题三：此问要求机器狗从原点出发，在源数量、频道、位置、接收半径均未知的条件下完成全部源的定位与清除。核心困难有三：一是发现性，没有任何先验位置信息，必须设计有限步内必能发现全部源的扫描方案，并能对“某频道不存在”给出结论；二是定位收敛性，每次测向只给 1° 角度信息，单次交会无法直接达到光学清除精度，需设计保证持续获得新观测的取点规则；三是计时口径，移动、换频、检测、清除耗时规则不同，策略必须与计时规则一一对应。求解时，先设计七点扫描骨架（原点加半径 1200m 正六边形）保证覆盖目标圆域；首次获得方位后，取 $S' = S + 400u + 300v$ 保证第二观测必有信号；对同一源的全部观测用半平面交求定位区域并取最小覆盖圆，半径不超过 20m 时执行清除，否则追加观测或用网格兜底。调度上采用“发现—批量定位清除”两阶段，将同一骨架点发现的源合并处理，减少往复里程。

问题四：此问在问题三基础上引入定向干扰源，其个数与朝向均未知。关键变化在于“扫不到信号”多了一种解释：检测点可能落在定向源盲区（有效覆盖角为朝向两侧各 90° ）。因此不能再用“全部扫描点无信号”直接证明该频道不存在。求解时，先设计对任意未知朝向都保证发现的扫描骨架：固定可能源位置 G ，若全部扫描点均不可见，则存在方向使所有 $P_j - G$ 落在同一开半平面内，等价于 $0 \notin \text{conv } V_G$ ；故只要保证 $G \in \text{conv}\{P_j : \|P_j - G\| \leq 1000\}$ ，就能对任意朝向至少有一个扫描点可见。据此可取边长 990m 的等边三角格扫描其与目标圆域相交的基本三角形顶点，或用 22 点认证集配合连续域凸包证书提速。定位与清除上，首个示向度仍给出位置扇形，但首点可能位于盲区边界，后续无信号不再视为故障，而是触发扇形 20m 网格覆盖式清除，有限步内必清除成功；一旦新增观测使最小覆盖圆半径降到 20m 内则回到单点清除。“确认不存在”必须满足上述凸包条件，其余状态转移与问题三相同。

三、模型假设

题面明确给出的区域尺度、设备阈值和动作耗时均作为已知条件，不再改写为假设。全文采用以下统一假设。

- **假设 1（静态平面几何）：**一次任务内干扰源位置保持不变，检测点坐标已知且执行无误差，按题面忽略测向机与干扰源的高度差（依据：题面条款与定位理想化；用于问题一至问题四）。
- **假设 2（目标属性稳定）：**同一频道的有效观测对应同一干扰源，且其有效接收半径和定向方向在一局内不变（依据：题面条款与任务时段稳定性；用于问题一至问题四）。
- **假设 3（确定性误差）：**示向误差只满足区间界，不预设分布或独立性；同地重复检测的误差固定，故不以重复平均缩小误差（依据：题面条款与最坏情形口径；用于问题一至问题四）。
- **假设 4（理想信号传播）：**除题面给出的接收距离和定向覆盖外，不另引入遮挡、多径或源间干扰（依据：模型闭合需要；用于问题一至问题四）。
- **假设 5（理想运动）：**机器狗视为可即时转向并准确到达指令坐标的质点，只计直线移动和题面规定的串行动作耗时（依据：题面运动规则与时间建模口径；用于问题二至问题四）。
- **假设 6（异常与退化）：**约束交集为空表示观测不相容而非零面积；区域退化为单点或线段时，直径分别按零或线段长度定义（依据：连续集合定义与数值稳健性；用于问题一至问题四）。

3.1 题面条件与计算口径

数学推导始终取题面界 $\delta = 1^\circ$ ；问题三、四的在线程序另以 1.01° 包围示向度，只用新增的 0.01° 吸收两位小数表示和浮点量化误差，不改变物理模型。正常示向度意味着 $5 < \|G - S_i\|_2 \leq R^{\text{recv}} \leq 1500\text{m}$ ；首次正检测还给出 $R^{\text{recv}} \geq \max\{1000, \|G - S_1\|_2\}$ ，问题二据此扩大保证接收区。

表 1 四问共用的题面条件

条件	数值	使用口径
目标圆域与源数 示向误差	$R_0 = 1800\text{ m}$, $10 \leq N_J \leq 16$ $ e_i \leq \delta = 1^\circ$	频道为 1–20, 每频道至多一个源 无分布; 保证性结论取确定性最坏情形
有效接收半径	1000–1500 m	保证接收用下限; 保证超距无信号用 上限
定向覆盖	中心方向两侧各 90°	取含边界的闭半平面, 朝向一局内固 定
距离阈值	近距 5 m; 清除 20 m	近距可直接清除; 清除成功与源朝向 无关
动作耗时	移动 5 m/s, 换频 1 s, 检测 5 s	光学定位 3 s, 激光清除 2 s, 动作串 行

问题三中, 未清除频道的 **no_signal** 可能表示无源或超距; 问题四还可能表示定向盲区, 只有方向完备的扫描证书才能消除这种三义性。若已发现 16 个异频源, 可由数量上界认证其余频道无源; 否则任务仅在 20 个频道均为“已清除”或“确认不存在”时闭合。程序只串行调用 `/enter`、`/measure`、`/clear`、`/exit`, 幂等重试复用同一 **request_id**, 不读取案例真值。

几何程序把全部法向量单位化并采用相对尺度容差。问题一以正 720 边形内外包围圆域并报告直径上下界; 在线清除使用保守外包围。保证接收、超距判定和认证清除均向安全侧留数值余量; 这些阈值属于实现细节, 在相应算法章节给出, 不作为新的物理参数。

四、符号说明

4.1 主要符号

表 2 问题一、二的主要符号

符号	含义	单位或取值	首次出现
S_i, G	第 i 个检测点与单个干扰源的位置向量	m	问题一
$\hat{\theta}_i, \theta_i, e_i$	示向度、真实方位角及示向误差	(°)	问题一
δ	题面示向误差半角	1°	问题一
$d(\phi)$	方位角 ϕ 对应的单位方向向量	无量纲	问题一
W_i, K	第 i 个测向锥与精确定位区域	集合	问题一
K_q^-, K_q^+	圆域内、外接正 q 边形所得定位区域	集合	问题一
$D(K), R_{\text{MEC}}(K)$	区域直径与最小覆盖圆半径	m	问题一
R^{rec}, U_1	有效接收半径与第一次观测后的可能源集合	m; 集合	问题二
X	第二检测点候选位置	m	问题二
$\mathcal{C}_{1000}, \mathcal{C}_{\text{det}}$	核心保证区与条件保证区	集合	问题二
J, K_{LS}	测向 Jacobian 与最小二乘误差增益矩阵	rad/m; m/rad	问题二
γ, GDOP	交会锐夹角与几何精度衰减因子	(°); m/rad	问题二
$F_1(X), f_2(X)$	最坏一阶位置误差与第二次检测耗时	m; s	问题二
$\mathcal{P}$	精度—耗时 Pareto 非支配集	集合	问题二

表 3 问题三、四的主要符号

符号	含义	单位或取值	首次出现
N_J, c	一局中的干扰源总数与频道编号	10–16; 1–20	问题三
K_c, A_c	频道 c 的精确可行区域及其数值外包围	集合	问题三
C_c, ρ_c	A_c 的覆盖圆心及相应保守覆盖半径	m; m	问题三
$\delta_{\text{num}}, \epsilon_{\text{num}}$	在线示向安全半角与清除判定数值余量	1.01° ; 0.001 m	问题三
$\mathcal{S}, P_k, d_{\max}$	七点扫描集、第 k 个扫描点及其最坏最近点距离	集合; m; m	问题三
$V_t^{\text{scan}}, V_t^{\text{service}}$	时刻 t 尚待访问的扫描节点集与频道服务节点集	集合	问题三
$\sigma, L(\sigma)$	节点访问顺序及对应开放路径长度	排列; m	问题三
$\mathbf{o}, V_G$	定向源发射方向及位置 G 的可接收扫描点相对向量集	单位向量; 集合	问题四
P_4, P'_4	31 点基准扫描集与 22 点连续域认证扫描集	集合	问题四
$E(T)$	三角形单元 T 的一致可接收扫描点集合	集合	问题四
$\mathcal{R}_0, s_c, \rho_{\text{cov}}$	首次正检测扇形、清除格边长及其数值覆盖半径	集合; m; m	问题四
T	一局的定位清除虚拟时间	s	问题四
$N_{\text{measure}}, N_{\text{switch}}, N_{\text{clear}}^{\text{success}}, N_{\text{clear}}^{\text{miss}}$	检测、换频以及成功或未发现清除的动作次数	次	问题四

五、问题一：交会区域、直径与圆覆盖

5.1 测向锥与精确定位区域

定义度制方位角的单位方向向量

$$d(\phi) = \left(\cos \frac{\pi \phi}{180}, \sin \frac{\pi \phi}{180} \right)^\top.$$

在检测点 $S_i = (x_i, y_i)^\top$ 得到示向度 $\hat{\theta}_i$ 后，真实方向位于 $[\hat{\theta}_i - \delta, \hat{\theta}_i + \delta]$ 。令 $d_i^\pm = d(\hat{\theta}_i \pm \delta)$ ，二维叉积记为 $[a, b] = a_x b_y - a_y b_x$ ，则闭测向锥为

$$W_i = \{G : [d_i^-, G - S_i] \geq 0, [d_i^+, G - S_i] \leq 0\}.$$

将叉积展开，可直接得到供半平面交使用的标准形式

$$\begin{cases} \sin(\hat{\theta}_i - \delta)x - \cos(\hat{\theta}_i - \delta)y \leq \sin(\hat{\theta}_i - \delta)x_i - \cos(\hat{\theta}_i - \delta)y_i, \\ -\sin(\hat{\theta}_i + \delta)x + \cos(\hat{\theta}_i + \delta)y \leq -\sin(\hat{\theta}_i + \delta)x_i + \cos(\hat{\theta}_i + \delta)y_i. \end{cases}$$

正弦、余弦均按度制角换算为弧度后计算，故跨越 0° 不需另设分支。题目所称交会定位区域按测向锥与目标圆域定义为

$$K = B_{R_0} \cap \bigcap_{i=1}^m W_i, \quad B_{R_0} = \{G : \|G\|_2 \leq R_0\}.$$

K 是有界闭凸集；空集表示观测与先验不相容，单点和线段按退化凸集处理。正常示向度还附带 $5 < \|G - S_i\| \leq 1500$ m，本问数值算例中这些距离约束不活跃；若将其作为定位区域的一部分，则外接圆盘仍保持凸性，而删除 5 m 近距盘会产生非凸开集，应按问题二的闭包口径单独处理。

5.2 半平面交与直径算法

5.2.1 圆域的双侧包围

圆弧边界不宜直接当作多边形顶点。取 $q = 720$ ，以圆周等分点的凸包构成内接正多边形 B_q^- ，以切向半平面

$$B_q^+ = \bigcap_{k=0}^{q-1} \{G : (\cos \phi_k, \sin \phi_k)G \leq R_0\}, \quad \phi_k = \frac{2k\pi}{q}$$

构成外接正多边形，并令

$$K_q^- = B_q^- \cap \bigcap_i W_i, \quad K_q^+ = B_q^+ \cap \bigcap_i W_i.$$

于是

$$K_q^- \subseteq K \subseteq K_q^+, \quad D(K_q^-) \leq D(K) \leq D(K_q^+).$$

外接多边形的最大径向超差为

$$\eta_q = R_0 \left(\sec \frac{\pi}{q} - 1 \right), \quad \eta_{720} < 0.02 \text{ m}.$$

若内包为空而外包非空，只能判定可行性未决；圆弧短到未包含内接顶点时，单纯加密不能替代连续边界证明。本文同时报告上下界；当能证明 $K_q^+ \subseteq B_{R_0}$ 时，有 $K = K_q^+$ ，外包结果可提升为精确结论。

5.2.2 逐半平面裁剪

对当前凸多边形边 XY 和单位化约束 $a^\top Z \leq r$ ，记残差 $g_X = a^\top X - r$ 、 $g_Y = a^\top Y - r$ 。若两端分居边界两侧，则交点为

$$Z = X - \frac{g_X}{g_Y - g_X}(Y - X).$$

按 Sutherland–Hodgman 规则逐边保留内侧点并插入交点，依次施加全部 $2m$ 个测向半平面 [?]。初始多边形有 q 个顶点，第 k 次裁剪后顶点数至多为 $q + k$ ，故时间复杂度为

$$O\left(\sum_{k=1}^{2m} (q + k)\right) = O(mq + m^2),$$

空间复杂度为 $O(q + m)$ 。实现中先单位化法向量，并以多边形局部中心计算残差，使容差具有长度量纲且避免大坐标消减；裁剪为空时返回不可行状态，而不是静默返回直径零。

5.2.3 凸包与旋转卡壳

凸函数 $X \mapsto \|X - Y\|_2$ 在紧凸集上的最大值可在顶点取得，故凸多边形直径必由一对凸包顶点实现。先用 Andrew 单调链去重并生成逆时针凸包 $V_0, \dots, V_{h-1}$ ，再执行下列旋转卡壳步骤 [?]:

1. 对每条边 $V_i V_{i+1}$ ，保持指针 j 指向使 $\|V_{i+1} - V_i, V_j - V_i\|$ 最大的对踵点；
2. 只要 $j + 1$ 对应面积继续增大便前移 j ，并更新 (V_i, V_j) 、 (V_{i+1}, V_j) 的平方距离；
3. 面积相等时同时检查 j 与 $j + 1$ ，防止平行边造成并列最远点对遗漏；扫描全部边后对最大平方距离开方。

对踵点随边界单调前进，卡壳阶段为 $O(h)$ 时间、 $O(1)$ 附加空间；含凸包构造时总复杂度为 $O(n \log n)$ 时间、 $O(n)$ 空间，输入已是严格凸边界时降为 $O(n)$ 。小规模多边形另以 $O(n^2)$ 全点对枚举校验。

5.3 直径圆的覆盖性

设 A, B 是紧集 K 的一组直径端点, $C_d = (A + B)/2$ 、 $R_d = D(K)/2$ 。恒等式

$$\|X - C_d\|_2^2 - R_d^2 = (X - A) \cdot (X - B)$$

说明点 X 位于直径圆内当且仅当右端不大于零。对凸多边形, 只需检查所有顶点; 边界含圆弧时, 还须检查弧端点以及圆弧上背向 C_d 的驻点。

定理 1 (直径圆覆盖的充要条件): 非空紧集 K 的直径圆覆盖 K , 当且仅当

$$R_{\text{MEC}}(K) = \frac{D(K)}{2}.$$

任意覆盖圆必须包含距离为 $D(K)$ 的两点, 故半径至少为 $D(K)/2$; 若直径圆覆盖便取得等号。反之, 若最小覆盖圆半径为 $D(K)/2$, 任一对直径端点都只能是该圆的对径点, 其圆心唯一为 $(A + B)/2$, 故相应直径圆就是最小覆盖圆。

该等式并非恒成立。取边长 $D_0 = 20 \text{ m}$ 的等边三角形 $T = \text{conv}\{A, B, C\}$, 以边 AB 为直径的圆半径为 $D_0/2$, 而第三点到边中点的距离为 $\sqrt{3}D_0/2 > D_0/2$ 。为证明反例与题设相容, 沿三条有向边的反向各延长 $L = 800 \text{ m}$ 设置检测点, 并将示向锥的一条边界与相应支撑边重合、标称方向向内旋转 1° 。每个检测点所见三角形的最大角跨度为

$$\alpha = \frac{180}{\pi} \arctan \frac{10\sqrt{3}}{810} \approx 1.23^\circ < 2^\circ,$$

故三角形完整落入三个测向锥, 而三个支撑半平面的交集恰为 T ; 任一站到 T 的距离不超过 $L + D_0 = 820 \text{ m} < 1000 \text{ m}$ 。把三角形外心平移到原点仍满足 $R_0 = 1800 \text{ m}$, 因此这是严格的题内反例。Jung 定理进一步给出平面有界集的普适上界

$$R_{\text{MEC}}(K) \leq \frac{D(K)}{\sqrt{3}},$$

等边三角形取等号; 因 $1/\sqrt{3} > 1/2$, 仅知直径不能保证半径 $D/2$ 的圆覆盖 [?]。

5.4 数值求解与验证

取名义源 $G_0 = (500, 300) \text{ m}$, 三个检测点为

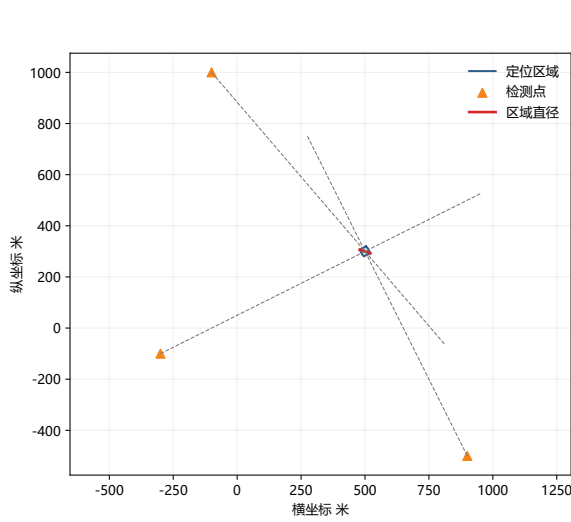
$$S_1 = (-300, -100), \quad S_2 = (900, -500), \quad S_3 = (-100, 1000) \quad (\text{m}),$$

对应名义示向度为 $26.57^\circ, 116.57^\circ, 310.60^\circ$, 站源距离分别为 $894.43, 894.43, 921.95 \text{ m}$ 。三者均小于接收半径下限, 且 $\|G_0\| = 583.10 \text{ m} < R_0$ 。

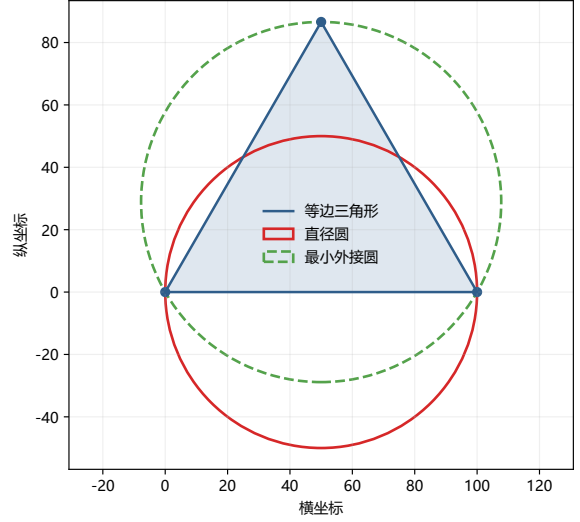
表 4 问题一数值结果及独立校验

指标	结果
定位区域顶点数、面积	6, 940.41 m^2
直径端点 (显示值)	$(520.82, 292.65), (478.94, 306.61) \text{ m}$
旋转卡壳直径	44.1447595094 m
最小覆盖圆半径与 $D/2$	均为 22.0723797547 m
最大半平面违反量	$7.82 \times 10^{-14} \text{ m}$
外包顶点最大原点距离	$599.01 \text{ m} < R_0$

外包多边形所有顶点均在目标圆域内; 由凸性知 $K_q^+ \subseteq B_{R_0}$, 结合 $K \subseteq K_q^+$ 得 $K = K_q^+$, 故表中直径是本例精确区域的结果, 不依赖圆域离散误差。旋转卡壳与全部 15 个顶点对枚举一致; 将顶点回代约束得到表中残差。逐顶点积判据均不大于零, 独立最小覆盖圆也满足 $R_{\text{MEC}} = D/2$, 所



(a) 数值算例的定位区域与最远点对



(b) 等边三角形反例（按相似比例放大显示）

图 1 问题一的实例计算与一般性反例

以本例直径圆恰能覆盖，但一般反例仍然成立。误差半角由 0.5° 增至 1.0° 、 1.5° 时，直径依次约为 22.08, 44.14, 66.20 m，说明本构型在小误差范围内近似线性放大。

图 ?? 为便于辨认按边长 100 m 绘制相似构型；严格反例及其角跨度、接收距离证明仍取 $D_0 = 20$ m、 $L = 800$ m，覆盖失败不受相似缩放影响。

六、问题二：第二检测点的稳健选址

6.1 首次观测后的不确定集合

设第一检测点为 S_1 ，正常示向度为 $\hat{\theta}$ 。真实源不仅有角度不确定性，还受正常检测的距离语义和目标圆域限制，因此完整可能源集合是二维环形扇区

$$U_1 = \left\{ G = S_1 + \rho d(\phi) : 5 < \rho \leq 1500, \left| \text{wrap}_{180}(\phi - \hat{\theta}) \right| \leq \delta, \|G\|_2 \leq R_0 \right\},$$

其中 wrap_{180} 把角差化到 $[-180^\circ, 180^\circ)$ 。把目标预先固定在名义射线或某一距离上，会同时削弱接收保证与精度评价，故不采用这种简化。

为处理一般的圆域截断，令

$$\beta(\phi) = S_1^T d(\phi), \quad \Delta(\phi) = \beta(\phi)^2 + R_0^2 - \|S_1\|_2^2.$$

当 $\Delta(\phi) \geq 0$ 时，射线落在圆域内的径向区间为

$$[\rho_-(\phi), \rho_+(\phi)] = \left[-\beta(\phi) - \sqrt{\Delta(\phi)}, -\beta(\phi) + \sqrt{\Delta(\phi)} \right],$$

故可行径向集合为 $R_f(\phi) = (5, 1500] \cap [\rho_-(\phi), \rho_+(\phi)]$ ；判别式为负或交集为空的方向直接剔除。严格下端点在极值计算中按上确界处理，避免把空集或切点退化误当作普通二维区域。

6.2 保证接收候选区及其边界

6.2.1 两层确定性保证

未知接收半径可能低至 1000 m，因此最保守的核心区为

$$\mathcal{C}_{1000} = \left\{ X : \sup_{G \in U_1} \|X - G\|_2 \leq 1000 \right\}.$$

首次检测已在 S_1 收到信号。若可能源到首站的距离为 ρ ，则该源的实际接收半径至少为 $\max(1000, \rho)$ ，由此得到更大的条件保证区

$$\mathcal{C}_{\text{det}} = \{X : \|X - G\|_2 \leq \max(1000, \|G - S_1\|_2), \forall G \in U_1\}.$$

对每个 $G \in U_1$ ，右端均不超过其实际接收半径，故 $X \in \mathcal{C}_{\text{det}}$ 仍保证第二次收到信号；并且显然有 $\mathcal{C}_{1000} \subseteq \mathcal{C}_{\text{det}}$ 。只按 1500 m 上限构造的覆盖区或面积覆盖率不能提供这一保证，故不进入主决策模型。

6.2.2 连续边界方程

取 $d_{\parallel} = d(\hat{\theta})$ 、 $d_{\perp} = (-d_{\parallel,y}, d_{\parallel,x})$ ，在第一示向局部坐标系中写成

$$X = S_1 + ad_{\parallel} + bd_{\perp}, \quad G = S_1 + \rho(\cos \eta d_{\parallel} + \sin \eta d_{\perp}), \quad |\eta| \leq \delta.$$

于是正确的局部距离平方为

$$L^2(a, b; \rho, \eta) = (a - \rho \cos \eta)^2 + (b - \rho \sin \eta)^2. \quad (1)$$

式 (1) 只含相对角 η ；若在该局部坐标中仍写 $\hat{\theta} + \eta$ ，便会重复旋转坐标轴。核心区的连续残差为

$$F_{\text{core}}(a, b) = \sup_{\substack{|\eta| \leq \delta \\ \rho \in R_f(\hat{\theta} + \eta)}} L(a, b; \rho, \eta) - 1000,$$

从而

$$\mathcal{C}_{1000} = \{S_1 + ad_{\parallel} + bd_{\perp} : F_{\text{core}}(a, b) \leq 0\}, \quad \partial \mathcal{C}_{1000} : F_{\text{core}}(a, b) = 0. \quad (2)$$

当目标圆域不截断 $5 < \rho \leq 1500$ 的扇形时， $\mathcal{C}_{\text{det}}$ 可进一步化简。对 $\rho > 1000$ ，令同向辅助点 $G_0 = S_1 + 1000d(\phi)$ ；若 $\|X - G_0\| \leq 1000$ ，则

$$\|X - G\| \leq \|X - G_0\| + (\rho - 1000) \leq \rho.$$

因此完整条件与只检查 $5 < \rho \leq 1000$ 等价，边界可写为

$$F_{\text{det}}(a, b) = \sup_{\substack{|\eta| \leq \delta \\ 5 < \rho \leq 1000}} L(a, b; \rho, \eta) - 1000 = 0. \quad (3)$$

固定 η 时， L^2 关于 ρ 为凸二次函数，最大值只在径向端点或开端极限取得；固定端点时，角度候选为 $\eta = \pm \delta$ 及满足

$$a \sin \eta - b \cos \eta = 0$$

的区间内驻点。故式 (2)、(3) 都是有限支函数的上包络，可非采样地判断边界内外。一般圆域截断时，式 (2) 仍是完整核心区；程序对 $\mathcal{C}_{\text{det}}$ 使用未截断楔形得到安全内近似，不把它声称完整边界。下文算例满足 $S_1 = (0, 0)$ 且 $1500 < R_0$ ，不存在截断，故式 (3) 给出的条件保证区是精确的。

6.3 精度—耗时 Pareto 模型

6.3.1 局部耗时

机器狗从 S_1 直线移动到 X 并完成第二次测向的耗时为

$$f_2(X) = \frac{\|X - S_1\|_2}{5} + 5 + \mathbf{1}_{\text{switch}}.$$

连续测同一频道时末项为零；若中间插入其他频道动作则为 1。设局部预算为 T_{max} ，以 $\mathcal{C}_{\text{det}}(T_{\text{max}}) = \{X \in \mathcal{C}_{\text{det}} : f_2(X) \leq T_{\text{max}}\}$ 作为主可行区。 T_{max} 是可调决策参数，不是题面常量。

6.3.2 误差传播与 GDOP

设 $G = (x, y)^\top$, 第 i 个检测点为 (x_i, y_i) , $r_i = \|G - S_i\|_2$ 。弧度制测向函数的 Jacobian 行向量为

$$J_i = \begin{bmatrix} -(y - y_i)/r_i^2 & (x - x_i)/r_i^2 \end{bmatrix}, \quad J = \begin{bmatrix} J_1 \\ J_2 \end{bmatrix}.$$

当 J 满秩时, 最小二乘误差增益矩阵为 $K_{LS} = (J^\top J)^{-1} J^\top$ 。令 $\tilde{\delta} = \pi\delta/180$, 角误差盒的四个顶点为 $u \in \{-1, 1\}^2$, 则固定源位置的一阶最坏位置误差为

$$f_{1,\text{loc}}(G; X) = \tilde{\delta} \max_{u \in \{-1, 1\}^2} \|K_{LS} u\|_2, \quad F_1(X) = \sup_{G \in U_1} f_{1,\text{loc}}(G; X). \quad (4)$$

线性映射把误差盒变为平行四边形; 若两视线锐夹角为 γ , 其面积为

$$A_{\text{loc}} = \frac{4\tilde{\delta}^2 r_1 r_2}{|\sin \gamma|} \leq \pi f_{1,\text{loc}}^2 \leq \pi F_1(X)^2.$$

据此取 $A_{\text{proxy}}(X) = \pi F_1(X)^2$ 作为面积代理。它是局部线性化误差集合包含圆面积的上界, 不是两次读数后真实定位区域的面积, 也不是其最小覆盖圆半径。作为独立几何诊断量, 定义

$$\text{GDOP} = \sqrt{\text{tr}[(J^\top J)^{-1}]} = \frac{\sqrt{r_1^2 + r_2^2}}{|\sin \gamma|} \quad (\text{m/rad}),$$

它揭示“距离短、交角近正交”共同改善精度, 而不引入误差分布 [?]。

6.3.3 双目标决策

连续先验在计算中离散为 $U_{1,N}$, 记 $\hat{F}_{1,N} = \max_{G \in U_{1,N}} f_{1,\text{loc}}(G; X)$ 。在保证接收、非近距和非共线条件下求

$$\min_X (\hat{F}_{1,N}(X), f_2(X)), \quad \text{s.t. } X \in \mathcal{C}_{\text{det}}(T_{\text{max}}), \quad \inf_{G \in U_{1,N}} \|X - G\| > 5, \quad \inf_{G \in U_{1,N}} |\sin \gamma| \geq 10^{-12}. \quad (5)$$

若不存在另一可行点在两项目标上均不劣且至少一项严格更优, 则 X 属于 Pareto 集 $\mathcal{P}$ 。本文先保留 $\mathcal{P}$, 再按“最小化 $\hat{F}_{1,N}$, 相同时最小化 f_2 ”作字典序选择, 避免把米与秒用任意权重相加。有限样本只负责搜索; 对最终站址再在连续 U_1 上作区间认证。

6.4 第二次观测后的清除证书

第二次正常示向度返回后, 不能用式 (4) 的代理直接决定清除。应重新构造

$$P_{\text{exact}}^{(2)} = \{G : \|G\| \leq R_0, G \in W_1 \cap W_2, 5 < \|G - S_i\| \leq 1500 \ (i = 1, 2)\}.$$

其闭凸外包由目标圆、两个测向锥和两个 1500 m 圆盘相交得到, 再显式删除两个 5 m 近距盘; 直径按上确界、最小覆盖圆按闭包计算。存在同一清除点覆盖所有可能位置的充要条件为

$$R_{\text{MEC}}(\overline{P_{\text{exact}}^{(2)}}) \leq 20 \text{ m}. \quad (6)$$

若第二次检测返回“距离过近”, 则直接进入近距清除分支; 若区域为空, 则报告几何不相容而非零误差。

6.5 算例与结果分析

取 $S_1 = (0, 0)$ 、 $\hat{\theta} = 25^\circ$ 、 $\delta = 1^\circ$ 、 $T_{\text{max}} = 260 \text{ s}$ 。此时 $U_1 = \{\rho d(25^\circ + \eta) : 5 < \rho \leq 1500, |\eta| \leq 1^\circ\}$, 圆域不截断扇形。在局部坐标网格 $a = 0, 10, \dots, 1500$ 、 $b = -800, -790, \dots, 800$ 上, 条件保证区保留 12093 点, 保守核心区保留 4464 点, Pareto 前沿含 1154 点。字典序选得

$$S_2 = (993.738585, -143.469943) \text{ m} = S_1 + 840d_{\parallel} - 550d_{\perp}.$$

表 5 问题二参考站址的数值结果

指标	结果
第二次检测局部耗时 $f_2(S_2)$	205.808366 s
完整先验上的最大站源距离	999.911130 m
连续域 $F_1(S_2)$ 认证区间	$[60.8760257892, 60.8769672571]$ m
加密样本下最坏 GDOP	2707.663746 m/rad
有限样本面积代理 $\hat{A}_N$	$1.16 \times 10^4 \text{ m}^2$
连续先验的最小站源距离	535.26 m > 5 m
连续先验的最小交会正弦下界	> 0.53

对推荐点采用 85 个区间分支包围连续先验，认证区间宽度小于 10^{-3} m ；最大站源距离则由式 (1) 的端点一驻点枚举直接取得，故“保证接收”不是有限采样推断。图 ?? 给出条件保证候选区、精度一耗时非支配前沿与参考站址。

参考站址方向与首次示向锥的最小线夹角保守取 $\gamma_0 \geq 32.21^\circ$ 。利用 $[G - S_1, X - G] = [G - S_1, X - S_1]$ 、 $\|X - S_1\| > 1004.04 \text{ m}$ 以及 $\|X - G\| \leq 999.92 \text{ m}$ ，对全部 $G \in U_1$ 有

$$|\sin \gamma| = \frac{\|X - S_1\| \sin \gamma_0}{\|X - G\|} \geq \frac{1004.04}{999.92} \sin 32.21^\circ > 0.53,$$

从而表 ?? 中的非退化下界也建立在连续先验上，而非有限源点样本上。

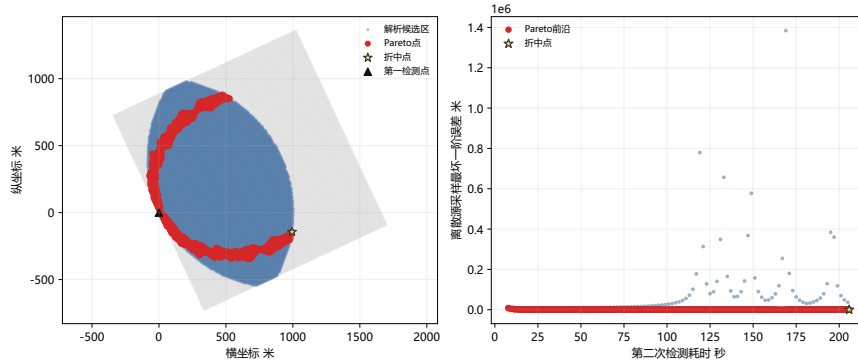


图 2 第二检测点条件保证候选区、Pareto 前沿与参考站址（程序图中的“解析候选区”“折中点”分别对应本文的条件保证区与参考站址）

参考点同时属于 C_{1000} ，但距 1000 m 边界仅约 0.09 m，部署时须保留额外模型余量。这里的“最优”只指给定搜索网格和字典序规则下的候选最优；区间算法认证的是该站址的连续先验最坏误差，而非连续站址域的全局最优。又因 $F_1 \approx 60.88 \text{ m}$ 与实际区域的 MEC 半径不是同一量，是否能够清除仍必须等待第二次示向度并检验式 (6)。

七、问题三的建立与求解

7.1 模型准备

7.1.1 动作协议与时间模型

机器狗的位置和当前频道由相邻动作的位置、频道参数唯一确定，求解过程只需维护该状态。为避免网络重试重复计入，重试沿用原 request_id 与载荷，并同时校验 HTTP 状态和响应中的 accepted 字段；在线算法记录剩余现实时间并保留截止余量。

7.1.2 几何证据与清除证书

对频道 c 在测站 S 的一次测量结果作如下解释：

1. **direction**: 返回正常示向度 $\hat{\theta}_c(S)$ ，用于生成示向锥；
2. **near**: 测站与目标的距离不超过 5m，可直接在测站清除；
3. **no_signal**: 当前测站不在该源的有效接收范围内，只能作为阴性证据，不能单独证明频道为空。

频道 c 的可行定位区域记为 K_c 。在线算法以多边形外包围 A_c 代替精确圆弧边界，并保持

$$K_c \subseteq A_c. \quad (1)$$

利用问题一的半平面裁剪算法更新 A_c 。设 C_c 为 A_c 的覆盖圆心，定义保守半径

$$\rho_c = \max_{X \in A_c} \|X - C_c\|_2. \quad (2)$$

由于欧氏范数是凸函数，其在凸多边形上的最大值必在顶点取得。若

$$\rho_c \leq r_{\text{opt}} - \epsilon_{\text{num}}, \quad \epsilon_{\text{num}} = 0.001\text{m}, \quad (3)$$

则任取 $G \in K_c$ ，均有 $\|G - C_c\|_2 \leq \rho_c < 20\text{m}$ ，故在 C_c 处执行的清除动作具有严格几何证书。若为衔接路径而偏移清除点，则仅当该点到 A_c 全部顶点的最远距离仍不超过 19.999m 时执行；由 $K_c \subseteq A_c$ ，此时对真实源的距离证书仍然成立。若检测结果直接为 **near**，则在当前测站清除。除这两种情形外，不执行试探性清除。

7.1.3 频道状态与终止条件

每个频道维护一个状态，其含义如表 ?? 所示。

表 6 问题三频道状态及进入条件

状态	含义	进入条件
unknown	尚未发现目标，也尚未认证为空	初始状态
localizing	已发现目标，正在收缩定位区域	已获得正常示向度，但尚未达到清除条件
clear_ready	当前外包围已满足光学清除半径	$\rho_c \leq 19.999\text{m}$
cleared	已成功清除	/clear 返回成功
absent_certified	已证明频道内不存在干扰源	七站均实测无信号或已有严格推断，或已发现 16 源
geometry_fault	定位区域为空或认证后的动作无法闭合证书	空区域、认证清除失败或有限覆盖仍未闭合等异常

终止判据要求，仅当所有频道均已清除或认证为空时，才可调用一次 /exit。若仍存在未发现频道、定位未闭合频道或几何故障状态，则任务尚未闭合，不能报告为完成。

7.2 模型建立

7.2.1 七点扫描覆盖与发现证书

取

$$P_0 = (0, 0), \quad P_k = 1200 \left(\cos \frac{k\pi}{3}, \sin \frac{k\pi}{3} \right), \quad k = 1, \dots, 6. \quad (4)$$

七点集合记为 $\mathcal{S}$ 。将目标圆域按六个 60° 扇区划分，第 k 个扇区以 P_k 为中心方向。对扇区内任一点 G ，记 $r = \|G\|_2$ ，其相对 P_k 的夹角为 $\varphi \in [-30^\circ, 30^\circ]$ 。由于构型关于每个扇区的中线对称，只需分析 $0 \leq \varphi \leq 30^\circ$ ；六个扇区除旋转外完全相同。

原点与 P_k 到 G 的距离相等当且仅当

$$r = \frac{600}{\cos \varphi}. \quad (5)$$

若 $r \leq 600/\cos \varphi$ ，则到原点的距离不超过

$$\frac{600}{\cos 30^\circ} = 400\sqrt{3} \approx 692.82\text{m}. \quad (6)$$

若 $r \geq 600/\cos \varphi$ ，则到 P_k 的平方距离关于 r 是凸二次函数，其在区间 $[600/\cos \varphi, 1800]$ 的端点取得最大值。径向左端点处两扫描点到 G 的距离相等且不超过 692.82m；目标圆域外端点对应的距离为

$$f(\varphi) = \sqrt{1800^2 + 1200^2 - 2 \cdot 1800 \cdot 1200 \cos \varphi}. \quad (7)$$

$f(\varphi)$ 在 $0 \leq \varphi \leq 30^\circ$ 上单调增加，故

$$d_{\max} = f(30^\circ) = \sqrt{1800^2 + 1200^2 - 2 \cdot 1800 \cdot 1200 \cos 30^\circ} = 968.9015717\text{m}. \quad (8)$$

因此，目标圆域内任一点到 $\mathcal{S}$ 中最近扫描点的距离不超过 $d_{\max}$ ；图 ?? 给出了扫描点、接收圆盘与最坏边界位置的关系。

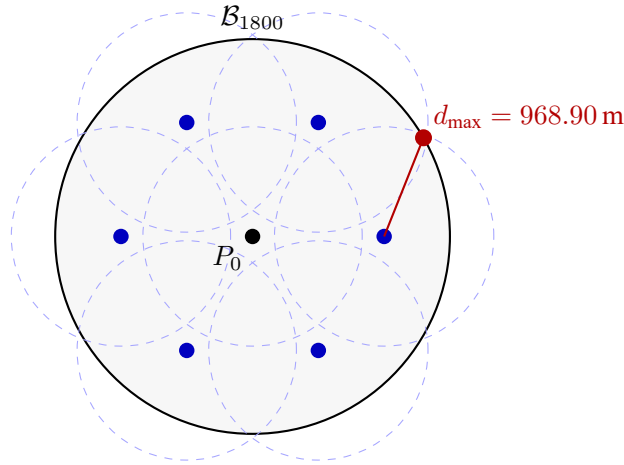


图 3 问题三七点扫描骨架、接收覆盖与最坏位置

命题 1 (发现完备性) 若 $1000\text{m} \leq r_j^{\text{recv}} \leq 1500\text{m}$ ，则每个干扰源在七个扫描点中至少一个点上会被检测到；若某频道在七个扫描点上均返回 `no_signal`，则该频道必不存在干扰源。

证明：由 $d_{\max} < 1000\text{m}$ ，任一干扰源到最近扫描点的距离不超过 968.9016m，位于其有效接收范围内。由于问题三中的干扰源为全向源，该点不会出现方向盲区，因此测量结果只能是 `direction` 或 `near`。反之，若七个点均无信号，则与上述必然有信号矛盾。证毕。

七个扫描点构成发现证书，其正确性只依赖扫描点的集合，与访问顺序无关。因此，后续为缩短路径而重排扫描站不会削弱该证书。

若某频道已经获得一次有效响应，则记为“已发现源”；同一频道的多次观测只对应一个源。当已发现不同频道数达到 16 时，根据干扰源总数不超过 16 以及频道唯一性，剩余频道必为空，可直接认证为 `absent_certified`。该结论可缩短扫描过程，且不需要额外的阴性观测。

7.2.2 阴性证据推理

设 Q 是频道 c 的一个无信号站， S 是尚未测量但可接收该源的候选站，并记该源的有效接收半径为 r^{recv} 。若目标 G 在 S 处可接收而在 Q 处不可接收，则

$$\|G - S\|_2 \leq r^{\text{recv}} < \|G - Q\|_2. \quad (9)$$

平方后可得

$$(Q - S)^T G < \frac{\|Q\|_2^2 - \|S\|_2^2}{2}. \quad (10)$$

右端等于 $(Q - S)^T(Q + S)/2$ ，因此上式定义一个开半平面。将其与目标圆域、距离上界以及其余无信号站产生的不等式求交，若交集为空，则候选站 S 不可能接收该源，扫描时可直接跳过并记为推断无信号。

另一种阴性证据来自圆盘覆盖：设候选区域被若干已测得无信号站周围半径 1000m 的圆盘覆盖。若源位于该区域，则它到某个无信号站的距离不超过 $1000\text{m} \leq r^{\text{recv}}$ ，该站本应收到信号，与事实矛盾。因此，只要连续区域的圆盘覆盖关系成立，即可安全跳过相应测量。推断阴性只用于更新认证状态，不记录为实际测量结果。

7.2.3 保证有信号的第二测站

设首次正常示向度为 $\hat{\theta}$ ，其单位方向为 $d = d(\hat{\theta})$ ，左法向量为 $d^\perp = d^\perp(\hat{\theta})$ 。取

$$S' = S + 400d + 300d^\perp. \quad (11)$$

该点与首点的距离为 500m，相对首条视线的横向偏移为 300m。

设首点测得方向时源距为 r ，真实方向与名义方向的夹角为 ε_θ ，满足 $|\varepsilon_\theta| \leq \delta_{\text{num}}$ 。位移向量 $w = S' - S = 400d + 300d^\perp$ ，令 $a = w \cdot d(\hat{\theta} + \varepsilon_\theta)$ ，则

$$a = 400 \cos \varepsilon_\theta + 300 \sin \varepsilon_\theta \geq 400 \cos \delta_{\text{num}} - 300 \sin \delta_{\text{num}} > 394.6\text{m}. \quad (12)$$

因此

$$\|G - S'\|_2^2 = r^2 + 500^2 - 2ra \triangleq \Phi(r). \quad (13)$$

$\Phi(r)$ 是开口向上的二次函数。若 $r \geq 1000\text{m}$ ，则由 $a > 394.6\text{m}$ 得 $250000 - 2ar < 0$ ，故 $\|G - S'\|_2 < r \leq r^{\text{recv}}$ 。若 $r < 1000\text{m}$ ，则 Φ 在 $[0, 1000]$ 上的最大值必在端点取得，而

$$\Phi(0) = 500^2 < 1000^2, \quad \Phi(1000) = 1000^2 + 500^2 - 2000a < 1000^2.$$

因此 $\|G - S'\|_2 < 1000\text{m} \leq r^{\text{recv}}$ 。综上， S' 必能收到该源的有效响应。该构造只改变测量位置，不增加误差分布假设。

7.2.4 定位区域更新与测站选择

初始化频道 c 的外包围为覆盖目标圆域的正多边形。获得正常示向度后，依次执行：

1. 用 δ_{num} 生成示向锥 $W(S, \hat{\theta}_c)$ 的两条半平面；
2. 与外包围 A_c 求交；
3. 用 $\|G - S\|_2 \leq 1500\text{m}$ 的圆域支撑半平面继续裁剪；
4. 对每个无信号站 $Q \in \mathcal{D}_c$ ，用半平面 $R(Q, S)$ 排除不可能位置；
5. 重新计算外包围的覆盖圆心 C_c 和保守半径 ρ_c 。

若 A_c 为空, 则报告观测矛盾; 若 $\rho_c \leq 19.999\text{m}$, 则转入清除; 否则继续选择新测站。正常测站候选满足以下条件:

1. 具有接收保证: 候选点到当前外包围中任一点的距离均小于 999.99m , 对应数学上的严格接收保证, 并保留 0.01m 的数值余量; 或候选点到该外包围中每一点的距离都严格小于某个既有成功观测站到相应点的距离, 从而利用首次成功接收所给出的接收半径下界;
2. 候选点到当前外包围的最远距离不超过 1499.99m , 从而除非目标距候选点不超过 5m 并转入 **near** 清除, 否则返回的方向必为有效 **direction**;
3. 名义方位预评估显示其能使定位半径下降;
4. 当 $\rho_c \leq 450\text{m}$ 时, 对可能返回的方位按不超过 2° 分箱, 并对每个分箱扩大误差半角后计算裁剪区域半径上界; 优先选择上界不超过 19.5m 或小于当前半径 75% 的测站。若不存在这样的改进候选, 则回到基础候选规则, 不把启发式上界误当成可行性硬约束。

最后一项使用误差上界而非名义方位做决策, 避免因未知返回值导致测站选择乐观失效。所有候选步骤仍以问题一的半平面交和最小覆盖圆为几何内核。

当 $\rho_c \leq 300\text{m}$ 时, 采用十二点环形测站包:

$$a_c = 1.2\rho_c + 8, \quad S_i = C_c + a_c (\cos \psi_i, \sin \psi_i), \quad \psi_i = \psi_0 + \frac{i\pi}{6}. \quad (14)$$

任一测站到可能源的距离不超过

$$a_c + \rho_c = 2.2\rho_c + 8 \leq 668\text{m}, \quad (15)$$

故十二个测站均有信号保证。十二个方位均匀分布的测站用于改善交会几何, 是否继续访问仍由保守半径的实际收缩判定。每轮最多访问十二个环点; 一旦半径降至本轮初始值的 55% 以下或达到清除阈值, 本轮提前结束。外层在 $\rho_c \leq 300\text{m}$ 时最多重复 8 轮, 每轮均按更新后的 C_c 和 ρ_c 重建测站包; 若仍不足以达到清除阈值, 则转入有限覆盖兜底。

7.2.5 有限覆盖清除与完备性

当快速定位分支停滞时, 对当前外包围 A_c 进行递归细分。每次沿投影跨度较大的方向将单元二分, 直到每个叶单元的保守半径不超过

$$\varepsilon_{\text{cov}} = 4.5\text{m}. \quad (16)$$

对单元中心 S 执行一次检测。若真实源位于该单元, 则

$$\|G - S\|_2 \leq \varepsilon_{\text{cov}} < 5\text{m}, \quad (17)$$

测量结果必为 **near**, 随后在该点执行光学清除。保证测站若返回 **no_signal**, 说明该观测与接收保证证书冲突; 算法不会将任务直接判为成功, 而是结束当前快速分支并进入本有限覆盖过程。只有有限覆盖完成后仍未取得合法清除证书时, 才报告几何故障。

覆盖细分只保留与当前外包围仍有交集的单元, 而真实源始终属于当前外包围, 因此真实源所在叶单元不会被丢弃。单元半径按全部顶点到覆盖圆心的最大距离保守计算; 每次对当前较大投影跨度作二分, 随着单元尺寸收缩, 有限步内即可达到 4.5m 。该分支不依赖空清除; 若数值退化导致单元覆盖无法推进, 算法报告几何故障, 而不是假定清除成功。

综上, 在本章假设下, 每个实际存在的全向干扰源都能被发现并最终获得合法清除路径; 每个未发现频道也都有实测 **no_signal**、严格阴性推断或 16 源上界之一作为认证依据。因此, 终止时不会出现未闭合频道。

7.2.6 联合路径调度模型

在时间表达式中，单次检测、换频和清除的时间固定，主要可控项为总移动距离 L 。设当前重规划时刻的必选任务节点集为

$$V_t = V_t^{\text{scan}} \cup V_t^{\text{service}}, \quad (18)$$

其中 V_t^{scan} 为尚未完成必要阴性认证的扫描站， V_t^{service} 为已完成发现但仍需定位或清除的频道服务点。后者以该频道当前的保守覆盖圆心 C_c 作为规划位置。对由 n_t 个节点组成的访问顺序 $\sigma = (\sigma_1, \dots, \sigma_{n_t})$ ，开放路径长度定义为

$$L(\sigma) = \|P_{\sigma_1} - P_{\text{cur}}\|_2 + \sum_{i=1}^{n_t-1} \|P_{\sigma_{i+1}} - P_{\sigma_i}\|_2, \quad (19)$$

其中 P_{cur} 为当前机器狗位置，开放路径不要求返回起点。当前节点集上的调度子问题为

$$\min_{\sigma} L(\sigma). \quad (20)$$

节点数不超过 10 时，采用 Held-Karp 动态规划求精确开放路径。令 d_{ij} 为节点间距离，状态 $F_{\text{path}}(M, j)$ 表示从当前点出发访问集合 M 且最后位于 j 的最短距离，则

$$F_{\text{path}}(M, j) = \min_{i \in M \setminus \{j\}} [F_{\text{path}}(M \setminus \{j\}, i) + d_{ij}], \quad F_{\text{path}}(\{j\}, j) = d_{\text{cur}, j}. \quad (21)$$

节点数超过 10 时，计算量显著上升，故先以最小增量插入构造初始解，并加入 3 个最近邻起点，再交替执行 2-opt 逆向交换和 Or-opt 邻域重定位直至无改善。此时只声称得到高质量可行路径，不声称是全局最优解。

在每个扫描站内部，除必需的未知频道外，还可在联合调度预测成本较低条件下，顺势补测已发现且尚未达到清除精度的频道。仅当预期定位半径明显下降，且预计节省的往返时间超过本次测量与换频成本时，才将该测量加入站内任务；否则留给后续必经站处理。该“顺路收获”不改变动作次数的下限，但可减少为同一频道多次往返。

每完成一个联合路径节点（一次扫描站访问或一个频道的服务过程）后，在线算法重新更新频道状态、删除已闭合的扫描站或服务点，并再次求解当前节点集上的开放路径。动态重规划使后续路径能够利用新增方位和清除结果，但不改变七点发现证书。

7.2.7 算法流程

问题三的在线求解过程如下：

1. 调用 `/enter`，记录剩余现实时间并设置截止保护；
2. 初始化七个扫描站和 20 个频道的状态；
3. 利用阴性证据推理退役不可能收到信号的扫描站；若已发现 16 个源，则认证其余频道为空；
4. 将未退役扫描站和待服务频道组成动态节点集，求联合开放路径；
5. 若下一节点为扫描站，则测量尚未闭合的频道，并在有严格收益时执行站内顺路收获；
6. 若下一节点为服务频道，则按保证信号测站、误差上界测站和十二点测站包的顺序继续定位；
7. 当外包围覆盖半径不超过 19.999m 时，在满足距离证书的保守覆盖圆内选择清除点；若检测为 `near`，就地清除；
8. 若正常定位分支停滞，或保证测站出现与证书矛盾的 `no_signal`，则细分当前外包围至半径不超过 4.5m，在叶单元中心检测；所测叶单元中心与真实源的距离不超过 4.5m，必然触发 `near` 清除；
9. 删去已清除与已认证为空的频道，重复路径规划和执行，直到 20 个频道均有确定状态；
10. 调用一次 `/exit`，输出清除数、检测次数、换频次数、总虚拟时间和证书状态。

7.3 模型求解与结果分析

7.3.1 计算参数

在线计算采用 $\delta_{\text{num}} = 1.01^\circ$ 作为安全半角。目标圆域初始外包围采用 360 条支撑半平面，距离圆域采用 128 条支撑半平面，清除半径使用 19.999m 的保守阈值。七点发现骨架的最大覆盖距离为 968.9016m，距有效接收半径下界仍有

$$1000 - 968.9016 = 31.0984\text{m}$$

(22)

的余量。该余量保证了在最不利接收半径下，扫描骨架仍能产生阴性认证。所有几何判断均使用外包围和保守半径，因而数值误差不会使真实源被排除在可行区域之外。

7.3.2 演练效果

使用提交版程序的最近三次演练共清除 39 个干扰源，清除失败 0 次，平均定位清除时间为 268.62s，最好与最坏分别为 255.35s 和 293.50s。该批演练均正常退出。

此前 15 局快速版迭代演练共清除 201 个源，平均时间为 262.23s，中位数为 259.59s，范围为 197.43–325.96s，空清除次数为 0。该批日志中在线决策模块的哈希前 16 位为 2704db2749dd7700；因其与提交版不同，只作迭代前后对照，不与正式版样本合并统计。

早期版本的官方演练中，立即调度 18 局的平均每源时间为 507.91s；批量调度 7 局的平均每源时间为 418.52s，较前者降低 17.60%。

进一步引入联合选路、站内顺路收获、带误差上界的局部测站和十二点测站包后，三次正式测试单局平均值的算术平均降至 289.91s/源。该对照说明时间收益主要来自减少重复往返和无效测站，而不是减少必要的几何认证。

7.3.3 正式测试结果

三次正式测试均正常退出，且与最近三次提交版演练使用同一组四文件哈希，其中在线决策模块的 SHA-256 前 16 位为 d5cf45ddcf5065f5。结果如表 ?? 所示。

表 7 问题三正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间/s	程序运行时间/ms
T5JT-TE93-CQUC-FDAD	12	384.94	2401
FX3P-PUBX-QE4N-783K	13	220.50	2198
Y4HS-EJMD-CXRP-BV5W	15	264.31	2404
合计（三次按局平均）	40	289.91	2198–2404

表中汇总行的 40 为三次清除数之和；289.91s 为三次单局平均定位清除时间的等权算术平均，按日志未舍入值计算得到。若改用总体合并口径，则三次总虚拟时间与总清除数之比为 $11450.33/40 = 286.26\text{s/源}$ 。两种口径含义不同，本章按三次单局等权汇总，采用前者。三次正式测试共清除 40 个干扰源，三次均无空清除。每局的 20 个频道最终均为已清除或认证为空状态，没有未知频道残留。由命题 1 的发现完备性，所有被判为不存在的频道必无干扰源；因此三次测试的清除个数均等于实际源数，被清除干扰源个数比例为 100%。

三次正式测试合计移动距离为 46.49km，虚拟总时间之和为 11450.33s。时间构成如表 ?? 所示。

移动时间占比超过八成，说明路径长度仍是总时间的主导因素。正式测试第 1 局的平均时间较高，主要原因是该局干扰源分布较分散，移动里程为 19.41km，移动时间占总时间的 84.0%；另两局

表 8 问题三三次正式测试的合计时间构成

时间项	数值/s	占比
移动时间	9297.33	81.20%
检测时间	1640.00	14.32%
换频时间	313.00	2.73%
成功清除时间	200.00	1.75%
失败清除时间	0.00	0.00%
合计	11450.33	100.00%

的移动时间占比分别为 76.3% 和 81.4%。这与演练批次的统计一致，表明差异主要来自案例空间分布。

程序运行时间保持在 2198 至 2404ms，远小于 1200s 的现实运行时上限。依据三局正式日志中的动作序列逐项复核：清除数与清除成功次数一致，检测次数和换频次数均可由动作序列复算；按四项时间分量独立重算的总时间与日志记录的虚拟时间残差不超过 $3.1 \times 10^{-6}s$ 。因此，正式测试的数值结果与动作序列及时间统计一致。

从模型层面看，问题三的求解没有依赖干扰源真值或有效接收半径的先验观测。发现阶段依靠七点覆盖证书，定位阶段依靠外包围与严格清除判据，调度阶段只利用已经发生的位置、频道和测量结果。在题目给定场景及已完成测试样本中，结果支持“覆盖认证优先、批量定位清除、动态联合选路”策略的可行性与可复现性；但这组实验本身不构成对全部随机案例最优性的证明。

八、问题四的模型建立与求解

8.1 模型准备

8.1.1 定向可见性与检测结果语义

设源 G 的定向发射方向为单位向量 $\mathbf{o}$ ，即 $\mathbf{o}$ 从源指向其有效覆盖区域的中心方向。对测点 S ，检测点相对源的位置向量为 $S - G$ 。定向源的可见条件为

$$\mathbf{o}^T(S - G) \geq 0. \quad (23)$$

该不等式表示测点落在以定向方向为中心的闭半平面内，恰好对应“定向方向两侧各 90° 且包含边界”的题面规则。于是检测结果可以统一写为

$$\text{result}(S, c) = \begin{cases} \text{near}, & \|G - S\|_2 \leq 5, \quad \mathbf{o}^T(S - G) \geq 0, \\ \text{direction}, & 5 < \|G - S\|_2 \leq r^{\text{recv}}, \quad \mathbf{o}^T(S - G) \geq 0, \\ \text{no_signal}, & \text{其他情形}. \end{cases} \quad (24)$$

其中 **direction** 还满足示向误差约束

$$\left| \text{wrap}_{180}(\hat{\theta}_c(S) - \theta(S, G)) \right| \leq \delta, \quad (25)$$

$\text{wrap}_{180}(\cdot)$ 将角差化为 $[-180^\circ, 180^\circ)$ 内的有符号角。上式说明：**no_signal** 可能由三种原因产生，即频道不存在、距离超过接收半径，或测点位于定向源背面。问题四不得把第三种原因误判为前

两种。

8.1.2 正证据更新与状态机

当频道 c 在某测点获得正常示向度时，先把该频道的可行区域多边形外包围 A_c 初始化为目标圆域的外逼近多边形，再依次与示向锥 $W(S, \hat{\theta}_c)$ 求交，并保留接收距离不超过 1500m 的部分。若得到多个正常示向度，则继续执行凸多边形裁剪 [?]。所有裁剪均使用 1.01° 安全半角，保证真实可行集始终包含在数值区域中。

由于 `no_signal` 在问题四中具有方向歧义，问题三中“在已成功观测点与无信号点之间比较距离”的阴性半平面推理不能直接沿用。问题四只在定向源分支使用正常示向度作为位置约束，不在未知定向方向下用 `no_signal` 收缩可行区域。该处理牺牲了部分可利用信息，但避免了由盲区产生的错误排除。

每个频道维护六类状态：未发现的 `unknown`，定位中的 `localizing`，待清除的 `clear_ready`，已清除的 `cleared`，确认不存在的 `absent_certified`，以及几何故障 `geometry_fault`。频道可在以下三种情形下结束：清除指令返回成功；该频道在全部扫描站均无正响应，由方向完备扫描证书证明无源；或已发现的不同频道数达到 16，此时由题设 $N_J \leq 16$ 直接判定其余频道不存在。只有 20 个频道全部进入已清除或确认不存在状态时，任务才满足终止条件。

8.1.3 时间模型与优化目标

设路径依次访问位置 $P_0, P_1, \dots, P_M$ ，则问题四的虚拟时间为

$$T = \sum_{k=1}^M \frac{\|P_k - P_{k-1}\|_2}{5} + 5N_{\text{measure}} + N_{\text{switch}} + 5N_{\text{clear}}^{\text{success}} + 3N_{\text{clear}}^{\text{miss}}. \quad (26)$$

式中各动作严格串行，清除动作不改变当前测向频道。

由于单次检测、换频和清除的耗时固定，而移动时间与总路径长度成正比，在线优化的主要目标是减少重复往返。每完成一个联合节点后，算法删除已闭合的扫描站或服务点，更新各频道的可行区域，并重新求解当前节点集上的开放路径。该动态重规划不改变扫描点集，因此不会削弱发现证书。

8.2 模型建立

8.2.1 方向完备发现判据

对固定位置 G ，记位于接收半径下界内的全部扫描点相对向量为

$$V_G = \{P_j - G : P_j \in P_{\text{scan}}, \|P_j - G\|_2 \leq 1000\}, \quad (27)$$

其中 P_{scan} 可以为基准扫描集 P_4 或提交版本的 P'_4 。由于真实接收半径满足 $r^{\text{recv}} \geq 1000\text{m}$ ， V_G 中每个扫描点都必定位于真实接收范围内。

定理 1 (方向完备发现判据) 若对目标圆域内每个可能位置 G ，均满足

$$0 \in \text{conv } V_G, \quad (28)$$

则无论定向方向如何，源 G 都至少会在一个扫描点产生有效响应。反之，若某个 G 不满足上式，则存在一个定向方向和接收半径 $r^{\text{recv}} = 1000\text{m}$ ，使该源对所有扫描点均返回 `no_signal`。

证明：先证充分性。设 $0 \in \text{conv } V_G$ ，任取单位发射方向 $\mathbf{o}$ 。若所有 $v \in V_G$ 都满足 $\mathbf{o}^\top v < 0$ ，则因 0 可写成 V_G 的凸组合，取组合系数 $\lambda_v \geq 0$ ($\sum_{v \in V_G} \lambda_v = 1$)，有

$$0 = \mathbf{o}^\top 0 = \sum_{v \in V_G} \lambda_v \mathbf{o}^\top v < 0,$$

产生矛盾。因此至少存在一个 $v = P - G \in V_G$, 使

$$\mathbf{o}^\top(P - G) \geq 0, \quad \|P - G\|_2 \leq 1000 \leq r^{\text{recv}}.$$

该扫描点位于定向源正面且处于有效接收范围内, 故必返回 **direction** 或 **near**。

再证必要性。若 $V_G = \emptyset$, 则接收半径下界内不存在可用扫描点, 该源对任意方向都不可见, 结论成立。以下设 $V_G \neq \emptyset$: 若 $0 \notin \text{conv } V_G$, 由有限点集凸包的闭性与严格分离定理, 存在单位向量 $\mathbf{o}$ 和常数 $\alpha < 0$, 使

$$\mathbf{o}^\top v \leq \alpha < 0, \quad \forall v \in V_G.$$

取该方向为定向源方向, 并令 $r^{\text{recv}} = 1000\text{m}$ 。此时所有位于接收半径内的扫描点都严格位于盲区, 半径外的扫描点也不可能收到信号, 因此全部测量均为 **no_signal**。定理得证。

扫描集构造只依赖目标圆域和接收半径下界。完整扫描后仍无正响应的频道可认证为无源; 累计发现 16 个不同频道后, 由 $N_J \leq 16$ 可直接将其余频道标记为确认不存在。

8.2.2 基准 31 点等边三角格

为得到证明简明、可用于对照的基准构造, 取边长 $s = 990\text{m}$ 的等边三角格

$$e_1 = (s, 0), \quad e_2 = \left(\frac{s}{2}, \frac{\sqrt{3}s}{2}\right), \quad \Lambda_s = \{ie_1 + je_2 : i, j \in \mathbb{Z}\}. \quad (29)$$

扫描集取与目标圆域相交的基本三角形的顶点并集

$$P_4 = \{P \in \Lambda_s : \|P\|_2 \leq 1800 + s\}. \quad (30)$$

在式 (??) 给定的格点相位下, 式 (??) 枚举得到 31 个扫描点。对任意 $G \in \mathcal{B}_{1800}$, 存在一个基本三角形包含 G , 其三顶点均在 P_4 中, 到 G 的距离不超过 $s = 990\text{m}$, 且 G 是三个顶点的凸组合。因此

$$0 \in \text{conv } V_G, \quad (31)$$

该构造对任意朝向均至少可见, 且证明直接, 仅作为可核验基准, 不主张其点数全局最少。

8.2.3 22 点连续域认证扫描集

提交版本进一步采用 22 点紧凑扫描集。记

$$P'_4 = \{(0, 0)\} \cup \left\{975d \left(\frac{2\pi i}{7}\right) : i = 0, \dots, 6\right\} \cup \left\{1850d \left(\frac{2\pi j}{14}\right) : j = 0, \dots, 13\right\}. \quad (32)$$

该集合由原点、半径 975m 的内环 7 点和半径 1850m 的外环 14 点组成, 合计 22 点, 其空间结构如图 ?? 所示。

由于 22 点集合不是完整三角格, 其方向完备性采用连续域递归证书, 而不是随机采样验证。首先用目标圆域的外接正 72 边形覆盖 $\mathcal{B}_{1800}$, 并划分为 72 个三角形。对任一三角形单元 T , 定义可用扫描点集合

$$E(T) = \left\{P \in P'_4 : \max_{X \in \text{vert}(T)} \|P - X\|_2 \leq 994.99\right\}. \quad (33)$$

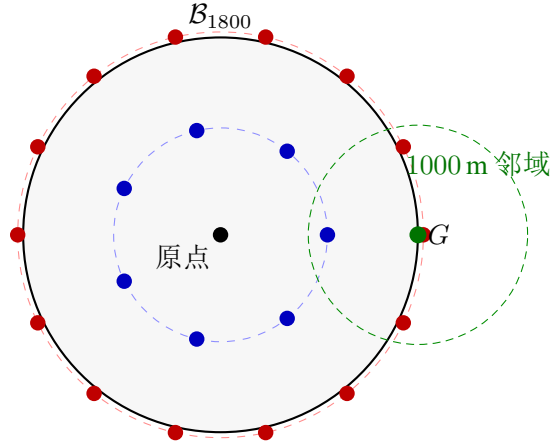


图 4 22 点紧凑扫描集及一个边界位置的接收邻域示意

若 $T \subseteq \text{conv } E(T)$, 则该单元通过认证。事实上, 三个顶点都在凸包内意味着整个三角形都在凸包内; 又因欧氏范数是凸函数, 距离在每个三角形上的最大值必在某个顶点取得, 所以 T 中任意点 G 到任意 $P \in E(T)$ 的距离均不超过 $994.99\text{m} < 1000\text{m}$, 从而

$$0 \in \text{conv}\{P - G : P \in E(T)\} \subseteq \text{conv } V_G. \quad (34)$$

若单元尚未通过, 则沿最长边递归二分; 程序设置最大深度 22 和最多 60000 个单元。程序核验表明, 该集合在 7372 个单元内完成认证; 一旦达到限制仍未完成, 程序拒绝证书并回退到 25 点双环解析覆盖集, 不在未证明的扫描集上继续执行。

8.2.4 定向源的定位与正证据收缩

对频道 c 的首次正常示向度, 初始化

$$K_c = \mathcal{B}_{1800} \cap W(S_0, \hat{\theta}_c) \cap \{G : 5 < \|G - S_0\|_2 \leq 1500\}. \quad (35)$$

数值实现使用覆盖 K_c 的外逼近多边形 A_c , 并始终保持

$$K_c \subseteq A_c. \quad (36)$$

每次获得新的正常示向度后, 执行

$$A_c \leftarrow A_c \cap W(S, \hat{\theta}_c) \cap \{G : \|G - S\|_2 \leq 1500\}. \quad (37)$$

设 C_c 为 A_c 的覆盖圆心, 定义保守半径

$$\rho_c = \max_{X \in A_c} \|X - C_c\|_2. \quad (38)$$

由于欧氏范数是凸函数, 最大值在凸多边形顶点取得。若

$$\rho_c \leq r_{\text{opt}} - \epsilon_{\text{num}} = 19.999\text{m}, \quad (39)$$

则可行集内任意真实源到 C_c 的距离都小于 20m; 由于清除成功只取决于该距离, 在 C_c 处执行清除即可通过距离证书保证成功。

若继续获得的都是正常示向度, 上述正证据收缩与问题三一致。区别在于: 当后续测量返回 `no_signal` 时, 问题四不将其视为几何矛盾, 而是转入覆盖式清除分支。原因是首测点可能恰位

于定向源覆盖边界，且源距等于接收半径下界；此时对任一备选下一测站 $S \neq S_0$ ，都存在与首测读数相容的源位置和定向方向，使 S 落入背面；因此不能保证下一测站必然收到信号，也不能将后续 `no_signal` 解释为几何矛盾。

8.2.5 与朝向无关的覆盖式清除

设首次正常示向度为 $\hat{\theta}_0$ ，在线安全半角为 δ_{num} 。首次观测成功说明真实源位于扇形

$$\mathcal{R}_0 = \left\{ G = S_0 + \rho d(\phi) : 5 < \rho \leq 1500, \left| \text{wrap}_{180}(\phi - \hat{\theta}_0) \right| \leq \delta_{\text{num}} \right\}. \quad (40)$$

在 $\mathcal{R}_0$ 上铺设等边三角格，并只保留到 $\mathcal{R}_0$ 的距离不超过 ρ_{cov} 的格点。采用边长

$$s_c = 20\sqrt{3} < 35\text{m} \quad (41)$$

时，平面等边三角格对任意点的覆盖半径不超过 $s_c/\sqrt{3} = 20\text{m}$ 。数值实现将边长再乘以 0.995 的安全系数，得到

$$\rho_{\text{cov}} = 19.9\text{m}. \quad (42)$$

因此，对任意 $G \in \mathcal{R}_0$ ，存在一个格点 Q 满足

$$\|Q - G\|_2 \leq 19.9\text{m} < 20\text{m}. \quad (43)$$

清除是否成功与朝向无关，故有限格点序列中至少一次成功。该过程不依赖后续测向，以有限次移动和清除换取方向盲区下的完备性。

提交版本按证明任务使用三个半径：理论覆盖半径不超过 20m，数值实现按 0.995 安全系数取 19.9m；预算内分支以 19.5m 单元中心覆盖当前外包围 A_c ，余下预算足以遍历全部单元时必命中。19.5m 仅用于预算遍历证书，实际清除仍以接口距离检查确认。预算不足时，改用 12 点近距测站包收缩区域；进入最后单元时，要求该单元全部顶点到历史失败圆或当前站圆心的最远距离不超过 19.999m。真源不在失败圆内，故当前站圆必包含真源。因此，每局 5 次空清除只是代价控制，不是完成性前提。

8.2.6 发现、定位与清除的完备性

在总扫描阶段，若累计发现的不同频道数达到 16，则由 $N_J \leq 16$ 可知其余频道均无源；否则扫描继续执行。对任一实际存在且尚未被清除的干扰源，由定理 1 和 22 点连续域证书可知，至少有一个扫描点能够看到它。因此扫描不会遗漏仍可能存在的源，也不可能把“源存在但所有扫描点均无信号”的频道错误认证为空。

对已发现频道，若正常示向度持续给出并使 $\rho_c \leq 19.999\text{m}$ ，则由外包围包含关系直接得到清除距离证书。出现 `no_signal` 时，前述覆盖证书已保证有限次清除内至少成功一次；预算、12 点测站包和最后单元证明仅优化成本，不削弱该保证。若出现空区域、认证清除失败或覆盖证明无法闭合，算法进入 `geometry_fault`，不会把异常改写为“确认不存在”。

因此，终止时每个频道都已清除，或由方向完备证书或 16 个已发现频道的上界证明无源，不会留下 `unknown` 或 `localizing` 状态。

8.2.7 动态路径调度

设当前重规划时刻仍需访问的扫描站和待服务频道服务点组成节点集 V_t ，其节点数为 n_t ，节点 i 与 j 之间的距离记为 ℓ_{ij} 。对访问顺序 $\sigma = (\sigma_1, \dots, \sigma_{n_t})$ ，开放路径长度为

$$L(\sigma) = \ell_{\text{cur}, \sigma_1} + \sum_{i=1}^{n_t-1} \ell_{\sigma_i \sigma_{i+1}}, \quad (44)$$

其中 $\ell_{\text{cur}, i} = \|P_i - P_{\text{cur}}\|_2$ 为当前机器狗位置 P_{cur} 到节点 i 的欧氏距离，路径不要求返回起点。当前节点集上的调度问题为

$$\min_{\sigma} L(\sigma). \quad (45)$$

节点数不超过 10 时，使用 Held-Karp 动态规划求精确开放路径；节点数超过 10 时，先以最小增量插入构造初始解，再使用最近邻起点、2-opt 逆交换和 Or-opt 邻域重定位进行改善。后一种情形只报告高质量可行路径，不宣称全局最优。扫描点集合在规划过程中保持不变，因此重排访问顺序不会改变方向完备发现证书。

8.2.8 求解算法

问题四的在线求解过程如下：

1. 调用 `/enter` 并记录剩余现实时间，在接口开放后开始任务；
2. 构造 22 点扫描集并执行连续域方向完备认证；若认证失败则回退到解析覆盖集；
3. 将未完成扫描站和待定位、待清除服务点组成动态任务图，求解当前开放路径；
4. 按规划顺序访问扫描站，依次检测尚未闭合的频道；
5. 对返回 `direction` 的频道记录示向观测，对返回 `near` 的频道直接清除；
6. 每处理一次正响应后检查已发现的不同频道数；一旦达到 16，立即依据 $N_J \leq 16$ 将其余 `unknown` 频道标记为 `absent_certified` 并停止访问剩余扫描站；
7. 对已发现频道，用示向锥和外包围继续收缩；当保守半径不超过 19.999m 时按距离证书清除；
8. 若定位阶段返回 `no_signal`，优先使用预算内覆盖清除；预算不足时转入方向完备测试站包或带最后单元覆盖证明的清除；
9. 扫描阶段结束时，对所有仍未发现的频道检查方向完备证书，满足条件则标记为“确认不存在”；
10. 删除已清除和已认证为空的频道，重新规划路径，直到 20 个频道全部闭合；
11. 调用 `/exit`，按定位清除总时间与清除个数计算题面要求的统计值。

8.3 模型求解与结果分析

8.3.1 证书与演练校验

以下先给出提交版演练与旧版对照，再报告三次正式测试。

使用提交版程序的最近三次演练均正常退出，合计清除 42/42 个实际存在的干扰源，空清除 0 次，平均定位清除时间为 477.66s，最好与最坏分别为 420.34s 和 580.98s；其中 2 局访问 22 个扫描站，1 局在发现 16 个不同频道后依据 $N_J \leq 16$ 提前终止，实际访问 21 个扫描站。该批日志中在线决策模块的哈希前 16 位为 `d5cf45ddcf5065f5`，与三次正式测试一致。旧版在线模块的 15 局演练清除 198 个源，平均、中位数和范围分别为 499.11s、507.11s 和 351.95–619.72s，空清除 12 次。其哈希与提交版不同，仅作前后对照，不并入提交版统计。

三次正式测试均使用 22 点集完成连续域认证，未触发 25 点回退；18 局演练中，17 局访问 22 站，1 局发现 16 个频道后提前终止并访问 21 站，同样未回退。

8.3.2 正式测试结果

三次正式测试均正常退出，结果如表 ?? 所示。表中平均定位清除时间为模拟器的定位清除总时间除以该局清除个数；程序运行时间为模拟器记录的实际运行时间。

表 9 问题四正式测试结果

测试序号	测试案例编码	清除干扰源个数	平均定位清除时间/s	程序运行时间/ms
1	NGBV-4HQZ-7B7S-26HK	11	519.99	2426
2	JGN3-394M-SDZB-Y3N7	13	493.46	2364
3	9U2N-ZBZQ-8AB2-VJYS	15	374.53	1976
合计 / 均值		39	462.66	1976–2426

表中 462.66s 为逐局平均时间的等权算术平均；若按源数合并，则 $17752.83/39 = 455.20\text{s}/\text{个}$ 。正式结果表采用题面要求的逐局报告口径。三次合计空清除 2 次，均在 5 次预算内；每局 20 个频道最终都已清除或认证为空。由定理 1，认证为空的频道必无源，因此 39 个被清除频道均真实存在，清除比例均为 $\lambda_{\text{clr}} = 100\%$ ，题面要求的两个统计量均已给出。

8.3.3 时间构成与结果分析

三次正式测试合计移动距离为 67.22km，时间构成如表 ??所示。

表 10 问题四三次正式测试的合计时间构成

时间项	数值/s	占比
移动时间	13443.83	75.73%
检测时间	3470.00	19.55%
换频时间	638.00	3.59%
成功清除时间	195.00	1.10%
失败清除时间	6.00	0.03%
合计	17752.83	100.00%

移动时间占总时间的 75.73%，远高于其他项，说明任务时间主要由路径长度决定；三次移动占比分别为 73.32%、75.07% 和 78.92%，差异来自干扰源空间分布。动态开放路径整合扫描站与清除服务点并持续重规划，减少重复往返；22 点扫描保证发现完备性。

三次程序运行时间为 1.976–2.426s，远低于 1200s 上限，不构成约束。日志复核显示，时间项之和与虚拟时间的最大残差不超过 $2 \times 10^{-6}\text{s}$ ；扫描证书、频道状态和动作序列逐项一致，未出现非接受记录或重复站位。

该策略仅使用在线可得的频道号、位置、示向度和清除结果，不使用干扰源、朝向或接收半径真值；发现、定位和盲区清除分别由方向完备证书、外包围和与朝向无关的覆盖保证。结果支持其在题设和测试样本内有效；22 点数和覆盖参数属于工程实现选择，不据此声称对所有随机案例平均最优。

九、模型的检验

本文的主要结论属于确定性保证：数值实验可以发现实现错误，却不能替代覆盖证明与误差上界证明。因此，本节分别检验几何程序与数学对象的一致性、关键参数变化时证书是否仍成立，以及

在线动作日志能否由题设时间模型独立复算；有限采样只作为连续域证明之外的交叉检查。

9.1 几何求解的一致性与数值误差

对问题一，采用与主算法不同的逐顶点对暴力枚举检验旋转卡壳，二者得到的直径完全一致。将定位多边形全部顶点回代所有示向半平面，最大约束违反量为 $7.82 \times 10^{-14} \text{ m}$ ；正 720 边形内、外逼近给出的直径原始差小于 $4 \times 10^{-13} \text{ m}$ ，按保守浮点合并规则记为零。将全部输入整体平移到坐标量级 $2 \times 10^6 \text{ m}$ 后，直径仍保持不变； 10^{-3} m 量级的小尺度点集也能与暴力法一致。这些对照分别排除了卡壳漏掉共线对踵点、半平面容差不具平移不变性和固定绝对容差吞掉小尺度顶点三类错误。

圆域外接正 N 边形的最大径向超差为 $\eta_N = 1800(\sec(\pi/N) - 1)$ 。取 $N = 90, 180, 360$ 时，超差依次为 $1.09718, 0.27419, 0.06854 \text{ m}$ ；取 $N = 720, 1440$ 时则降为 $0.01713, 0.00428 \text{ m}$ ，边数加倍时约缩小为原来的四分之一。这说明 $N = 720$ 的径向离散误差已远小于 1° 示向误差造成的区域尺度；但本例直径上下界几乎重合是活动约束未落在圆边界上的特例，不能推广为任意测站构型下都无离散化误差。

对问题二，方位 Jacobian 先与中心有限差分对照，再以正交两站算例交叉验证矩阵式和闭式，二者均给出 5.000000 m/rad 。最终候选点的有限采样误差代理为 60.8760257892 m ；对完整连续源域作区间分支定界后得到

$$60.8760257892 \leq F_1(S_2) \leq 60.8769672571 \quad (\text{m}),$$

区间宽度仅 $9.41 \times 10^{-4} \text{ m}$ ，相对宽度为 1.55×10^{-5} 。采样值恰位于认证下端，表明加密采样没有漏掉数量级不同的极端位置；同时，全部保证区网格点的有限源采样下界均大于 20 m ，故“该场景、该候选网格和该两站一阶最坏误差指标下阈值不可达”成立。该检验不把面积代理等同于真实交会区域面积，也不证明任意初始测站和任意连续第二站址都不可能达到 20 m 。

9.2 参数灵敏度与证书边界

在问题一、二的同一验证场景中，将物理示向误差界 δ 从 0.5° 增至 1.5° ，得到表 ??。

表 11 示向误差界的灵敏度检验

$\delta/(^\circ)$	0.50	0.75	1.00	1.25	1.50
问题一直径 /m	22.0757	33.1115	44.1448	55.1746	66.2003
问题二保证区网格点数	4528	4500	4464	4436	4398

问题一直径关于 δ 的线性拟合斜率为 $44.1249 \text{ m}/(^\circ)$ ，五点最大拟合残差仅 0.00353 m ，说明本构型中直径几乎按比例放大；这不是全局闭式规律，活动半平面改变时只能保证直径随 δ 单调不减。问题二的保证区网格点数仅减少 2.87% ，说明候选区的存在性对该区间内的角误差较稳定；点数依赖 151×161 网格，不能据此声称连续面积只减少同一比例，也不能推出误差指标 F_1 不敏感。表中 δ 始终是题设物理误差界；问题三、四在线外包围另取 $\delta_{\text{num}} = 1.01^\circ$ ，多出的 0.01° 只承担示向度量化与浮点余量，不参与上述物理参数敏感性比较。

对问题三，将六个环点半径由 1200 m 推广为 a ，同一 Voronoi 分解给出

$$d_{\max}(a) = \max \left\{ \frac{a}{\sqrt{3}}, \sqrt{1800^2 + a^2 - 2 \cdot 1800a \cos 30^\circ} \right\}. \quad (46)$$

解不等式 $d_{\max}(a) \leq 1000$ 得

$$1122.9558 \leq a \leq 1732.0508 \quad (\text{m}).$$

当前 $a = 1200\text{ m}$ 时 $d_{\max} = 968.9016\text{ m}$, 保留 31.0984 m 的接收余量。若为缩短路径把半径降到 1100 m , 最坏距离将增至 1010.2175 m , 不存在证书立即失效。式 (??) 检验的是发现完备性, 不代表区间内任一半径都有相同的定位精度或路径代价。

对问题四, 22 点集合用外接正 72 边形递归剖分, 在 7372 个三角形单元上完成连续域认证。每个认证单元只使用到三个顶点距离均不超过 994.99 m 的扫描点, 因此相对题设接收半径下限保留 5.01 m 的严格余量。另以与递归剖分不同的“邻域向量最大极角间隙不超过 180° ”判据, 在 361 个半径层、每层 720 个方位上完成 259920 次网格判定, 违反数为 0, 最少邻域点数为 3; 该离散对照只用于排查实现错误, 连续保证仍由三角形凸包证书给出。逐一删除 22 个扫描点并重新认证, 22 次均未通过, 说明现有点集与证法对删点敏感; 认证失败不等价于真实判据失败, 因而不能据此宣称 22 是所有点集中的最小点数。

方向完备证明还明确给出了适用边界: 定向覆盖必须是含边界的 180° 半平面, 且有效接收半径下限不得低于认证所用的 994.99 m 。若覆盖角变窄或边界改为不含端点, $0 \in \text{conv } V_G$ 不再是充分条件; 若接收半径下限降至 994.99 m 以下, 现有距离证书也不再成立。

9.3 在线闭环的一致性与稳健性

以 36 局演练和 6 局正式测试的脱敏动作日志为对象, 从初始位置和频道出发, 仅根据每条请求的坐标、频道与返回值, 按题设的移动、检测、换频和清除耗时独立累加。联合核验结果见表 ??; 其中“重复”同时检查重复站位与同站同频道重复测量。

表 12 在线动作日志的独立一致性核验

证据集	局数	20 频道闭合	成功清除	空清除	非接受/重复
演练日志	36	36/36	480	12	0/0
正式日志	6	6/6	79	2	0/0

六次正式测试的清除、检测、换频、空清除次数与模拟器独立记账逐项一致; 独立重算的虚拟时间与日志值最大相差 $3.1 \times 10^{-6}\text{ s}$, 扩展到 42 局时最大相差 $3.6 \times 10^{-6}\text{ s}$ 。正式测试中, 问题三全部采用 7 点距离覆盖证书, 问题四全部采用 22 点方向完备证书; 每局均为 20/20 频道闭合。归档程序还对这 42 局进入后的 8043 条业务动作作了逐动作回放, 动作类型、坐标和频道一致。因此, 时间成绩、状态终止与实际动作之间不存在统计口径错位, 重试机制也未产生重复计时动作。

为区分案例波动与程序不稳定, 又按干扰源数对 18 局演练分别分层。只考察样本数不少于 2 的层, 问题三同层最大极差为 89.09 s/源 , 问题四为 132.99 s/源 ; 跨演练与正式批次共 21 局时, 总虚拟时间与移动里程的相关系数分别为 0.996 和 0.962。这表明主要波动随空间分布和清除节点路由而变, 而非由接口重发、重复测量或状态机漂移造成。但各源数层仅有 1–7 局, 且演练日志跨越两个冻结版本, 以上统计只能解释已观察波动, 不能估计所有随机案例的尾部概率。

综上, 数值对拍、连续域证书和官方记账三条证据链相互独立: 前者验证实现, 中者提供题设范围内的完备性, 后者验证闭环执行。结论仍受题设边界约束: 若源在任务中移动、示向误差超过规定区间、问题三接收半径低于七点覆盖距离、问题四覆盖角或接收半径突破上述边界, 现有保证均需重新建立; 路由结果是动态开放路径上的高质量可行解, 也不构成全局最短性证明。

十、模型评价、改进与推广

10.1 模型评价

主要优点如下:

1. **保证口径完整。**问题一、二始终在 $\pm 1^\circ$ 误差盒和未知接收半径上取确定性最坏情形；问题三用距离覆盖认证全向源不存在，问题四用连续域凸包条件消除定向盲区三义性，均不以有限随机采样替代完备证书。
2. **几何结论可核验。**问题一同时给出圆域内外包、半平面残差、卡壳—暴力一致性和严格题内反例；问题二区分核心保证区与条件保证区，并对推荐站址的连续最坏误差给出宽度小于 10^{-3} m 的区间认证。
3. **精度与效率分层。**一阶误差传播和 GDOP 只用于观测前排序，观测后仍以实际可行集的最小覆盖圆作清除证书，避免把代理误当作物理阈值。在线算法则把扫描、顺路观测和清除节点统一规划，减少无效往返。
4. **实测闭合。**冻结版本的三次正式测试中，问题三清除 40 个源、空清除 0 次，平均每源耗时 289.91 s；问题四清除 39 个源、空清除 2 次，平均每源耗时 462.66 s。各局 20 个频道均取得“已清除”或“确认不存在”证书，程序运行时间均小于 2.5 s。

与替代做法的比较：若把首次示向度简化为名义射线，虽然可以把第二检测点选择降为一维问题，却会遗漏角误差锥内部的真实位置；若只按交会角接近正交选点，又会忽略测站距离对 GDOP 的放大作用。在线阶段，有限随机扫描或规定次数的多方位复测能够降低平均动作数，但不能排除问题三的覆盖空洞和问题四的统一背向反例。本文采用“连续可行集、确定性证书、代价路由”三层结构，计算与路径开销较大，但每一项效率优化都不改变发现和清除的正确性条件。

局限性如下：

1. 题面未给示向误差分布，确定性模型不能利用概率平均效应；其代价是结果偏保守，但换取了无需额外统计假设的最坏情形保证。
2. 问题二的推荐点是给定网格和字典序规则下的候选最优，连续认证只证明该点的指标界，不证明连续站址域的全局最优；其最大接收距离距边界约 0.09 m，对题面外的传播扰动较敏感。
3. 六次正式测试足以验证接口闭合和案例内正确性，却不足以精确估计随机案例总体的均值与尾部风险；效率结论还需结合多局演练分布解释。

10.2 改进与推广

在不改变确定性证书的前提下，可用区间分支与支撑函数补齐一般圆域截断下的完整条件保证区，并以连续优化替代问题二的固定网格；在线部分可继续把认证站、观测站和清除点统一为带证据约束的开放路径问题，采用滚动时域与局部精确路由压缩移动常数项。测向锥交、连续覆盖证书和证据约束路由也可迁移到无人机搜救、移动传感器部署及其他无源定位任务，只需替换观测集合与动作代价。

AI 工具使用声明

本参赛队在竞赛过程中使用了 GPT-5.6 sol，仅用于论文语言润色与 LaTeX 排版、部分文献与建模方法的初步查找，以及官方评测器 HTTP 交互接口的代码编写与调试。AI 未参与算法设计、数学推导、核心几何程序或问题三、四主体智能体代码的完成，也未用于生成或改写测试数据与结论；最终方法、算法、主体代码、数值计算和结论均由参赛队独立完成并人工复核。详细使用情况见支撑材料《AI 工具使用详情》。

参考文献

- [1] 王凯. 无线电测向定位方法在新型无线电干扰查找中的运用研究[D]. 南昌: 南昌大学, 2019.
- [2] SUTHERLAND I E, HODGMAN G W. Reentrant polygon clipping[J/OL]. Communications of the ACM, 1974, 17(1):32-42. DOI: 10.1145/360767.360802.
- [3] TOUSSAINT G T. Applications of the rotating calipers to geometric problems in two and three dimensions[J/OL]. International Journal of Digital Information and Wireless Communications, 2014, 4(3): 372-386. DOI: 10.17781/P001290.
- [4] JUNG H. Ueber die kleinste kugel, die eine räumliche figur einschliesst[J/OL]. Journal für die reine und angewandte Mathematik, 1901, 1901(123):241-257. DOI: 10.1515/crll.1901.123.241.
- [5] BISHOP A N, FIDAN B, ANDERSON B D O, et al. Optimality analysis of sensor-target localization geometries[J/OL]. Automatica, 2010, 46(3):479-492. DOI: 10.1016/j.automatica.2009.12.003.

附录 A 支撑材料文件清单

支撑材料与本文的模型、结果和正式测试一一对应，文件清单如下。

1. **行为日志**：问题三与问题四各三份正式测试原始日志，共 6 份，文件名保持模拟器导出形式不变：
 - formal-p3-1-T5JT-TE93-CQUC-FDAD.jlog;
 - formal-p3-2-FX3P-PUBX-QE4N-783K.jlog;
 - formal-p3-3-Y4HS-EJMD-CXRP-BV5W.jlog;
 - formal-p4-1-NGBV-4HQZ-7B7S-26HK.jlog;
 - formal-p4-2-JGN3-394M-SDZB-Y3N7.jlog;
 - formal-p4-3-9U2N-ZBZQ-8AB2-VJYS.jlog。
2. **完整程序**：
 - geometry_solver.py;
 - robot/__init__.py、robot/client.py;
 - robot/q3_agent.py、robot/q4_agent.py、robot/main.py;
 - robot/geometry_solver_fast.py、robot/q3_agent_fast.py;
 - robot/q4_agent_fast.py、robot/main_fast.py，以及运行说明 robot/README.md。
3. **结果数据**：6 份正式测试脱敏动作日志、36 份演练脱敏动作日志，问题一定位多边形与角误差敏感性数据，问题二 Pareto 候选点与候选区敏感性数据，问题三调度对照数据，以及汇总文件 geometry_summary.json。
4. **AI 工具使用说明**：文件 AI工具使用详情.pdf 记录所用工具、具体用途、主要提示方式以及人工核验过程，与正文声明一致。

其中，正式测试原始日志用于满足题目规定的测试记录要求；脱敏动作日志用于复核频道、坐标、动作结果和虚拟时间；完整程序与下节打印的源码一致。

附录 B 完整源程序代码

根据格式规范第五条，本节打印建模和正式测试程序的完整源码，而非仅列核心函数片段。共收录 10 个 Python 文件、3110 行代码；问题三、四的提交版入口为 robot/main_fast.py，其依赖的基础数据结构、客户端和几何模块一并完整列出。所有队号均由命令行参数或环境变量在运行时传入，源码不含参赛者身份信息。

B.1 问题一、二共用几何与统计模块

```
1 from __future__ import annotations
2
3 import math
4 import heapq
5 from collections.abc import Sequence
6
7 import numpy as np
8
9 Arr = np.ndarray
10 eps = 1e-10
11 rank_rel_tol = 1e-12
12 sin_angle_tol = 1e-12
```

```

13
14 def _relative_tol(value: float, name: str = "tol") -> float:
15     if not math.isfinite(value) or value < 0.0:
16         raise ValueError(f"{name} must be finite and nonnegative")
17     return float(value)
18
19 def _point(x: Sequence[float] | Arr, name: str) -> Arr:
20     p = np.asarray(x, dtype=float)
21     if p.shape != (2,) or not np.all(np.isfinite(p)):
22         raise ValueError(f"{name} must be a finite point")
23     return p
24
25 def _pts(x: Sequence[Sequence[float]] | Arr) -> Arr:
26     p = np.asarray(x, dtype=float)
27     if p.size == 0:
28         return np.empty((0, 2), dtype=float)
29     if p.ndim != 2 or p.shape[1] != 2:
30         raise ValueError("points must have shape (n, 2)")
31     if not np.all(np.isfinite(p)):
32         raise ValueError("points must be finite")
33     return p
34
35 def cross(a: Sequence[float] | Arr, b: Sequence[float] | Arr) -> float:
36     x = np.asarray(a, dtype=float)
37     y = np.asarray(b, dtype=float)
38     return float(x[0] * y[1] - x[1] * y[0])
39
40 def convex_hull(x: Sequence[Sequence[float]] | Arr, tol: float = eps) ->
    Arr:
41     p = _pts(x)
42     tol = _relative_tol(tol)
43     if len(p) <= 1:
44         return p.copy()
45     p = np.unique(p, axis=0)
46     p = p[np.lexsort((p[:, 1], p[:, 0]))]
47
48     span = max(float(np.ptp(p[:, 0])), float(np.ptp(p[:, 1])),
49                 np.finfo(float).tiny)
50     area_tol = tol * span * span
51
52     def build(a: Arr) -> list[Arr]:
53         s: list[Arr] = []
54         for q in a:
55             while len(s) >= 2 and cross(s[-1] - s[-2], q - s[-1]) <= area_tol:
56                 s.pop()
57             s.append(q)
58     return s

```

```

59
60     if len(p) <= 2:
61         return p.copy()
62     lo = build(p)
63     hi = build(p[::-1])
64     return np.asarray(lo[:-1] + hi[:-1], dtype=float)
65
66 def regular_bound(rad: float = 1800.0, n: int = 720,
67                 center: Sequence[float] = (0.0, 0.0),
68                 outer: bool = True) -> Arr:
69     if (not math.isfinite(rad) or rad <= 0 or not isinstance(n, int)
70         or isinstance(n, bool) or n < 3):
71         raise ValueError("rad must be positive and n must be at least 3")
72     c = _point(center, "center")
73     r = rad / math.cos(math.pi / n) if outer else rad
74     off = math.pi / n if outer else 0.0
75     ang = off + np.arange(n) * 2.0 * math.pi / n
76     return c + r * np.column_stack((np.cos(ang), np.sin(ang)))
77
78 def bearing_halfplanes(site: Sequence[float], bearing_deg: float,
79                       err_deg: float = 1.0) -> Arr:
80     if (not math.isfinite(bearing_deg) or not math.isfinite(err_deg)
81         or not 0 <= err_deg < 90):
82         raise ValueError("err_deg must be in [0, 90)")
83     s = _point(site, "site")
84     lo = math.radians(bearing_deg - err_deg)
85     hi = math.radians(bearing_deg + err_deg)
86     dl = np.array([math.cos(lo), math.sin(lo)])
87     dh = np.array([math.cos(hi), math.sin(hi)])
88     h1 = np.array([dl[1], -dl[0], dl[1] * s[0] - dl[0] * s[1]])
89     h2 = np.array([-dh[1], dh[0], -dh[1] * s[0] + dh[0] * s[1]])
90     return np.vstack((h1, h2))
91
92 def _clean_poly(x: Arr, tol: float) -> Arr:
93     if len(x) == 0:
94         return np.empty((0, 2), dtype=float)
95     span = max(float(np.ptp(x[:, 0])), float(np.ptp(x[:, 1])),
96                np.finfo(float).tiny)
97     cut = tol * span
98     out = [x[0]]
99     for p in x[1:]:
100         if np.linalg.norm(p - out[-1]) > cut:
101             out.append(p)
102     if len(out) > 1 and np.linalg.norm(out[0] - out[-1]) <= cut:
103         out.pop()
104     return np.asarray(out, dtype=float)
105

```

```

106 def clip_halfplane(poly: Sequence[Sequence[float]] | Arr,
107                    hp: Sequence[float] | Arr, tol: float = eps) -> Arr:
108     p = _pts(poly)
109     tol = _relative_tol(tol)
110     h = np.asarray(hp, dtype=float)
111     norm = float(np.linalg.norm(h[:2])) if h.shape == (3,) else 0.0
112     if (h.shape != (3,) or not np.all(np.isfinite(h))
113         or norm <= np.finfo(float).tiny):
114         raise ValueError("half-plane must be [a, b, c] with nonzero normal")
115     if len(p) == 0:
116         return p
117     org = p.mean(axis=0)
118     nrm = h[:2] / norm
119     cut = float((h[2] - h[:2] @ org) / norm)
120     q = p - org
121     span = max(float(np.ptp(q[:, 0])), float(np.ptp(q[:, 1])),
122                np.finfo(float).tiny)
123     roundoff = 32.0 * np.finfo(float).eps * max(
124         float(np.linalg.norm(org)), abs(cut), span)
125     cut_tol = tol * span + roundoff
126     out: list[Arr] = []
127     for i, a in enumerate(q):
128         b = q[(i + 1) % len(q)]
129         fa = float(nrm @ a - cut)
130         fb = float(nrm @ b - cut)
131         ina, inb = fa <= cut_tol, fb <= cut_tol
132         if ina:
133             out.append(a)
134         if ina != inb:
135             den = fa - fb
136             den_tol = 32.0 * np.finfo(float).eps * max(
137                 abs(fa), abs(fb), span)
138             if abs(den) > den_tol:
139                 out.append(a + fa / den * (b - a))
140     if not out:
141         return np.empty((0, 2), dtype=float)
142     return _clean_poly(np.asarray(out, dtype=float) + org, tol)
143
144 def halfplane_intersection(hps: Sequence[Sequence[float]] | Arr,
145                           bound: Sequence[Sequence[float]] | Arr,
146                           tol: float = eps) -> Arr:
147     h = np.asarray(hps, dtype=float)
148     tol = _relative_tol(tol)
149     if h.size == 0:
150         return _pts(bound).copy()
151     if h.ndim != 2 or h.shape[1] != 3 or not np.all(np.isfinite(h)):
152         raise ValueError("hps must have shape (n, 3)")

```

```

153 p = _pts(bound).copy()
154 for row in h:
155     p = clip_halfplane(p, row, tol)
156     if len(p) == 0:
157         break
158 return p
159
160 def localization_polygon(sites: Sequence[Sequence[float]] | Arr,
161                         bearings_deg: Sequence[float] | Arr,
162                         err_deg: float = 1.0, rad: float = 1800.0,
163                         n_bound: int = 720,
164                         center: Sequence[float] = (0.0, 0.0),
165                         outer: bool = True) -> tuple[Arr, Arr]:
166     s = _pts(sites)
167     ang = np.asarray(bearings_deg, dtype=float)
168     if ang.shape != (len(s),) or not np.all(np.isfinite(ang)):
169         raise ValueError("one bearing is required for each site")
170     if len(s) == 0:
171         raise ValueError("at least one bearing is required")
172     c = _point(center, "center")
173     local_sites = s - c
174     local_hps = np.vstack([
175         bearing_halfplanes(p, a, err_deg) for p, a in zip(local_sites, ang)
176     ])
177     local_poly = halfplane_intersection(
178         local_hps, regular_bound(rad, n_bound, outer=outer))
179     global_hps = np.vstack([
180         bearing_halfplanes(p, a, err_deg) for p, a in zip(s, ang)
181     ])
182     return local_poly + c, global_hps
183
184 def localization_diameter_bounds(
185     sites: Sequence[Sequence[float]] | Arr,
186     bearings_deg: Sequence[float] | Arr,
187     err_deg: float = 1.0, rad: float = 1800.0,
188     n_bound: int = 720,
189     center: Sequence[float] = (0.0, 0.0),
190 ) -> dict[str, Arr | float]:
191     outer_poly, hps = localization_polygon(
192         sites, bearings_deg, err_deg, rad, n_bound, center, outer=True)
193     if len(outer_poly) == 0:
194         raise ValueError("bearing constraints have empty intersection")
195     inner_poly, _ = localization_polygon(
196         sites, bearings_deg, err_deg, rad, n_bound, center, outer=False)
197     upper, upper_pair = polygon_diameter(outer_poly)
198     if len(inner_poly):
199         lower, lower_pair = polygon_diameter(inner_poly)

```

```

200     if lower > upper:
201         slack = eps * max(lower, upper, np.finfo(float).tiny)
202         if lower - upper > slack:
203             raise RuntimeError("inner diameter exceeds outer diameter")
204         lower = upper
205     else:
206         lower = 0.0
207         lower_pair = np.empty((0, 2), dtype=float)
208     return {
209         "inner_poly": inner_poly,
210         "outer_poly": outer_poly,
211         "halfplanes": hps,
212         "lower_m": lower,
213         "upper_m": upper,
214         "lower_pair": lower_pair,
215         "upper_pair": upper_pair,
216     }
217
218 def polygon_diameter(x: Sequence[Sequence[float]] | Arr,
219                     tol: float = eps) -> tuple[float, Arr]:
220     raw = _pts(x)
221     tol = _relative_tol(tol)
222     if len(raw) == 0:
223         raise ValueError("diameter is undefined for an empty set")
224     org = raw.mean(axis=0)
225     p = convex_hull(raw - org, tol)
226     n = len(p)
227     if n == 1:
228         return 0.0, np.vstack((p[0], p[0])) + org
229     if n == 2:
230         return float(np.linalg.norm(p[1] - p[0])), p.copy() + org
231     span = max(float(np.ptp(p[:, 0])), float(np.ptp(p[:, 1])),
232                np.finfo(float).tiny)
233     area_tol = tol * span * span
234     dist2_tol = tol * span * span
235     j = 1
236     best = -1.0
237     pair = np.vstack((p[0], p[1]))
238
239     def upd(a: int, b: int) -> None:
240         nonlocal best, pair
241         val = float((p[a] - p[b]) @ (p[a] - p[b]))
242         if val > best + dist2_tol:
243             best = val
244             pair = np.vstack((p[a], p[b]))
245
246     for i in range(n):

```

```

247     ni = (i + 1) % n
248     while True:
249         nj = (j + 1) % n
250         cur = abs(cross(p[ni] - p[i], p[j] - p[i]))
251         nxt = abs(cross(p[ni] - p[i], p[nj] - p[i]))
252         if nxt > cur + area_tol:
253             j = nj
254         else:
255             break
256     upd(i, j)
257     upd(ni, j)
258     nj = (j + 1) % n
259     cur = abs(cross(p[ni] - p[i], p[j] - p[i]))
260     nxt = abs(cross(p[ni] - p[i], p[nj] - p[i]))
261     if abs(nxt - cur) <= area_tol:
262         upd(i, nj)
263         upd(ni, nj)
264     return math.sqrt(max(best, 0.0)), pair + org
265
266 def diameter_circle_coverage(x: Sequence[Sequence[float]] | Arr,
267                             pair: Sequence[Sequence[float]] | Arr,
268                             tol: float = eps) -> tuple[bool, Arr, float
269                                     , float]:
270     p = _pts(x)
271     if len(p) == 0:
272         raise ValueError("coverage is undefined for an empty set")
273     q = _pts(pair)
274     if len(q) != 2:
275         raise ValueError("pair must contain two points")
276     c = q.mean(axis=0)
277     r = float(np.linalg.norm(q[1] - q[0])) / 2.0
278     far = float(np.max(np.linalg.norm(p - c, axis=1)))
279     return far <= r + tol, c, r, far
280
281 def _circle3(a: Arr, b: Arr, c: Arr, tol: float) -> tuple[Arr, float] |
282     None:
283     ab = b - a
284     ac = c - a
285     d = 2.0 * cross(ab, ac)
286     span = max(float(np.linalg.norm(ab)), float(np.linalg.norm(ac)),
287               np.finfo(float).tiny)
288     if abs(d) <= tol * span * span:
289         return None
290     ab2 = float(ab @ ab)
291     ac2 = float(ac @ ac)
292     off = np.array([
293         (ab2 * ac[1] - ac2 * ab[1]) / d,

```

```

292     (ab[0] * ac2 - ac[0] * ab2) / d,
293 ])
294 cen = a + off
295 return cen, float(np.linalg.norm(cen - a))
296
297 def minimum_enclosing_circle(
298     x: Sequence[Sequence[float]] | Arr,
299     tol: float = eps,
300 ) -> tuple[Arr, float]:
301     p = _pts(x)
302     tol = _relative_tol(tol)
303     if len(p) == 0:
304         raise ValueError("minimum circle is undefined for an empty set")
305     org = p.mean(axis=0)
306     span = max(float(np.ptp(p[:, 0])), float(np.ptp(p[:, 1])),
307                np.finfo(float).tiny)
308     cut = tol * span
309     order = np.random.default_rng(0).permutation(len(p))
310     q = p[order] - org
311     cen = q[0].copy()
312     rad = 0.0
313     for i in range(len(q)):
314         if np.linalg.norm(q[i] - cen) <= rad + cut:
315             continue
316         cen = q[i].copy()
317         rad = 0.0
318         for j in range(i):
319             if np.linalg.norm(q[j] - cen) <= rad + cut:
320                 continue
321             cen = (q[i] + q[j]) / 2.0
322             rad = float(np.linalg.norm(q[i] - q[j])) / 2.0
323             for k in range(j):
324                 if np.linalg.norm(q[k] - cen) <= rad + cut:
325                     continue
326                 cir = _circle3(q[i], q[j], q[k], tol)
327                 if cir is None:
328                     tri = np.vstack((q[i], q[j], q[k]))
329                     _, pair = polygon_diameter(tri)
330                     cen = pair.mean(axis=0)
331                     rad = float(np.linalg.norm(pair[1] - pair[0])) / 2.0
332                 else:
333                     cen, rad = cir
334     return cen + org, rad
335
336 def bearing_jacobian(target: Sequence[float],
337                      sites: Sequence[Sequence[float]] | Arr) -> Arr:
338     g = _point(target, "target")

```

```

339 s = _pts(sites)
340 if len(s) == 0:
341     raise ValueError("at least one site is required")
342 d = g - s
343 r2 = np.sum(d * d, axis=1)
344 if np.any(r2 <= np.finfo(float).tiny):
345     raise ValueError("target and site must be distinct")
346 return np.column_stack((-d[:, 1] / r2, d[:, 0] / r2))
347
348 def gdop(target: Sequence[float], sites: Sequence[Sequence[float]] | Arr
    ,
349         tol: float = rank_rel_tol) -> float:
350     tol = _relative_tol(tol)
351     h = bearing_jacobian(target, sites)
352     info = h.T @ h
353     val = np.linalg.eigvalsh(info)
354     if val[-1] <= 0.0 or val[0] <= tol * val[-1]:
355         return math.inf
356     cov = np.linalg.solve(info, np.eye(2))
357     return math.sqrt(float(np.trace(cov)))
358
359 def error_propagation(target: Sequence[float],
360                      sites: Sequence[Sequence[float]] | Arr,
361                      sig_deg: float | Sequence[float] = 1.0 / math.sqrt
    (3.0),
362                      tol: float = rank_rel_tol) -> dict[str, Arr |
    float]:
363     tol = _relative_tol(tol)
364     h = bearing_jacobian(target, sites)
365     sig = np.asarray(sig_deg, dtype=float)
366     if sig.ndim == 0:
367         sig = np.full(len(h), float(sig))
368     if (sig.shape != (len(h),) or not np.all(np.isfinite(sig))
369         or np.any(sig <= 0)):
370         raise ValueError("sig_deg must be positive for each site")
371     sig = np.radians(sig)
372     r = np.diag(sig * sig)
373     w = np.diag(1.0 / (sig * sig))
374     info = h.T @ w @ h
375     val = np.linalg.eigvalsh(info)
376     if val[-1] <= 0.0 or val[0] <= tol * val[-1]:
377         raise ValueError("bearing geometry is singular")
378     cov = np.linalg.solve(info, np.eye(2))
379     gain = np.linalg.solve(info, h.T @ w)
380     return {
381         "h": h,
382         "r": r,

```

```

383     "gain": gain,
384     "cov": cov,
385     "gdop": gdop(target, sites, tol),
386     "rms": math.sqrt(float(np.trace(cov))),
387 }
388
389 def intersection_angle(target: Sequence[float], site1: Sequence[float],
390                       site2: Sequence[float]) -> float:
391     g = _point(target, "target")
392     a = _point(site1, "site1") - g
393     b = _point(site2, "site2") - g
394     den = float(np.linalg.norm(a) * np.linalg.norm(b))
395     if den <= np.finfo(float).tiny:
396         raise ValueError("target and site must be distinct")
397     val = min(1.0, abs(float(a @ b)) / den)
398     return math.degrees(math.acos(val))
399
400 def two_site_gdop(target: Sequence[float], site1: Sequence[float],
401                  site2: Sequence[float],
402                  angle_tol: float = sin_angle_tol) -> float:
403     angle_tol = _relative_tol(angle_tol, "angle_tol")
404     g = _point(target, "target")
405     a = _point(site1, "site1") - g
406     b = _point(site2, "site2") - g
407     r1, r2 = float(np.linalg.norm(a)), float(np.linalg.norm(b))
408     if min(r1, r2) <= np.finfo(float).tiny:
409         raise ValueError("target and site must be distinct")
410     sn = abs(cross(a, b)) / (r1 * r2)
411     if sn <= angle_tol:
412         return math.inf
413     return math.sqrt(r1 * r1 + r2 * r2) / sn
414
415 def second_site_time(site1: Sequence[float], site2: Sequence[float],
416                     speed: float = 5.0, switched: bool = False,
417                     measure_s: float = 5.0, switch_s: float = 1.0) ->
418     float:
419     if (not math.isfinite(speed) or speed <= 0
420         or not math.isfinite(measure_s) or measure_s < 0
421         or not math.isfinite(switch_s) or switch_s < 0):
422         raise ValueError("time parameters must be nonnegative and speed
423     positive")
424     d = _point(site2, "site2") - _point(site1, "site1")
425     return float(np.linalg.norm(d)) / speed + measure_s + switch_s *
426     switched
427
428 def nominal_range_bounds(site: Sequence[float], bearing_deg: float,
429                          target_rad: float = 1800.0,

```

```

427         recv_max: float = 1500.0,
428         near: float = 5.0) -> tuple[float, float]:
429     s = np.asarray(site, dtype=float)
430     ang = math.radians(bearing_deg)
431     u = np.array([math.cos(ang), math.sin(ang)])
432     su = float(s @ u)
433     det = su * su + target_rad * target_rad - float(s @ s)
434     if det < 0:
435         raise ValueError("bearing ray misses target disk")
436     lo = max(near, -su - math.sqrt(det), 0.0)
437     hi = min(recv_max, -su + math.sqrt(det))
438     if hi <= lo:
439         raise ValueError("no feasible nominal target range")
440     return lo, hi
441
442 def candidate_region_mask(cand: Sequence[Sequence[float]] | Arr,
443                          site1: Sequence[float], bearing_deg: float,
444                          ranges: tuple[float, float], angle_tol_deg:
445                          float,
446                          recv_rad: float = 1000.0,
447                          time_limit_s: float = math.inf,
448                          switched: bool = False, near: float = 5.0,
449                          target_rad: float | None = None,
450                          speed: float = 5.0,
451                          measure_s: float = 5.0,
452                          switch_s: float = 1.0) -> Arr:
453     p = _pts(cand)
454     s = np.asarray(site1, dtype=float)
455     lo, hi = ranges
456     if not 0 <= lo <= hi or not 0 < angle_tol_deg < 90:
457         raise ValueError("invalid ranges or angle tolerance")
458     ang = math.radians(bearing_deg)
459     u = np.array([math.cos(ang), math.sin(ang)])
460     v = np.array([-math.sin(ang), math.cos(ang)])
461     d = p - s
462     a, b = d @ u, d @ v
463     dev = np.maximum(np.abs(a - lo), np.abs(a - hi))
464     ok = dev <= np.abs(b) * math.tan(math.radians(angle_tol_deg)) + eps
465     far2 = np.maximum((a - lo) ** 2 + b * b, (a - hi) ** 2 + b * b)
466     ok &= far2 <= recv_rad * recv_rad + eps
467     q = np.clip(a, lo, hi)
468     near2 = (a - q) ** 2 + b * b
469     ok &= near2 > near * near + eps
470     if speed <= 0 or measure_s < 0 or switch_s < 0:
471         raise ValueError("time parameters must be nonnegative and speed
472         positive")
473     t = (np.linalg.norm(d, axis=1) / speed + measure_s

```

```

472         + switch_s * float(swapped))
473 ok &= t <= time_limit_s + eps
474 if target_rad is not None:
475     ok &= np.linalg.norm(p, axis=1) <= target_rad + eps
476 return ok
477
478 def second_site_metrics(cand: Sequence[Sequence[float]] | Arr,
479                        site1: Sequence[float], bearing_deg: float,
480                        ranges: Sequence[float] | Arr,
481                        recv_rad: float = 1000.0, near: float = 5.0,
482                        quantile: float = 1.0,
483                        switched: bool = False) -> dict[str, Arr]:
484     p = _pts(cand)
485     s = np.asarray(site1, dtype=float)
486     rho = np.asarray(ranges, dtype=float)
487     if rho.ndim != 1 or len(rho) == 0 or np.any(rho <= 0):
488         raise ValueError("ranges must be a positive vector")
489     if not 0 <= quantile <= 1:
490         raise ValueError("quantile must be in [0, 1]")
491     ang = math.radians(bearing_deg)
492     u = np.array([math.cos(ang), math.sin(ang)])
493     g = s + rho[:, None] * u
494     d = p[:, None, :] - g[None, :, :]
495     r2 = np.linalg.norm(d, axis=2)
496     cs = np.abs(np.sum((-u)[None, None, :] * d, axis=2)) / np.maximum(r2,
497                               eps)
498     cs = np.clip(cs, 0.0, 1.0)
499     sn = np.sqrt(np.maximum(1.0 - cs * cs, 0.0))
500     gp = np.sqrt(rho[None, :] ** 2 + r2 * r2) / np.maximum(sn, eps)
501     valid = (r2 <= recv_rad + eps) & (r2 > near + eps)
502     cov = valid.mean(axis=1)
503     acc = np.quantile(cs, quantile, axis=1)
504     gq = np.quantile(gp, quantile, axis=1)
505     tm = np.asarray([second_site_time(s, q, switched=switched) for q in p
506                     ])
507     return {"angle_cost": acc, "gdop": gq, "time_s": tm, "coverage": cov}
508
509 def source_cone_samples(
510     site: Sequence[float], bearing_deg: float,
511     ranges: tuple[float, float] = (5.0, 1500.0),
512     err_deg: float = 1.0, target_rad: float = 1800.0,
513     n_range: int = 41, n_angle: int = 21,
514 ) -> Arr:
515     s = _point(site, "site")
516     lo, hi = ranges
517     if (not math.isfinite(bearing_deg) or not math.isfinite(lo)
518         or not math.isfinite(hi) or not math.isfinite(err_deg)

```

```

517     or not 0 <= lo < hi or not 0 <= err_deg < 90):
518     raise ValueError("invalid cone ranges or angle error")
519 if (not math.isfinite(target_rad) or target_rad <= 0
520     or not isinstance(n_range, int) or isinstance(n_range, bool)
521     or not isinstance(n_angle, int) or isinstance(n_angle, bool)
522     or n_range < 2 or n_angle < 2):
523     raise ValueError("invalid disk radius or sample counts")
524 offsets = np.unique(np.append(
525     np.linspace(-err_deg, err_deg, n_angle), 0.0))
526 angles = np.radians(bearing_deg + offsets)
527 samples = []
528 for angle in angles:
529     direction = np.array([math.cos(angle), math.sin(angle)])
530     projection = float(s @ direction)
531     discriminant = projection * projection + target_rad * target_rad -
532     float(s @ s)
533     if discriminant < 0.0:
534         continue
535     root = math.sqrt(max(discriminant, 0.0))
536     feasible_lo = max(lo, -projection - root)
537     feasible_hi = min(hi, -projection + root)
538     if feasible_lo > feasible_hi + eps:
539         continue
540     rho = np.linspace(feasible_lo, feasible_hi, n_range)
541     samples.append(s + rho[:, None] * direction)
542 if not samples:
543     return np.empty((0, 2), dtype=float)
544 return np.vstack(samples)
545
546 def sector_max_distance(
547     cand: Sequence[Sequence[float]] | Arr,
548     site: Sequence[float], bearing_deg: float,
549     ranges: tuple[float, float] = (5.0, 1500.0),
550     err_deg: float = 1.0,
551 ) -> Arr:
552     p = _pts(cand)
553     s = _point(site, "site")
554     lo, hi = ranges
555     if (not math.isfinite(bearing_deg) or not math.isfinite(lo)
556         or not math.isfinite(hi) or not math.isfinite(err_deg)
557         or not 0 <= lo <= hi or not 0 <= err_deg < 90):
558         raise ValueError("invalid cone ranges or angle error")
559     ang = math.radians(bearing_deg)
560     u = np.array([math.cos(ang), math.sin(ang)])
561     v = np.array([-math.sin(ang), math.cos(ang)])
562     d = p - s
563     a, b = d @ u, d @ v

```

```

563     err = math.radians(err_deg)
564     end_min = np.minimum(
565         a * math.cos(err) + b * math.sin(err),
566         a * math.cos(err) - b * math.sin(err),
567     )
568     phase = np.arctan2(b, a)
569     opp = (phase + math.pi + math.pi) % (2.0 * math.pi) - math.pi
570     dot_min = np.where(np.abs(opp) <= err + eps, -np.hypot(a, b), end_min)
571     q_lo = np.sum(d * d, axis=1) + lo * lo - 2.0 * lo * dot_min
572     q_hi = np.sum(d * d, axis=1) + hi * hi - 2.0 * hi * dot_min
573     return np.sqrt(np.maximum(np.maximum(q_lo, q_hi), 0.0))
574
575 def robust_candidate_mask(
576     cand: Sequence[Sequence[float]] | Arr,
577     site1: Sequence[float], bearing_deg: float,
578     ranges: tuple[float, float] = (5.0, 1500.0),
579     err_deg: float = 1.0, recv_rad: float = 1000.0,
580     time_limit_s: float = math.inf, speed: float = 5.0,
581     measure_s: float = 5.0, switched: bool = False,
582     switch_s: float = 1.0,
583 ) -> Arr:
584     p = _pts(cand)
585     s = _point(site1, "site1")
586     if (not math.isfinite(recv_rad) or recv_rad <= 0
587         or not math.isfinite(speed) or speed <= 0
588         or not math.isfinite(measure_s) or measure_s < 0
589         or not math.isfinite(switch_s) or switch_s < 0
590         or math.isnan(time_limit_s)):
591         raise ValueError("invalid reception or time parameters")
592     far = sector_max_distance(p, s, bearing_deg, ranges, err_deg)
593     t = (np.linalg.norm(p - s, axis=1) / speed + measure_s
594         + switch_s * float(switched))
595     return (far <= recv_rad + eps) & (t <= time_limit_s + eps)
596
597 def detected_candidate_mask(
598     cand: Sequence[Sequence[float]] | Arr,
599     site1: Sequence[float], bearing_deg: float,
600     ranges: tuple[float, float] = (5.0, 1500.0),
601     err_deg: float = 1.0, recv_min: float = 1000.0,
602     time_limit_s: float = math.inf, speed: float = 5.0,
603     measure_s: float = 5.0, switched: bool = False,
604     switch_s: float = 1.0,
605 ) -> Arr:
606     p = _pts(cand)
607     s = _point(site1, "site1")
608     lo, hi = ranges
609     if (not math.isfinite(recv_min) or recv_min <= 0

```

```

610     or not math.isfinite(speed) or speed <= 0
611     or not math.isfinite(measure_s) or measure_s < 0
612     or not math.isfinite(switch_s) or switch_s < 0
613     or math.isnan(time_limit_s)):
614     raise ValueError("invalid reception or time parameters")
615 if lo <= recv_min:
616     far = sector_max_distance(
617         p, s, bearing_deg, (lo, min(hi, recv_min)), err_deg)
618     reception = far <= recv_min + eps
619 else:
620     far = sector_max_distance(p, s, bearing_deg, (lo, lo), err_deg)
621     reception = far <= lo + eps
622 time_s = (np.linalg.norm(p - s, axis=1) / speed + measure_s
623           + switch_s * float(swached))
624 return reception & (time_s <= time_limit_s + eps)
625
626 def second_site_metrics_sources(
627     cand: Sequence[Sequence[float]] | Arr,
628     site1: Sequence[float], sources: Sequence[Sequence[float]] | Arr,
629     sig_deg: float = 1.0 / math.sqrt(3.0),
630     err_deg: float = 1.0,
631     recv_rad: float = 1000.0, quantile: float = 1.0,
632     speed: float = 5.0, measure_s: float = 5.0,
633     swached: bool = False, switch_s: float = 1.0,
634 ) -> dict[str, Arr]:
635     p = _pts(cand)
636     g = _pts(sources)
637     s = _point(site1, "site1")
638     if len(g) == 0 or not 0 <= quantile <= 1:
639         raise ValueError("sources must be nonempty and quantile valid")
640     if (not math.isfinite(sig_deg) or not math.isfinite(err_deg)
641         or sig_deg <= 0 or err_deg <= 0):
642         raise ValueError("angle scales must be finite and positive")
643     if (not math.isfinite(recv_rad) or recv_rad <= 0
644         or not math.isfinite(speed) or speed <= 0
645         or not math.isfinite(measure_s) or measure_s < 0
646         or not math.isfinite(switch_s) or switch_s < 0):
647         raise ValueError("invalid reception or time parameters")
648     first = s[None, :] - g
649     r1 = np.linalg.norm(first, axis=1)
650     second = p[:, None, :] - g[None, :, :]
651     r2 = np.linalg.norm(second, axis=2)
652     dot = np.sum(first[None, :, :] * second, axis=2)
653     den = r1[None, :] * r2
654     cos_val = np.abs(dot) / np.maximum(den, np.finfo(float).tiny)
655     cos_val = np.clip(cos_val, 0.0, 1.0)
656     cross_val = (first[None, :, 0] * second[:, :, 1]

```

```

657         - first[None, :, 1] * second[:, :, 0])
658 sin_val = np.abs(cross_val) / np.maximum(den, np.finfo(float).tiny)
659 sin_val = np.clip(sin_val, 0.0, 1.0)
660 unsafe = ((r1[None, :] <= np.finfo(float).tiny)
661           | (r2 <= np.finfo(float).tiny)
662           | (sin_val <= sin_angle_tol))
663 raw = np.sqrt(r1[None, :] ** 2 + r2 * r2) / np.where(
664     unsafe, math.nan, sin_val)
665 raw[unsafe] = math.inf
666 h1x = first[:, 1] / np.maximum(r1 * r1, np.finfo(float).tiny)
667 h1y = -first[:, 0] / np.maximum(r1 * r1, np.finfo(float).tiny)
668 h2x = second[:, :, 1] / np.maximum(r2 * r2, np.finfo(float).tiny)
669 h2y = -second[:, :, 0] / np.maximum(r2 * r2, np.finfo(float).tiny)
670 det = h1x[None, :] * h2y - h1y[None, :] * h2x
671 safe_det = np.where(unsafe, math.nan, det)
672 k1x = h2y / safe_det
673 k1y = -h2x / safe_det
674 k2x = -h1y[None, :] / safe_det
675 k2y = h1x[None, :] / safe_det
676 plus = np.hypot(k1x + k2x, k1y + k2y)
677 minus = np.hypot(k1x - k2x, k1y - k2y)
678 bound = math.radians(err_deg) * np.maximum(plus, minus)
679 bound[~np.isfinite(bound)] = math.inf
680 if quantile == 1.0:
681     ang_cost = np.max(cos_val, axis=1)
682     gdop_val = np.max(raw, axis=1)
683     sampled_error = np.max(bound, axis=1)
684 elif quantile == 0.0:
685     ang_cost = np.min(cos_val, axis=1)
686     gdop_val = np.min(raw, axis=1)
687     sampled_error = np.min(bound, axis=1)
688 else:
689     ang_cost = np.quantile(cos_val, quantile, axis=1)
690     gdop_val = np.quantile(raw, quantile, axis=1)
691     sampled_error = np.quantile(bound, quantile, axis=1)
692 rms = math.radians(sig_deg) * gdop_val
693 coverage = np.mean(r2 <= recv_rad + eps, axis=1)
694 detection_coverage = np.mean(
695     r2 <= np.maximum(recv_rad, r1)[None, :] + eps, axis=1)
696 time_s = (np.linalg.norm(p - s, axis=1) / speed + measure_s
697          + switch_s * float(swapped))
698 return {
699     "angle_cost": ang_cost,
700     "gdop": gdop_val,
701     "rms": rms,
702     "sampled_error": sampled_error,
703     "time_s": time_s,

```

```

704     "coverage": coverage,
705     "detection_coverage": detection_coverage,
706 }
707
708 def _cos_interval(lo: float, hi: float) -> tuple[float, float]:
709     values = [math.cos(lo), math.cos(hi)]
710     first = math.ceil(lo / math.pi)
711     last = math.floor(hi / math.pi)
712     for index in range(first, last + 1):
713         values.append(math.cos(index * math.pi))
714     return min(values), max(values)
715
716 def _contains_sin_zero(lo: float, hi: float) -> bool:
717     return math.ceil(lo / math.pi) <= math.floor(hi / math.pi)
718
719 def continuous_two_site_error_bound(
720     site1: Sequence[float] | Arr, site2: Sequence[float] | Arr,
721     bearing_deg: float, ranges: tuple[float, float] = (5.0, 1500.0),
722     err_deg: float = 1.0, abs_tol_m: float = 1e-3,
723     max_intervals: int = 100_000,
724 ) -> dict[str, float | int | bool]:
725     first_site = _point(site1, "site1")
726     second_site = _point(site2, "site2")
727     lo_range, hi_range = ranges
728     if (not math.isfinite(bearing_deg) or not math.isfinite(err_deg)
729         or not math.isfinite(lo_range) or not math.isfinite(hi_range)
730         or not 0 <= err_deg < 90 or not 0 < lo_range <= hi_range):
731         raise ValueError("invalid bearing sector")
732     if (not math.isfinite(abs_tol_m) or abs_tol_m <= 0
733         or max_intervals < 1):
734         raise ValueError("invalid interval controls")
735     delta = second_site - first_site
736     baseline = float(np.linalg.norm(delta))
737     if baseline <= np.finfo(float).tiny:
738         return {"lower_m": math.inf, "upper_m": math.inf,
739             "interval_count": 0, "certified": True}
740     alpha = math.atan2(float(delta[1]), float(delta[0]))
741     center = math.radians(bearing_deg)
742     err = math.radians(err_deg)
743     beta_lo = alpha - (center + err)
744     beta_hi = alpha - (center - err)
745     if _contains_sin_zero(beta_lo, beta_hi):
746         return {"lower_m": math.inf, "upper_m": math.inf,
747             "interval_count": 0, "certified": True}
748     error_rad = err
749
750 def exact(beta: float, source_range: float) -> float:

```

```

751 cosine = math.cos(beta)
752 sine = abs(math.sin(beta))
753 if sine == 0.0:
754     return math.inf
755 range2 = (source_range * source_range + baseline * baseline
756           - 2.0 * source_range * baseline * cosine)
757 numerator = (range2
758              + source_range * abs(source_range - baseline * cosine))
759 return error_rad * math.hypot(
760     source_range, numerator / (baseline * sine))
761
762 def point_value(beta: float) -> float:
763     return max(exact(beta, lo_range), exact(beta, hi_range))
764
765 def upper(box_lo: float, box_hi: float) -> float:
766     cos_lo, cos_hi = _cos_interval(box_lo, box_hi)
767     cos_lo = math.nextafter(cos_lo, -math.inf)
768     cos_hi = math.nextafter(cos_hi, math.inf)
769     sine_min = min(abs(math.sin(box_lo)), abs(math.sin(box_hi)))
770     sine_min = max(math.nextafter(sine_min, 0.0), np.finfo(float).tiny)
771     values = []
772     for source_range in (lo_range, hi_range):
773         range2_upper = (source_range * source_range + baseline * baseline
774                        - 2.0 * source_range * baseline * cos_lo)
775         abs_upper = max(
776             abs(source_range - baseline * cos_lo),
777             abs(source_range - baseline * cos_hi),
778         )
779         numerator_upper = range2_upper + source_range * abs_upper
780         value = error_rad * math.hypot(
781             source_range, numerator_upper / (baseline * sine_min))
782         values.append(math.nextafter(value, math.inf))
783     return max(values)
784
785 midpoint = (beta_lo + beta_hi) / 2.0
786 lower = max(point_value(beta_lo), point_value(midpoint),
787             point_value(beta_hi))
788 heap = [(-upper(beta_lo, beta_hi), beta_lo, beta_hi)]
789 interval_count = 1
790 while heap and -heap[0][0] > lower + abs_tol_m:
791     if interval_count >= max_intervals:
792         break
793     _, box_lo, box_hi = heapq.heappop(heap)
794     middle = (box_lo + box_hi) / 2.0
795     lower = max(lower, point_value(middle))
796     for left, right in ((box_lo, middle), (middle, box_hi)):
797         heapq.heappush(heap, (-upper(left, right), left, right))

```

```

798     interval_count += 1
799 certified_upper = max(lower, -heap[0][0] if heap else lower)
800 return {
801     "lower_m": lower,
802     "upper_m": certified_upper,
803     "interval_count": interval_count,
804     "certified": certified_upper <= lower + abs_tol_m,
805 }
806
807 def pareto_mask(vals: Sequence[Sequence[float]] | Arr,
808               tol: float = eps) -> Arr:
809     x = np.asarray(vals, dtype=float)
810     tol = _relative_tol(tol)
811     if x.ndim != 2 or len(x) == 0:
812         raise ValueError("vals must have shape (n, m) with n positive")
813     ok = np.all(np.isfinite(x), axis=1)
814     ids = np.flatnonzero(ok)
815     keep = np.zeros(len(x), dtype=bool)
816     if len(ids) == 0:
817         return keep
818     scale = np.maximum(np.max(np.abs(x[ids])), axis=0),
819                     np.finfo(float).tiny)
820     cut = tol * scale
821     for i in ids:
822         dom = np.all(x[ids] <= x[i] + cut, axis=1)
823         strict = np.any(x[ids] < x[i] - cut, axis=1)
824         if not np.any(dom & strict):
825             keep[i] = True
826     return keep
827
828 def pareto_second_sites(cand: Sequence[Sequence[float]] | Arr,
829                       site1: Sequence[float], bearing_deg: float,
830                       ranges: Sequence[float] | Arr,
831                       recv_rad: float = 1000.0, quantile: float = 0.9,
832                       min_coverage: float = 1.0,
833                       switched: bool = False) -> dict[str, Arr]:
834     res = second_site_metrics(cand, site1, bearing_deg, ranges, recv_rad,
835                             quantile=quantile, switched=switched)
836     vals = np.column_stack((res["angle_cost"], res["time_s"]))
837     feas = res["coverage"] + eps >= min_coverage
838     vals[~feas] = math.inf
839     res["pareto"] = pareto_mask(vals)
840     return res
841
842 def pareto_second_sites_cone(
843     cand: Sequence[Sequence[float]] | Arr,
844     site1: Sequence[float], bearing_deg: float,

```

```

845     ranges: tuple[float, float] = (5.0, 1500.0),
846     err_deg: float = 1.0, recv_rad: float = 1000.0,
847     quantile: float = 1.0, min_coverage: float = 1.0,
848     n_range: int = 41, n_angle: int = 21,
849     switched: bool = False, conditional_reception: bool = True,
850 ) -> dict[str, Arr]:
851     src = source_cone_samples(
852         site1, bearing_deg, ranges, err_deg, n_range=n_range, n_angle=
            n_angle)
853     res = second_site_metrics_sources(
854         cand, site1, src, err_deg=err_deg, recv_rad=recv_rad,
855         quantile=quantile,
856         switched=switched)
857     vals = np.column_stack((res["sampled_error"], res["time_s"]))
858     if conditional_reception:
859         guaranteed = detected_candidate_mask(
860             cand, site1, bearing_deg, ranges, err_deg, recv_rad,
861             switched=switched)
862         sampled_coverage = res["detection_coverage"]
863     else:
864         guaranteed = robust_candidate_mask(
865             cand, site1, bearing_deg, ranges, err_deg, recv_rad,
866             switched=switched)
867         sampled_coverage = res["coverage"]
868     feasible = guaranteed & (sampled_coverage + eps >= min_coverage)
869     safe_vals = vals.copy()
870     safe_vals[~feasible] = math.inf
871     res["guaranteed"] = guaranteed
872     res["pareto"] = pareto_mask(safe_vals)
873     return res

```

B.2 在线程序公共模块

包导出文件 robot/__init__.py

```

1 from robot.q3_agent import MissionResult, Q3Agent
2 from robot.client import ApiClient, ApiError
3 from robot.q4_agent import Q4Agent
4
5 __all__ = ["ApiClient", "ApiError", "MissionResult", "Q3Agent", "Q4Agent"]

```

HTTP 客户端 robot/client.py

```

1 from __future__ import annotations
2
3 import json
4 import threading

```

```

5 import time
6 from typing import Any
7 from urllib.error import HTTPError, URLError
8 from urllib.request import Request, urlopen
9
10 Json = dict[str, Any]
11
12 class ApiError(RuntimeError):
13
14     def __init__(self, message: str, rejected: bool = False) -> None:
15         super().__init__(message)
16         self.rejected = rejected
17
18 class ApiClient:
19     def __init__(
20         self, base_url: str, robot_id: str,
21         timeout_s: float = 3.0, max_retry: int = 5,
22         backoff_s: float = 0.05, deadline_guard_s: float = 2.0,
23     ) -> None:
24         if not robot_id or not 1 <= len(robot_id.encode("utf-8")) <= 64:
25             raise ValueError("robot_id must contain 1 to 64 UTF-8 bytes")
26         if (timeout_s <= 0 or max_retry < 0 or backoff_s < 0
27             or deadline_guard_s < 0):
28             raise ValueError("invalid retry parameters")
29         self.base_url = base_url.rstrip("/")
30         self.robot_id = robot_id
31         self.timeout_s = timeout_s
32         self.max_retry = max_retry
33         self.backoff_s = backoff_s
34         self.deadline_guard_s = deadline_guard_s
35         self.seq = 0
36         self.deadline: float | None = None
37         self.lock = threading.Lock()
38
39     def _new_id(self, tag: str) -> str:
40         self.seq += 1
41         return f"{tag}-{self.seq:06d}"
42
43     def _base(self, request_id: str) -> Json:
44         return {
45             "arena_id": "default",
46             "robot_id": self.robot_id,
47             "request_id": request_id,
48         }
49
50     def _action(self, request_id: str, x: float, y: float, channel: int)
51         -> Json:

```

```

51     if not 1 <= channel <= 20:
52         raise ValueError("channel must be in 1..20")
53     if not abs(x) <= 2_000_000 or not abs(y) <= 2_000_000:
54         raise ValueError("position exceeds simulator limit")
55     out = self._base(request_id)
56     out["position"] = {"x": float(x), "y": float(y)}
57     out["channel"] = int(channel)
58     return out
59
60 def _check_deadline(self, path: str) -> None:
61     if path == "/exit" or self.deadline is None:
62         return
63     if time.monotonic() + self.deadline_guard_s >= self.deadline:
64         raise ApiError("real-time deadline guard reached")
65
66 def _request_timeout(self, path: str) -> float:
67     if path == "/exit" or self.deadline is None:
68         return self.timeout_s
69     remaining = self.deadline - time.monotonic() - self.deadline_guard_s
70     if remaining <= 0.0:
71         raise ApiError("real-time deadline guard reached")
72     return min(self.timeout_s, max(remaining, 0.001))
73
74 def _post_once(self, path: str, payload: Json) -> tuple[int, Json]:
75     data = json.dumps(payload, ensure_ascii=False,
76                       separators=(",", ":")).encode("utf-8")
77     req = Request(
78         self.base_url + path, data=data,
79         headers={"Content-Type": "application/json"}, method="POST")
80     try:
81         with urlopen(req, timeout=self._request_timeout(path)) as resp:
82             status = int(resp.status)
83             body = json.loads(resp.read().decode("utf-8"))
84     except HTTPError as exc:
85         status = int(exc.code)
86         raw = exc.read().decode("utf-8", errors="replace")
87         try:
88             body = json.loads(raw)
89         except json.JSONDecodeError:
90             body = {"accepted": False, "raw": raw}
91     if not isinstance(body, dict):
92         raise ApiError("response must be a JSON object")
93     return status, body
94
95 def _post(self, path: str, payload: Json) -> Json:
96     with self.lock:
97         self._check_deadline(path)

```

```

98     saw_response = False
99     last_status = 0
100    for attempt in range(self.max_retry + 1):
101        try:
102            status, body = self._post_once(path, payload)
103            saw_response = True
104            last_status = status
105            if status == 200 and body.get("accepted") is True:
106                return body
107            if status not in (429, 500):
108                detail = body.get("error") or body.get("message") or body.
get("raw")
109                extra = f", detail={detail}" if detail else ""
110                hint = ""
111                if path == "/enter" and status == 200:
112                    hint = " (a duplicate /enter is rejected; each run allows
one entry)"
113                raise ApiError(
114                    f"{path} rejected: status={status}, "
115                    f"accepted={body.get('accepted')}{extra}{hint}", rejected=
True)
116            except (TimeoutError, ConnectionError, URLError,
117                    json.JSONDecodeError):
118                pass
119            if attempt == self.max_retry:
120                break
121            self._check_deadline(path)
122            time.sleep(min(self.backoff_s * 2 ** attempt, 1.0))
123            if saw_response:
124                raise ApiError(
125                    f"{path} still rejected after {self.max_retry + 1} attempts:
status={last_status}",
126                    rejected=True)
127            raise ApiError(
128                f"{path} connection failed after {self.max_retry + 1} attempts):
the interface may not be open yet, or the run has ended")
129
130    def enter_payload(self, payload: Json) -> Json:
131        body = self._post("/enter", payload)
132        remain = float(body["remaining_real_duration_s"])
133        self.deadline = time.monotonic() + remain
134        return body
135
136    def enter(self) -> Json:
137        return self.enter_payload(self._base(self._new_id("enter")))
138
139    def enter_when_open(self, wait_s: float = 300.0,

```

```

140         request_id: str = "enter-wait-000001") -> Json:
141     payload = self._base(request_id)
142     end = time.monotonic() + wait_s
143     delay = 0.5
144     rejected: ApiError | None = None
145     while True:
146         try:
147             return self.enter_payload(payload)
148         except ApiError as exc:
149             if exc.rejected:
150                 rejected = exc
151                 break
152         except (TimeoutError, ConnectionError, URLError):
153             pass
154         if time.monotonic() + delay >= end:
155             raise ApiError(
156                 f"the interface did not open within {wait_s:.0f} s; start the
test and wait for the 5 s countdown")
157             time.sleep(delay)
158             delay = min(delay * 1.5, 2.0)
159         raise ApiError(
160             f"/enter rejected: {rejected}. Check whether this run was already
entered, "
161             "and whether another program is still running")
162
163     def measure(self, x: float, y: float, channel: int) -> Json:
164         payload = self._action(self._new_id("measure"), x, y, channel)
165         return self._post("/measure", payload)
166
167     def clear(self, x: float, y: float, channel: int) -> Json:
168         payload = self._action(self._new_id("clear"), x, y, channel)
169         return self._post("/clear", payload)
170
171     def exit(self) -> Json:
172         payload = self._base(self._new_id("exit"))
173         return self._post("/exit", payload)

```

B.3 问题三、四基础版完整程序

问题三基础智能体 robot/q3_agent.py

```

1 from __future__ import annotations
2
3 import math
4 from dataclasses import dataclass, field
5
6 import numpy as np

```

```

7
8 from geometry_solver import localization_polygon,
   minimum_enclosing_circle
9 from robot.client import ApiClient, ApiError
10
11 Arr = np.ndarray
12
13 @dataclass
14 class Observation:
15     site: Arr
16     bearing_deg: float
17
18 @dataclass
19 class ChannelTrack:
20     channel: int
21     status: str = "unknown"
22     obs: list[Observation] = field(default_factory=list)
23     radii: list[float] = field(default_factory=list)
24     clear_site: Arr | None = None
25
26 @dataclass
27 class MissionResult:
28     cleared: int
29     absent_certified: int
30     measures: int
31     clear_calls: int
32     virtual_time_s: float
33     complete: bool
34
35 def q3_scan_sites(radius: float = 1200.0) -> Arr:
36     angle = np.arange(6) * math.pi / 3.0
37     ring = radius * np.column_stack((np.cos(angle), np.sin(angle)))
38     return np.vstack((np.zeros(2), ring))
39
40 def polygon_clear_sites(poly: Arr, cover_rad: float = 20.0):
41     p = np.asarray(poly, dtype=float)
42     if p.ndim != 2 or p.shape[1] != 2 or len(p) == 0 or cover_rad <= 0:
43         raise ValueError("invalid fallback polygon")
44     lo = p.min(axis=0)
45     hi = p.max(axis=0)
46     step = cover_rad * math.sqrt(2.0)
47     nx = max(1, int(math.ceil((hi[0] - lo[0]) / step)))
48     ny = max(1, int(math.ceil((hi[1] - lo[1]) / step)))
49     xs = np.linspace(lo[0], hi[0], nx + 1)
50     ys = np.linspace(lo[1], hi[1], ny + 1)
51     points = []
52     for i, x in enumerate(xs):

```

```

53     row = ys if i % 2 == 0 else ys[::-1]
54     points.extend(np.array([x, y]) for y in row)
55     return np.asarray(points)
56
57 def safe_lateral_site(site: Arr, bearing_deg: float, side: float = 1.0)
    -> Arr:
58     angle = math.radians(bearing_deg)
59     u = np.array([math.cos(angle), math.sin(angle)])
60     v = np.array([-math.sin(angle), math.cos(angle)])
61     return site + 400.0 * u + side * 300.0 * v
62
63 class Q3Agent:
64     def __init__(self, client: ApiClient, err_deg: float = 1.01,
65                 max_obs: int = 8, schedule: str = "batch") -> None:
66         if schedule not in {"batch", "immediate"}:
67             raise ValueError("schedule must be batch or immediate")
68         self.client = client
69         self.err_deg = err_deg
70         self.max_obs = max_obs
71         self.schedule = schedule
72         self.tracks = {channel: ChannelTrack(channel) for channel in range
(1, 21)}
73         self.measures = 0
74         self.clear_calls = 0
75         self.virtual_s = 0.0
76         self.pos = np.zeros(2)
77         self.current_channel = 1
78         self.cleared_mask = 0
79         self.absent_mask = 0
80         self.entered_here = False
81
82     def _measure(self, site: Arr, channel: int) -> dict[str, object]:
83         body = self.client.measure(float(site[0]), float(site[1]), channel)
84         self.measures += 1
85         self.virtual_s = float(body["virtual_time_s"])
86         self.pos = site.copy()
87         self.current_channel = channel
88         return body
89
90     def _clear(self, site: Arr, track: ChannelTrack,
91               certified: bool = True) -> bool:
92         body = self.client.clear(float(site[0]), float(site[1]), track.
channel)
93         self.clear_calls += 1
94         self.virtual_s = float(body["virtual_time_s"])
95         self.pos = site.copy()
96         if body.get("clear_result") != "success":

```

```

97         if not certified:
98             return False
99         track.status = "geometry_fault"
100         raise ApiError(f"certified clear failed on channel {track.channel}
    ")
101     track.status = "cleared"
102     track.clear_site = site.copy()
103     self.cleared_mask |= 1 << (track.channel - 1)
104     return True
105
106 def _record(self, track: ChannelTrack, site: Arr,
107             body: dict[str, object]) -> None:
108     result = body.get("measure_result")
109     if result == "near":
110         track.status = "clear_ready"
111         self._clear(site, track)
112         return
113     if result != "direction":
114         raise ApiError(f"expected positive detection on channel {track.
channel}")
115     bearing = float(body["svd_deg"])
116     if not math.isfinite(bearing):
117         raise ApiError(f"invalid bearing on channel {track.channel}")
118     track.obs.append(Observation(site.copy(), bearing))
119     track.status = "positively_detected" if len(track.obs) == 1 else "
localizing"
120
121 def _region(self, track: ChannelTrack) -> tuple[Arr, Arr, float]:
122     sites = np.vstack([obs.site for obs in track.obs])
123     bearings = np.array([obs.bearing_deg for obs in track.obs])
124     poly, _ = localization_polygon(
125         sites, bearings, err_deg=self.err_deg, n_bound=1440, outer=True)
126     if len(poly) == 0:
127         track.status = "geometry_fault"
128         raise ApiError(f"empty bearing intersection on channel {track.
channel}")
129     center, radius = minimum_enclosing_circle(poly)
130     return poly, center, radius
131
132 def _next_site(self, track: ChannelTrack, center: Arr, radius: float)
-> Arr:
133     first = track.obs[0]
134     if len(track.obs) == 1:
135         return safe_lateral_site(first.site, first.bearing_deg, 1.0)
136     if radius <= 900.0:
137         if not self._seen(track, center):
138             return center

```

```

139     step = min(40.0, max((1000.0 - radius) / 4.0, 0.01))
140     for index in range(16):
141         angle = math.radians(first.bearing_deg + 137.5 * index)
142         candidate = center + step * np.array([math.cos(angle), math.sin(
angle)])
143         if not self._seen(track, candidate):
144             return candidate
145     if len(track.obs) == 2:
146         return safe_lateral_site(first.site, first.bearing_deg, -1.0)
147     track.status = "geometry_fault"
148     raise ApiError(f"localization radius remains unsafe on channel {
track.channel}")
149
150 @staticmethod
151 def _seen(track: ChannelTrack, site: Arr):
152     return any(np.linalg.norm(obs.site - site) <= 1e-7 for obs in track.
obs)
153
154 def _lost_signal(self, track: ChannelTrack) -> None:
155     track.status = "geometry_fault"
156     raise ApiError(f"guaranteed follow-up lost signal on channel {track.
channel}")
157
158 def _finish_channel(self, track: ChannelTrack) -> None:
159     last_poly: Arr | None = None
160     for _ in range(self.max_obs):
161         if track.status == "cleared":
162             return
163         if len(track.obs) == 1:
164             center = track.obs[0].site
165             radius = math.inf
166         else:
167             poly, center, radius = self._region(track)
168             last_poly = poly
169             if radius <= 20.0:
170                 track.status = "clear_ready"
171                 self._clear(center, track)
172                 return
173             track.radii.append(radius)
174             site = self._next_site(track, center, radius)
175             if self._seen(track, site):
176                 track.status = "geometry_fault"
177                 raise ApiError(f"repeated localization site on channel {track.
channel}")
178             body = self._measure(site, track.channel)
179             result = body.get("measure_result")
180             if result == "no_signal":

```

```

181         self._lost_signal(track)
182         return
183     self._record(track, site, body)
184     track.status = "geometry_fault"
185     detail = "after intersection" if last_poly is not None else "before
intersection"
186     raise ApiError(
187         f"measurement cap reached {detail} on channel {track.channel}")
188
189 def _scan_sites(self) -> Arr:
190     return q3_scan_sites()
191
192 def _known_count(self) -> int:
193     return sum(track.status != "unknown" for track in self.tracks.values
())
194
195 def _channel_order(self) -> list[int]:
196     unknown = [
197         channel for channel, track in self.tracks.items()
198         if track.status == "unknown"
199     ]
200     return sorted(unknown, key=lambda channel: (channel - self.
current_channel) % 20)
201
202 def _discover(self, immediate: bool) -> None:
203     stop = False
204     for site in self._scan_sites():
205         for channel in self._channel_order():
206             track = self.tracks[channel]
207             body = self._measure(site, channel)
208             if body.get("measure_result") == "no_signal":
209                 continue
210             self._record(track, site, body)
211             if immediate and track.status != "cleared":
212                 self._finish_channel(track)
213             if self._known_count() >= 16:
214                 stop = True
215                 break
216         if stop:
217             break
218
219 def _finish_pending(self) -> None:
220     while True:
221         pending = [
222             track for track in self.tracks.values()
223             if track.status in {"positively_detected", "localizing"}
224         ]

```

```

225     if not pending:
226         return
227     def travel(track: ChannelTrack) -> float:
228         first = track.obs[0]
229         site = safe_lateral_site(first.site, first.bearing_deg)
230         return float(np.linalg.norm(site - self.pos))
231
232     track = min(pending, key=lambda item: (travel(item), item.channel)
233 )
234     self._finish_channel(track)
235
236 def _certify_unknown(self) -> None:
237     for track in self.tracks.values():
238         if track.status == "unknown":
239             track.status = "absent_certified"
240             self.absent_mask |= 1 << (track.channel - 1)
241
242 def run(self, enter_body: dict[str, object] | None = None) ->
243     MissionResult:
244     if enter_body is None:
245         enter_body = self.client.enter()
246         self.entered_here = True
247     self.virtual_s = float(enter_body["virtual_time_s"])
248     try:
249         self._discover(immediate=self.schedule == "immediate")
250         if self.schedule == "batch":
251             self._finish_pending()
252             self._certify_unknown()
253             complete = self.cleared_mask | self.absent_mask == (1 << 20) - 1
254             if not complete:
255                 raise ApiError("mission ended without a channel certificate")
256             out = self.client.exit()
257             self.virtual_s = float(out["virtual_time_s"])
258     except Exception:
259         try:
260             self.client.exit()
261         except Exception:
262             pass
263         raise
264     cleared = sum(track.status == "cleared" for track in self.tracks.
265 values())
266     absent = sum(track.status == "absent_certified" for track in self.
267 tracks.values())
268     return MissionResult(
269         cleared, absent, self.measures, self.clear_calls,
270         self.virtual_s, complete)

```

问题四基础智能体 robot/q4_agent.py

```

1  from __future__ import annotations
2
3  import math
4
5  import numpy as np
6
7  from robot.q3_agent import Arr, ChannelTrack, Q3Agent
8  from robot.client import ApiError
9
10 def ordered_route(points: Arr, start: Arr) -> Arr:
11     p = np.asarray(points, dtype=float)
12     if p.ndim != 2 or p.shape[1] != 2 or len(p) == 0:
13         raise ValueError("points must have shape (n, 2) with n positive")
14     current = np.asarray(start, dtype=float)
15     unused = list(range(len(p)))
16     order: list[int] = []
17     while unused:
18         pick = min(unused, key=lambda i: (float(np.linalg.norm(p[i] -
19             current)), i))
20         order.append(pick)
21         unused.remove(pick)
22         current = p[pick]
23     route = p[order].copy()
24     improved = True
25     while improved:
26         improved = False
27         for i in range(1, len(route) - 1):
28             for j in range(i + 1, len(route)):
29                 before = float(np.linalg.norm(route[i - 1] - route[i]))
30                 after = float(np.linalg.norm(route[i - 1] - route[j]))
31                 if j + 1 < len(route):
32                     before += float(np.linalg.norm(route[j] - route[j + 1]))
33                     after += float(np.linalg.norm(route[i] - route[j + 1]))
34                 if after + 1e-9 < before:
35                     route[i:j + 1] = route[i:j + 1][::-1]
36                     improved = True
37     return route
38
39 def q4_scan_sites(spacing: float = 990.0,
40     target_rad: float = 1800.0) -> Arr:
41     if spacing <= 0 or spacing >= 1000.0 or target_rad <= 0:
42         raise ValueError("invalid Q4 lattice parameters")
43     e1 = np.array([spacing, 0.0])
44     e2 = np.array([spacing / 2.0, math.sqrt(3.0) * spacing / 2.0])
45     cutoff = target_rad + spacing

```

```

45     limit = int(math.ceil(cutoff / spacing)) * 2 + 2
46     points = []
47     for i in range(-limit, limit + 1):
48         for j in range(-limit, limit + 1):
49             point = i * e1 + j * e2
50             if np.linalg.norm(point) <= cutoff + 1e-9:
51                 points.append(point)
52     return ordered_route(np.asarray(points), np.zeros(2))
53
54 def _distance_to_sector(points: Arr, lo: float, hi: float,
55                        err_rad: float) -> Arr:
56     x = points[:, 0]
57     y = points[:, 1]
58     radius = np.hypot(x, y)
59     angle = np.arctan2(y, x)
60     inside = np.abs(angle) <= err_rad
61     radial = np.abs(radius - np.clip(radius, lo, hi))
62     abs_y = np.abs(y)
63     projection = x * math.cos(err_rad) + abs_y * math.sin(err_rad)
64     edge_r = np.clip(projection, lo, hi)
65     edge = np.hypot(
66         x - edge_r * math.cos(err_rad),
67         abs_y - edge_r * math.sin(err_rad),
68     )
69     return np.where(inside, radial, edge)
70
71 def bearing_clear_sites(
72     site: Arr, bearing_deg: float, err_deg: float = 1.01,
73     ranges: tuple[float, float] = (5.0, 1500.0),
74     cover_rad: float = 20.0, start: Arr | None = None,
75 ) -> Arr:
76     origin = np.asarray(site, dtype=float)
77     if origin.shape != (2,) or not np.all(np.isfinite(origin)):
78         raise ValueError("site must be a finite point")
79     lo, hi = ranges
80     if not 0 <= lo < hi or not 0 <= err_deg < 90 or cover_rad <= 0:
81         raise ValueError("invalid bearing sector parameters")
82     spacing = cover_rad * math.sqrt(3.0) * 0.995
83     e1 = np.array([spacing, 0.0])
84     e2 = np.array([spacing / 2.0, math.sqrt(3.0) * spacing / 2.0])
85     height = float(e2[1])
86     limit_i = int(math.ceil((hi + cover_rad) / spacing)) + 4
87     limit_j = int(math.ceil((hi * math.sin(math.radians(err_deg))
88                             + cover_rad) / height)) + 4
89     local = []
90     for i in range(-limit_i, limit_i + 1):
91         for j in range(-limit_j, limit_j + 1):

```

```

92     point = i * e1 + j * e2
93     local.append(point)
94     local_points = np.asarray(local)
95     keep = _distance_to_sector(
96         local_points, lo, hi, math.radians(err_deg)) <= cover_rad + 1e-9
97     local_points = local_points[keep]
98     angle = math.radians(bearing_deg)
99     rotation = np.array([
100         [math.cos(angle), -math.sin(angle)],
101         [math.sin(angle), math.cos(angle)],
102     ])
103     global_points = origin + local_points @ rotation.T
104     route_start = origin if start is None else np.asarray(start, dtype=
105         float)
106     return ordered_route(global_points, route_start)
107
108 class Q4Agent(Q3Agent):
109
110     def _scan_sites(self) -> Arr:
111         return q4_scan_sites()
112
113     def _lost_signal(self, track: ChannelTrack) -> None:
114         first = track.obs[0]
115         track.status = "clear_fallback"
116         for site in bearing_clear_sites(
117             first.site, first.bearing_deg, self.err_deg, start=self.pos):
118             if self._clear(site, track, certified=False):
119                 return
120         track.status = "geometry_fault"
121         raise ApiError(f"directional fallback exhausted on channel {track.
122             channel}")

```

基础版入口 robot/main.py

```

1  from __future__ import annotations
2
3  import argparse
4  import json
5  import os
6  import sys
7  import time
8  from pathlib import Path
9
10 root = Path(__file__).resolve().parents[1]
11 sys.path.insert(0, str(root))
12
13 from robot.q3_agent import Q3Agent
14 from robot.client import ApiClient, ApiError

```

```

15 from robot.q4_agent import Q4Agent
16
17 log_path: Path | None = None
18
19 def redact_identity(value: object) -> object:
20     if isinstance(value, dict):
21         return {
22             key: "<redacted>" if key in {"robot_id", "team_no"}
23             else redact_identity(item)
24             for key, item in value.items()
25         }
26     if isinstance(value, list):
27         return [redact_identity(item) for item in value]
28     return value
29
30 def log_call(path: str, payload: dict, status: int, body: dict,
31             ms: int) -> None:
32     if log_path is None:
33         return
34     try:
35         with log_path.open("a", encoding="utf-8") as file:
36             file.write(json.dumps({
37                 "t": ms, "path": path, "request": redact_identity(payload),
38                 "status": status, "response": redact_identity(body),
39             }, ensure_ascii=False) + "\n")
40     except OSError:
41         pass
42
43 def attach_logging(client: ApiClient) -> None:
44     original = client._post_once
45
46     def wrapped(path: str, payload: dict) -> tuple[int, dict]:
47         status, body = original(path, payload)
48         log_call(path, payload, status, body, time.time_ns() // 1_000_000)
49         return status, body
50
51     client._post_once = wrapped
52
53 def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
54     parser = argparse.ArgumentParser(
55         description="Q3/Q4 robot-dog automatic localization and clearing")
56     parser.add_argument(
57         "--team", default=os.environ.get("CUMCM_TEAM_NO", ""),
58         help="team number; must match the account logged in to the simulator")
59     parser.add_argument("--base-url", default="http://127.0.0.1:2026",
60                         help="simulator base URL")

```

```

61 parser.add_argument("--problem", type=int, choices=(3, 4), default=3,
62                     help="solve problem 3 or problem 4")
63 parser.add_argument("--schedule", choices=("batch", "immediate"),
64                     default="batch", help="batch scheduling, or
        localize immediately after discovery")
65 parser.add_argument("--wait-s", type=float, default=300.0,
66                     help="maximum seconds to wait for the interface")
67 parser.add_argument("--max-obs", type=int, default=8,
68                     help="maximum observations per channel")
69 parser.add_argument("--err-deg", type=float, default=1.01,
70                     help="bearing-error safety margin in degrees")
71 parser.add_argument("--log-dir", default=None,
72                     help="command log directory (default: code/outputs
        )")
73 return parser.parse_args(argv)
74
75 def main(argv: list[str] | None = None) -> int:
76     global log_path
77     args = parse_args(argv)
78     if not args.team:
79         print("missing team number: pass --team or set CUMCM_TEAM_NO")
80         return 2
81
82     log_dir = Path(args.log_dir) if args.log_dir else root / "code" / "
        outputs"
83     log_dir.mkdir(parents=True, exist_ok=True)
84     log_path = log_dir / (
85         f"q{args.problem}_{time.strftime('%Y%m%d-%H%M%S')}.jsonl")
86
87     client = ApiClient(args.base_url, args.team)
88     attach_logging(client)
89     started = time.perf_counter()
90     try:
91         enter = client.enter_when_open(wait_s=args.wait_s)
92     except ApiError as exc:
93         print(f"enter failed: {exc}")
94         print(f"command log for this run: {log_path}")
95         return 2
96
97     remain = float(enter.get("remaining_real_duration_s", 0.0))
98     print(f"entered; remaining real time {remain:.0f} s")
99
100    agent_type = Q3Agent if args.problem == 3 else Q4Agent
101    agent = agent_type(
102        client, err_deg=args.err_deg, max_obs=args.max_obs,
103        schedule=args.schedule)
104    error: str | None = None

```

```

105     result = None
106     try:
107         result = agent.run(enter_body=enter)
108     except Exception as exc:
109         error = f"{type(exc).__name__}: {exc}"
110         print(f"run interrupted: {error}")
111
112     elapsed = time.perf_counter() - started
113     out: dict[str, object] = {
114         "program_s": round(elapsed, 3),
115         "error": error,
116         "log": str(log_path),
117     }
118     if result is not None:
119         out.update({
120             "cleared": result.cleared,
121             "absent_certified": result.absent_certified,
122             "measures": result.measures,
123             "clear_calls": result.clear_calls,
124             "virtual_time_s": result.virtual_time_s,
125             "mean_virtual_per_cleared_s": (
126                 result.virtual_time_s / result.cleared if result.cleared else
127                 None),
128             "complete": result.complete,
129         })
130     print(json.dumps(out, ensure_ascii=False, indent=2))
131     return 0 if (result is not None and result.complete) else 1
132
133 if __name__ == "__main__":
134     raise SystemExit(main())

```

B.4 六次正式测试所用提交版完整程序

提速几何与路径模块 robot/geometry_solver_fast.py

```

1 from __future__ import annotations
2
3 import math
4
5 import numpy as np
6
7 from geometry_solver import (bearing_halfplanes, convex_hull,
8                               minimum_enclosing_circle, regular_bound)
9
10 Arr = np.ndarray
11 guard = 1e-6
12

```

```

13 def cut(poly: Arr, normal: Arr, limit: float) -> Arr:
14     if len(poly) == 0:
15         return poly.copy()
16     normal = np.asarray(normal, dtype=float)
17     size = float(np.linalg.norm(normal))
18     if not math.isfinite(size) or size <= 0:
19         raise ValueError("invalid clipping normal")
20     n = normal / size
21     origin = poly[0]
22     local = poly - origin
23     pad = max(guard, 64 * np.finfo(float).eps * (
24         float(np.max(np.abs(poly))) + abs(limit / size)))
25     bound = limit / size - float(n @ origin) + pad
26     residual = local @ n - bound
27     if np.all(residual <= 0):
28         return poly.copy()
29     if np.all(residual > 0):
30         return np.empty((0, 2))
31     out = []
32     for i, a in enumerate(local):
33         j = (i + 1) % len(local)
34         fa, fb = float(residual[i]), float(residual[j])
35         if fa <= 0:
36             out.append(a + origin)
37         if (fa <= 0) != (fb <= 0):
38             t = min(1.0, max(0.0, fa / (fa - fb)))
39             out.append(a + t * (local[j] - a) + origin)
40     return np.asarray(out, dtype=float).reshape(-1, 2)
41
42 def cut_bearing(poly: Arr, site: Arr, angle: float, err: float) -> Arr:
43     out = poly
44     for hp in bearing_halfplanes(site, angle, err):
45         out = cut(out, hp[:2], float(hp[2]))
46     return out
47
48 def cut_disk(poly: Arr, center: Arr, radius: float) -> Arr:
49     out = poly
50     for angle in np.arange(128) * (2 * math.pi / 128):
51         n = np.array([math.cos(angle), math.sin(angle)])
52         out = cut(out, n, float(n @ center) + radius)
53         if len(out) == 0:
54             break
55     return out
56
57 def enclosing(poly: Arr) -> tuple[Arr, float]:
58     if len(poly) == 0 or not np.all(np.isfinite(poly)):
59         raise ValueError("empty or nonfinite localization region")

```

```

60     center, _ = minimum_enclosing_circle(poly)
61     radius = float(np.max(np.linalg.norm(poly - center, axis=1))) + guard
62     if not np.all(np.isfinite(center)) or not math.isfinite(radius):
63         raise ValueError("nonfinite enclosing circle")
64     return center, radius
65
66 def forecast(poly: Arr, center: Arr, radius: float,
67             site: Arr, err: float) -> float:
68     delta = center - site
69     if np.linalg.norm(delta) <= max(5.0, radius * 0.15):
70         return max(2.5, radius * 0.55)
71     angle = math.degrees(math.atan2(delta[1], delta[0]))
72     pred = cut_bearing(poly, site, angle, err)
73     if len(pred) == 0:
74         return radius
75     _, value = enclosing(pred)
76     return min(radius, value)
77
78 def path_length(points: Arr, start: Arr) -> float:
79     if len(points) == 0:
80         return 0.0
81     p = np.vstack((start, points))
82     return float(np.linalg.norm(np.diff(p, axis=0), axis=1).sum())
83
84 def route_order(points: Arr, start: Arr) -> list[int]:
85     if len(points) == 0:
86         return []
87     left = set(range(len(points)))
88     order = []
89     pos = start
90     while left:
91         i = min(left, key=lambda k: (float(np.linalg.norm(points[k] - pos)),
92                                     k))
93         order.append(i)
94         left.remove(i)
95         pos = points[i]
96     for _ in range(12):
97         changed = False
98         for i in range(len(order) - 1):
99             a = start if i == 0 else points[order[i - 1]]
100             b, c = points[order[i]], points[order[i + 1]]
101             before = float(np.linalg.norm(a - b))
102             after = float(np.linalg.norm(a - c))
103             if j + 1 < len(order):
104                 d = points[order[j + 1]]
105                 before += float(np.linalg.norm(c - d))

```

```

106         after += float(np.linalg.norm(b - d))
107     if after + 1e-6 < before:
108         order[i:j + 1] = reversed(order[i:j + 1])
109         changed = True
110     if not changed:
111         break
112     return order
113
114 def insertion(center: Arr, start: Arr, remaining: Arr) -> tuple[int,
    float]:
115     if len(remaining) == 0:
116         return 0, float(np.linalg.norm(center - start))
117     p = np.vstack((start, remaining))
118     costs = [float(np.linalg.norm(center - p[i])
119                 + np.linalg.norm(center - p[i + 1])
120                 - np.linalg.norm(p[i + 1] - p[i]))
121              for i in range(len(p) - 1)]
122     costs.append(float(np.linalg.norm(center - p[-1])))
123     i = min(range(len(costs)), key=lambda j: (costs[j], j))
124     return i, costs[i]
125
126 def cover_cells(poly: Arr, radius: float, angle: float) -> list[Arr]:
127     u = np.array([math.cos(angle), math.sin(angle)])
128     v = np.array([-u[1], u[0]])
129     stack = [poly.copy()]
130     leaves = []
131     while stack:
132         p = stack.pop()
133         _, r = enclosing(p)
134         if r <= radius:
135             leaves.append(p)
136             continue
137         spans = [float(np.ptp(p @ axis)) for axis in (u, v)]
138         axis = u if spans[0] >= spans[1] else v
139         proj = p @ axis
140         mid = (float(proj.min()) + float(proj.max())) / 2
141         left = cut(p, axis, mid)
142         right = cut(p, -axis, -mid)
143         if len(left) == 0 or len(right) == 0 or max(spans) < 1e-4:
144             raise ValueError("cover subdivision failed to progress")
145         stack.extend((right, left))
146     return leaves
147
148 def definitely_disjoint(first: Arr, second: Arr) -> bool:
149     if len(first) == 0 or len(second) == 0:
150         return True
151     axes = [np.array([1.0, 0.0]), np.array([0.0, 1.0])]

```

```

152     for p in (first, second):
153         for edge in np.roll(p, -1, axis=0) - p:
154             size = float(np.linalg.norm(edge))
155             if size > guard:
156                 axes.append(np.array([-edge[1], edge[0]]) / size)
157     for n in axes:
158         a, b = first @ n, second @ n
159         if float(a.max()) < float(b.min()) - guard:
160             return True
161         if float(b.max()) < float(a.min()) - guard:
162             return True
163     return False
164
165 def inside_hull(hull: Arr, points: Arr) -> bool:
166     if len(hull) < 3:
167         return False
168     edges = np.roll(hull, -1, axis=0) - hull
169     lengths = np.linalg.norm(edges, axis=1)
170     if np.any(lengths <= 1e-9):
171         return False
172     offset = points[:, None, :] - hull[None, :, :]
173     signed = (edges[None, :, 0] * offset[:, :, 1]
174              - edges[None, :, 1] * offset[:, :, 0]) / lengths[None, :]
175     return bool(np.all(signed >= 1e-6))
176
177 def direction_certificate(sites: Arr) -> tuple[bool, int]:
178     boundary = regular_bound(1800.0, 72, outer=True)
179     stack = [(np.vstack((np.zeros(2), boundary[i], boundary[(i + 1) % 72])), 0)
180              for i in range(72))]
181     count = 0
182     while stack:
183         cell, depth = stack.pop()
184         count += 1
185         distances = np.linalg.norm(cell[:, None, :] - sites[None, :, :],
186                                   axis=2)
187         eligible = sites[np.max(distances, axis=0) <= 994.99]
188         if len(eligible) >= 3:
189             hull = convex_hull(eligible, tol=0.0)
190             if inside_hull(hull, cell):
191                 continue
192         if depth >= 22 or count >= 60000:
193             return False, count
194         lengths = np.linalg.norm(np.roll(cell, -1, axis=0) - cell, axis=1)
195         i = int(np.argmax(lengths))
196         j, k = (i + 1) % 3, (i + 2) % 3
197         middle = (cell[i] + cell[j]) / 2.0

```

```

197     stack.append((np.vstack((cell[i], middle, cell[k])), depth + 1))
198     stack.append((np.vstack((middle, cell[j], cell[k])), depth + 1))
199     return True, count
200
201 def compact_q4_sites() -> Arr:
202     inner_r, outer_r = 975.0, 1850.0
203     a = np.arange(7) * (2 * math.pi / 7)
204     b = np.arange(14) * (2 * math.pi / 14)
205     inner = inner_r * np.column_stack((np.cos(a), np.sin(a)))
206     outer = outer_r * np.column_stack((np.cos(b), np.sin(b)))
207     return np.vstack((np.zeros(2), inner, outer[np.r_[12, 13, np.arange
(12)])))
208
209 def covered_by_negative_disks(poly: Arr, negatives: Arr) -> bool:
210     stack = [(poly, 0)]
211     count = 0
212     while stack:
213         cell, depth = stack.pop()
214         count += 1
215         far = np.max(np.linalg.norm(cell[:, None, :] - negatives[None, :,
:], axis=2),
216                     axis=0)
217         if np.any(far <= 999.99):
218             continue
219         if depth >= 9 or count >= 80:
220             return False
221         center = cell.mean(axis=0)
222         if np.min(np.linalg.norm(negatives - center, axis=1)) >= 999.99:
223             return False
224         horizontal = np.ptp(cell[:, 0]) >= np.ptp(cell[:, 1])
225         axis = np.array([1.0, 0.0]) if horizontal else np.array([0.0, 1.0])
226         projection = cell @ axis
227         middle = (float(projection.min()) + float(projection.max())) / 2.0
228         for normal, limit in ((axis, middle), (-axis, -middle)):
229             part = cut(cell, normal, limit)
230             if len(part):
231                 stack.append((part, depth + 1))
232     return True
233
234 def unknown_negative_implied(site: Arr, negatives: list[Arr]) -> bool:
235     if not negatives:
236         return False
237     old = np.asarray(negatives)
238     if np.min(np.linalg.norm(old - site, axis=1)) <= 1e-7:
239         return True
240     if len(old) < 2:
241         return False

```

```

242 poly = cut_disk(regular_bound(1800.0, 72, outer=True), site, 1500.0)
243 for previous in old:
244     delta = previous - site
245     limit = float(delta @ site + 0.5 * (delta @ delta))
246     poly = cut(poly, delta, limit)
247     if len(poly) == 0:
248         return True
249 return covered_by_negative_disks(poly, old)
250
251 def route_cost(order: list[int], distances: Arr) -> float:
252     previous = len(distances) - 1
253     total = 0.0
254     for i in order:
255         total += float(distances[previous, i])
256         previous = i
257     return total
258
259 def exact_open(distances: Arr) -> list[int]:
260     n = len(distances) - 1
261     dp = np.full((1 << n, n), math.inf)
262     parent = np.full((1 << n, n), -1, dtype=np.int16)
263     for i in range(n):
264         dp[1 << i, i] = distances[n, i]
265     for mask in range(1, 1 << n):
266         for last in range(n):
267             if not mask & (1 << last) or not math.isfinite(float(dp[mask, last]
268 ])):
269                 continue
270             for nxt in range(n):
271                 if mask & (1 << nxt):
272                     continue
273                 new_mask = mask | (1 << nxt)
274                 value = float(dp[mask, last] + distances[last, nxt])
275                 if value < dp[new_mask, nxt]:
276                     dp[new_mask, nxt] = value
277                     parent[new_mask, nxt] = last
278             mask = (1 << n) - 1
279             last = int(np.argmin(dp[mask]))
280             order = []
281             while last >= 0:
282                 order.append(last)
283                 previous = int(parent[mask, last])
284                 mask ^= 1 << last
285                 last = previous
286             return order[::-1]
287
288 def improve_open(order: list[int], distances: Arr) -> list[int]:

```

```

288 out = order.copy()
289 n = len(out)
290 start = n
291 for _ in range(8):
292     changed = False
293     for i in range(n - 1):
294         a = start if i == 0 else out[i - 1]
295         for j in range(i + 1, n):
296             b, c = out[i], out[j]
297             before = float(distances[a, b])
298             after = float(distances[a, c])
299             if j + 1 < n:
300                 d = out[j + 1]
301                 before += float(distances[c, d])
302                 after += float(distances[b, d])
303                 if after + 1e-6 < before:
304                     out[i:j + 1] = out[i:j + 1][::-1]
305                     changed = True
306 best_cost = route_cost(out, distances)
307 best = None
308 for i in range(n):
309     reduced = out[:i] + out[i + 1:]
310     for j in range(n):
311         candidate = reduced[:j] + [out[i]] + reduced[j:]
312         value = route_cost(candidate, distances)
313         if value + 1e-6 < best_cost:
314             best_cost, best = value, candidate
315 if best is not None:
316     out = best
317     changed = True
318 if not changed:
319     break
320 return out
321
322 def joint_route(points: Arr, start: Arr, station_count: int) -> list[int]:
323     n = len(points)
324     if n == 0:
325         return []
326     p = np.vstack((points, start))
327     distances = np.linalg.norm(p[:, None, :] - p[None, :, :], axis=2)
328     if n <= 10:
329         return exact_open(distances)
330     incumbent = list(range(station_count))
331     for i in range(station_count, n):
332         candidates = [incumbent[:j] + [i] + incumbent[j:]]
333         for j in range(len(incumbent) + 1)]

```

```

334     incumbent = min(candidates, key=lambda order: route_cost(order,
335                        distances))
336 seeds = [incumbent]
337 for first in np.argsort(distances[n, :n])[:3]:
338     order = [int(first)]
339     left = set(range(n)) - {int(first)}
340     while left:
341         i = min(left, key=lambda j: (float(distances[order[-1], j]), j))
342         order.append(i)
343         left.remove(i)
344         seeds.append(order)
345 best = incumbent
346 best_cost = route_cost(best, distances)
347 for seed in seeds:
348     order = improve_open(seed, distances)
349     value = route_cost(order, distances)
350     if value + 1e-6 < best_cost:
351         best, best_cost = order, value
352 if sorted(best) != list(range(n)):
353     raise ValueError("route lost a required node")
354 return best
355
356 def bearing_radius_bound(poly: Arr, site: Arr, err: float = 1.01,
357                        step: float = 2.0) -> float:
358     if not 0 < step <= 10 or not 0 < err < 10:
359         raise ValueError("invalid bearing enclosure parameters")
360     if len(poly) == 0 or not np.all(np.isfinite(poly)) or not np.all(np.
361        isfinite(site)):
362         raise ValueError("invalid bearing enclosure geometry")
363     delta = poly - site
364     angles = np.sort(np.mod(np.arctan2(delta[:, 1], delta[:, 0]), 2 * math
365        .pi))
366     gaps = np.diff(np.r_[angles, angles[0] + 2 * math.pi])
367     k = int(np.argmax(gaps))
368     span = 2 * math.pi - float(gaps[k])
369     if span < math.pi - 1e-8 and np.min(np.linalg.norm(delta, axis=1)) > 1
370        e-7:
371         lo = math.degrees(float(angles[(k + 1) % len(angles)])) - err - 1e-7
372         hi = lo + math.degrees(span) + 2 * err + 2e-7
373     else:
374         lo, hi = 0.0, 360.0
375     count = max(1, math.ceil((hi - lo) / step))
376     width = (hi - lo) / count
377     worst = 0.0
378     for i in range(count):
379         angle = lo + (i + 0.5) * width
380         outer = cut_bearing(poly, site, angle, err + width / 2 + 1e-7)

```

```

377     if len(outer):
378         worst = max(worst, enclosing(outer)[1])
379     return worst + 1e-3
380
381 def visible_kernel(poly: Arr, positives: Arr) -> Arr:
382     if len(positives) < 2 or len(poly) == 0:
383         return np.empty((0, 2))
384     hulls = [convex_hull(np.vstack((positives, vertex)), tol=0.0) for
385               vertex in poly]
386     if any(len(hull) < 3 for hull in hulls):
387         return np.empty((0, 2))
388     kernel = hulls[0].copy()
389     for hull in hulls:
390         for a, b in zip(hull, np.roll(hull, -1, axis=0)):
391             edge = b - a
392             length = float(np.linalg.norm(edge))
393             if length <= 1e-8:
394                 return np.empty((0, 2))
395             normal = np.array([edge[1], -edge[0]]) / length
396             kernel = cut(kernel, normal, float(normal @ a) - 1e-4)
397             if len(kernel) == 0:
398                 return kernel
399     for hull in hulls:
400         for a, b in zip(hull, np.roll(hull, -1, axis=0)):
401             edge = b - a
402             cross = edge[0] * (kernel[:, 1] - a[1]) - edge[1] * (kernel[:, 0]
403             - a[0])
404             if np.min(cross) < 1e-5 * np.linalg.norm(edge):
405                 return np.empty((0, 2))
406     return kernel
407
408 def packet_sites(center: Arr, radius: float, start: Arr) -> Arr:
409     if not 0 <= radius <= 300.0:
410         raise ValueError("packet radius exceeds its receiving certificate")
411     size = 1.2 * radius + 8.0
412     phase = math.atan2(start[1] - center[1], start[0] - center[0])
413     angles = phase + np.arange(12) * math.pi / 6.0
414     return center + size * np.column_stack((np.cos(angles), np.sin(angles)
415     ))
416
417 def packet_localization_bound(radius: float, err: float = 1.01) -> float
418 :
419     size = 1.2 * radius + 8.0
420     h, w = size * math.cos(math.pi / 12), size * math.sin(math.pi / 12)
421     low = 2 * math.atan2(w, h + radius)
422     high = 2 * math.atan2(w, h - radius)
423     error = math.radians(err)

```

```

420 gap = min(low - 2 * error, math.pi - high - 2 * error)
421 if gap <= 0:
422     return math.inf
423 factor = math.sin(error) / math.sin(gap / 2)
424 if factor >= 1:
425     return math.inf
426 return 2 * factor * (size + radius) / (1 - factor) + 1e-3

```

问题三提交版智能体 robot/q3_agent_fast.py

```

1 from __future__ import annotations
2
3 import math
4 from collections import Counter
5 from dataclasses import dataclass, field
6
7 import numpy as np
8
9 from geometry_solver import regular_bound
10 from robot.q3_agent import MissionResult, Observation, q3_scan_sites,
    safe_lateral_site
11 from robot.client import ApiClient, ApiError
12 from robot.geometry_solver_fast import (
13     Arr, bearing_radius_bound, cover_cells, cut, cut_bearing, cut_disk,
14     definitely_disjoint, enclosing, forecast, guard, insertion,
15     joint_route,
16     packet_sites, route_order, unknown_negative_implied, visible_kernel,
17 )
18 @dataclass
19 class FastTrack:
20     channel: int
21     status: str = "unknown"
22     obs: list[Observation] = field(default_factory=list)
23     negatives: list[Arr] = field(default_factory=list)
24     history: list[tuple[Arr, dict[str, object]]] = field(default_factory=
        list)
25     failures: list[Arr] = field(default_factory=list)
26     poly: Arr | None = None
27     center: Arr | None = None
28     radius: float = math.inf
29     station_mask: int = 0
30     opportunities: int = 0
31
32 class Q3FastBase:
33     directional = False
34
35     def __init__(self, client: ApiClient, err_deg: float = 1.01,

```

```

36         opportunistic: bool = True, inline: bool = True) -> None:
37     if not math.isfinite(err_deg) or not 1.01 <= err_deg <= 2.0:
38         raise ValueError("err_deg must be in [1.01, 2.0]")
39     self.client = client
40     self.err_deg = err_deg
41     self.opportunistic = opportunistic
42     self.inline = inline
43     self.tracks = {i: FastTrack(i) for i in range(1, 21)}
44     self.pos = np.zeros(2)
45     self.current_channel = 1
46     self.measures = 0
47     self.clear_calls = 0
48     self.virtual_s = 0.0
49     self._entered = False
50     self._exited = False
51     self.sites = self._scan_sites()
52     self.full_mask = (1 << len(self.sites)) - 1
53
54     def _scan_sites(self) -> Arr:
55         return q3_scan_sites()
56
57     @staticmethod
58     def _cached(track: FastTrack, site: Arr) -> dict[str, object] | None:
59         for old, body in track.history:
60             if np.linalg.norm(site - old) <= 1e-7:
61                 return body
62         return None
63
64     def _refresh(self, track: FastTrack) -> None:
65         if track.poly is None or len(track.poly) == 0:
66             track.status = "geometry_fault"
67             raise ApiError(f"empty localization region on channel {track.
channel}")
68         track.center, track.radius = enclosing(track.poly)
69         track.status = "clear_ready" if track.radius <= 19.999 else "
localizing"
70
71     def _range_order(self, poly: Arr, positive: Arr, negative: Arr) -> Arr
:
72         delta = negative - positive
73         if np.linalg.norm(delta) <= 1e-7:
74             raise ApiError("conflicting measurements at the same position")
75         limit = float(delta @ positive + 0.5 * (delta @ delta))
76         return cut(poly, delta, limit)
77
78     def _accept_direction(self, track: FastTrack, site: Arr,
79                          bearing: float) -> None:

```

```

80     if not math.isfinite(bearing):
81         raise ApiError("nonfinite bearing")
82     if track.poly is None:
83         track.poly = regular_bound(1800.0, 360, outer=True)
84     track.poly = cut_bearing(track.poly, site, bearing, self.err_deg)
85     track.poly = cut_disk(track.poly, site, 1500.0)
86     if not self.directional:
87         for negative in track.negatives:
88             track.poly = self._range_order(track.poly, site, negative)
89     track.obs.append(Observation(site.copy(), bearing))
90     self._refresh(track)
91
92 def _accept_negative(self, track: FastTrack, site: Arr) -> None:
93     track.negatives.append(site.copy())
94     if track.poly is not None and not self.directional:
95         for obs in track.obs:
96             track.poly = self._range_order(track.poly, obs.site, site)
97         self._refresh(track)
98
99 def _sense(self, track: FastTrack, site: Arr) -> dict[str, object]:
100     cached = self._cached(track, site)
101     if cached is not None:
102         return cached
103     body = self.client.measure(float(site[0]), float(site[1]), track.
channel)
104     self.pos = site.copy()
105     self.current_channel = track.channel
106     self.measures += 1
107     self.virtual_s = float(body["virtual_time_s"])
108     track.history.append((site.copy(), dict(body)))
109     result = body.get("measure_result")
110     if result == "direction":
111         self._accept_direction(track, site, float(body["svd_deg"]))
112     elif result == "no_signal":
113         self._accept_negative(track, site)
114     elif result == "near":
115         self._clear(track, site, near=True)
116     else:
117         raise ApiError(f"unknown measurement result on channel {track.
channel}")
118     return body
119
120 def _clear(self, track: FastTrack, site: Arr, near: bool = False,
certified: bool = True) -> bool:
121     if track.status == "cleared":
122         return True
123     if certified and not near:

```

```

125     if track.poly is None:
126         raise ApiError("clear requested without geometric evidence")
127     far = float(np.max(np.linalg.norm(track.poly - site, axis=1)))
128     if not math.isfinite(far) or far > 19.999:
129         raise ApiError("clear position failed the vertex certificate")
130     if not certified and not self.directional:
131         raise ApiError("uncertified clear is disabled in Q3")
132     body = self.client.clear(float(site[0]), float(site[1]), track.
channel)
133     self.pos = site.copy()
134     self.clear_calls += 1
135     self.virtual_s = float(body["virtual_time_s"])
136     result = body.get("clear_result")
137     if result == "success":
138         track.status = "cleared"
139         return True
140     if result != "no_target_in_range":
141         raise ApiError(f"unknown clear result on channel {track.channel}")
142     if certified:
143         track.status = "geometry_fault"
144         raise ApiError(f"certified clear failed on channel {track.channel}
")
145     track.failures.append(site.copy())
146     return False
147
148 def _clear_position(self, track: FastTrack) -> Arr:
149     if track.center is None:
150         raise ApiError("missing localization center")
151     delta = self.pos - track.center
152     distance = float(np.linalg.norm(delta))
153     slack = max(0.0, 19.998 - track.radius)
154     if distance <= slack:
155         return self.pos.copy()
156     if distance == 0:
157         return track.center.copy()
158     return track.center + delta * (slack / distance)
159
160 def _ready_clear(self, track: FastTrack) -> bool:
161     if track.status == "cleared":
162         return True
163     if track.radius <= 19.999:
164         return self._clear(track, self._clear_position(track))
165     return False
166
167 def _guaranteed_signal(self, track: FastTrack, site: Arr) -> bool:
168     if self.directional or track.poly is None:
169         return False

```

```

170     if np.max(np.linalg.norm(track.poly - site, axis=1)) < 999.99:
171         return True
172     for obs in track.obs:
173         delta = site - obs.site
174         value = (float(delta @ delta)
175                 - 2.0 * ((track.poly - obs.site) @ delta))
176         if float(value.max()) < -guard:
177             return True
178     return False
179
180 def _forecast(self, track: FastTrack, site: Arr) -> float:
181     if track.poly is None or track.center is None:
182         return math.inf
183     return forecast(track.poly, track.center, track.radius, site, self.
184                     err_deg)
185
186 def _harvest(self, site: Arr, excluded: int = 0) -> None:
187     if not self.opportunistic:
188         return
189     pending = []
190     for track in self.tracks.values():
191         if (track.channel == excluded or track.status in {"unknown", "
192         cleared",
193         "absent_certified"}) or track.center is None or track.poly is
194         None
195         or track.opportunities >= 6 or self._cached(track, site) is
196         not None):
197         continue
198         if track.radius <= 19.999:
199         continue
200         if np.linalg.norm(site - track.center) > 1500.0 + track.radius:
201         continue
202         pred = self._forecast(track, site)
203         if pred <= 19.5 or pred < 0.68 * track.radius:
204             pending.append(track)
205     pending.sort(key=lambda t: (t.channel != self.current_channel, t.
206     channel))
207     for track in pending:
208         track.opportunities += 1
209         self._sense(track, site)
210         if track.status != "cleared" and track.poly is not None:
211             if np.max(np.linalg.norm(track.poly - site, axis=1)) <= 19.999:
212                 self._clear(track, site)
213
214 def _certify(self) -> None:
215     found = sum(bool(t.obs) or t.status == "cleared"
216                 for t in self.tracks.values())

```

```

212     if found > 16:
213         raise ApiError("observations exceed the source-count upper bound")
214     for track in self.tracks.values():
215         if track.status == "unknown":
216             if found == 16 or track.station_mask == self.full_mask:
217                 track.status = "absent_certified"
218
219     def _scan_station(self, index: int) -> None:
220         site = self.sites[index]
221         unknown = [t for t in self.tracks.values() if t.status == "unknown"]
222         unknown.sort(key=lambda t: (t.channel != self.current_channel, t.
channel))
223         for track in unknown:
224             if track.status != "unknown":
225                 continue
226             body = self._sense(track, site)
227             if body.get("measure_result") == "no_signal":
228                 track.station_mask |= 1 << index
229             self._certify()
230             if np.linalg.norm(self.pos - site) <= 1e-7:
231                 self._harvest(site)
232
233     def _choose_probe(self, track: FastTrack) -> Arr | None:
234         if track.center is None or track.poly is None:
235             return None
236         center, radius = track.center, track.radius
237         first = track.obs[0]
238         theta = math.radians(first.bearing_deg)
239         u = np.array([math.cos(theta), math.sin(theta)])
240         v = np.array([-u[1], u[0]])
241         candidates = [center.copy(), self.pos.copy()]
242         for scale in (0.6, 1.5):
243             distance = max(30.0, min(240.0, radius * scale))
244             for axis in (u, v):
245                 candidates.extend((center + distance * axis, center - distance *
axis))
246         if len(track.obs) == 1:
247             for side in (-1.0, 1.0):
248                 candidates.append(safe_lateral_site(first.site, first.
bearing_deg, side))
249         best = None
250         best_score = math.inf
251         for site in candidates:
252             if self._cached(track, site) is not None:
253                 continue
254             if not self.directional and not self._guaranteed_signal(track,
site):

```

```

255         continue
256     if np.max(np.linalg.norm(track.poly - site, axis=1)) > 1499.99:
257         continue
258     pred = self._forecast(track, site)
259     travel = float(np.linalg.norm(site - self.pos))
260     onward = max(0.0, float(np.linalg.norm(site - center)) - 20.0)
261     score = (travel + onward + 2.0 * pred) / 5.0
262     score += 6.0 * max(0.0, math.log2(max(pred, 20.0) / 20.0))
263     if self.directional:
264         old = first.site - center
265         new = site - center
266         if float(old @ new) < 0:
267             score += 12.0
268     if score < best_score:
269         best_score, best = score, site.copy()
270     return best
271
272 def _localize(self, track: FastTrack) -> None:
273     stagnant = 0
274     for _ in range(10):
275         if self._ready_clear(track):
276             return
277         site = self._choose_probe(track)
278         if site is None:
279             break
280         old = track.radius
281         body = self._sense(track, site)
282         self._harvest(site, excluded=track.channel)
283         if track.status == "cleared":
284             return
285         if body.get("measure_result") == "no_signal":
286             break
287         if track.radius > 0.995 * old:
288             stagnant += 1
289         else:
290             stagnant = 0
291         if stagnant >= 2:
292             break
293     for _ in range(8):
294         if self._ready_clear(track):
295             return
296         if track.radius > 300.0:
297             break
298         self._close_packet(track)
299         self._finish_cover(track)
300
301 def _close_packet(self, track: FastTrack) -> None:

```

```

302     if track.center is None or track.radius > 300.0:
303         return
304     center, old = track.center.copy(), track.radius
305     radius = 2.0 * old + 40.0
306     phase = math.atan2(self.pos[1] - center[1], self.pos[0] - center[0])
307     angles = phase + np.arange(6) * math.pi / 3.0
308     sites = center + radius * np.column_stack((np.cos(angles), np.sin(
309         angles)))
310     for i in route_order(sites, self.pos):
311         if self._ready_clear(track):
312             return
313         self._sense(track, sites[i])
314         if track.status == "cleared" or track.radius < 0.55 * old:
315             return
316
317 def _finish_cover(self, track: FastTrack) -> None:
318     if self._ready_clear(track):
319         return
320     if track.poly is None:
321         raise ApiError("missing fallback region")
322     angle = math.radians(track.obs[0].bearing_deg)
323     cells = cover_cells(track.poly, 4.5, angle)
324     centers = np.vstack([enclosing(cell)[0] for cell in cells])
325     left = set(range(len(cells)))
326     while left:
327         if self._ready_clear(track):
328             return
329         i = min(left, key=lambda j: (float(np.linalg.norm(centers[j] -
330             self.pos)), j))
331         left.remove(i)
332         if definitely_disjoint(cells[i], track.poly):
333             continue
334         self._sense(track, centers[i])
335         if not self._ready_clear(track):
336             track.status = "geometry_fault"
337             raise ApiError(f"near-coverage exhausted on channel {track.channel
338                 }")
339
340 def _insert_services(self, remaining: Arr) -> None:
341     if not self.inline or len(remaining) == 0:
342         return
343     while True:
344         choices = []
345         for track in self.tracks.values():
346             if (track.status in {"unknown", "cleared", "absent_certified"}
347                 or track.center is None or track.radius > 180.0):
348                 continue

```

```

346         index, cost = insertion(track.center, self.pos, remaining)
347         if index != 0:
348             continue
349         if track.radius > 19.999 and any(
350             self._cached(track, site) is None
351             and self._forecast(track, site) <= 19.5 for site in
remaining[:3]):
352             continue
353         choices.append((cost + track.radius, track.channel, track))
354         if not choices:
355             return
356         _, _, track = min(choices, key=lambda x: (x[0], x[1]))
357         self._localize(track)
358
359     def _finish_pending(self) -> None:
360         while True:
361             pending = [t for t in self.tracks.values()
362                 if t.status not in {"cleared", "absent_certified", "
unknown"}]
363             if not pending:
364                 return
365             if any(t.center is None for t in pending):
366                 raise ApiError("pending channel has no localization region")
367             points = np.vstack([t.center for t in pending])
368             order = route_order(points, self.pos)
369             self._localize(pending[order[0]])
370
371     def _exit(self) -> None:
372         if self._entered and not self._exited:
373             body = self.client.exit()
374             self._exited = True
375             self.virtual_s = float(body["virtual_time_s"])
376
377     def run(self, enter_body: dict[str, object] | None = None) ->
MissionResult:
378         if self._entered:
379             raise ApiError("agent instances cannot enter a second mission")
380         if enter_body is None:
381             enter_body = self.client.enter()
382         self._entered = True
383         self.virtual_s = float(enter_body["virtual_time_s"])
384         try:
385             for index in range(len(self.sites)):
386                 self._certify()
387                 if not any(t.status == "unknown" for t in self.tracks.values()):
388                     break
389                 self._insert_services(self.sites[index])

```

```

390         self._scan_station(index)
391     self._certify()
392     self._finish_pending()
393     self._certify()
394     if any(t.status not in {"cleared", "absent_certified"}
395           for t in self.tracks.values()):
396         raise ApiError("mission has unresolved channel certificates")
397     cleared = sum(t.status == "cleared" for t in self.tracks.values())
398     if not 10 <= cleared <= 16:
399         raise ApiError("cleared count violates the stated source-count
400 bounds")
401     self._exit()
402     return MissionResult(cleared, 20 - cleared, self.measures,
403                          self.clear_calls, self.virtual_s, True)
404 except BaseException:
405     try:
406         self._exit()
407     except Exception:
408         pass
409     raise
410
411 class Q3FastPlanner(Q3FastBase):
412     def __init__(self, client: ApiClient, err_deg: float = 1.01,
413                  opportunistic: bool = True, inline: bool = True) -> None:
414         super().__init__(client, err_deg, opportunistic, inline)
415         self.todo = set(range(len(self.sites)))
416         self.future = list(range(len(self.sites)))
417         self.inference_cache: dict[tuple[int, int, int], bool] = {}
418         self.inferred_negative_count = 0
419         self.next_channel = 0
420
421     def _inferred_at(self, track: FastTrack, index: int) -> bool:
422         if self.directional:
423             return False
424         key = (track.channel, len(track.negatives), index)
425         if key not in self.inference_cache:
426             self.inference_cache[key] = unknown_negative_implied(
427                 self.sites[index], track.negatives)
428         return self.inference_cache[key]
429
430     def _retire_stations(self) -> None:
431         self._certify()
432         unknown = [t for t in self.tracks.values() if t.status == "unknown"]
433         if not unknown:
434             self.todo.clear()
435             return
436         for index in sorted(self.todo):

```

```

436         bit = 1 << index
437         for track in unknown:
438             if not track.station_mask & bit and self._inferred_at(track,
index):
439                 track.station_mask |= bit
440                 self.inferred_negative_count += 1
441             if all(t.station_mask & bit for t in unknown):
442                 self.todo.remove(index)
443         self._certify()
444
445     def _known_negative(self, track: FastTrack, site: Arr) -> bool:
446         if track.center is None or track.poly is None:
447             return False
448         if np.linalg.norm(site - track.center) - track.radius > 1500.01:
449             return True
450         if self.directional:
451             return False
452         for previous in track.negatives:
453             delta = previous - site
454             value = float(delta @ delta) - 2.0 * ((track.poly - site) @ delta)
455             if float(np.max(value)) < -1e-5:
456                 return True
457         return False
458
459     def _opportunity_jobs(self, site: Arr, excluded: int = 0) -> list[
FastTrack]:
460         if not self.opportunistic:
461             return []
462         jobs = []
463         future = [self.sites[i] for i in self.future
464                     if i in self.todo and np.linalg.norm(self.sites[i] - site)
> 1e-7]
465         for track in self.tracks.values():
466             if (track.channel == excluded or track.center is None or track.
poly is None
467                 or track.status in {"unknown", "cleared", "absent_certified"}
468                 or track.radius <= 19.999 or track.opportunities >= 5
469                 or self._cached(track, site) is not None or self.
_known_negative(track, site)):
470                 continue
471             pred = self._forecast(track, site)
472             saving = 2.0 * max(0.0, track.radius - pred) / 5.0
473             cost = 5.0 + float(track.channel != self.current_channel) + float(
excluded != 0)
474             if pred > 19.5 and saving < 1.5 * cost:
475                 continue
476             if pred > 19.5 and pred >= 0.68 * track.radius:

```

```

477         continue
478     if excluded and pred > 19.5 and pred >= 0.35 * track.radius:
479         continue
480     defer = False
481     for later in future[:3]:
482         if self._cached(track, later) is not None or self.
_known_negative(track, later):
483             continue
484         if (np.linalg.norm(later - track.center) + 100.0
485             < np.linalg.norm(site - track.center)
486             and self._forecast(track, later) <= pred + 1.0):
487             defer = True
488             break
489     if not defer:
490         jobs.append(track)
491     return jobs
492
493 def _order_jobs(self, jobs: list[FastTrack]) -> list[FastTrack]:
494     def key(track: FastTrack) -> tuple[int, int]:
495         if track.channel == self.current_channel:
496             return 0, track.channel
497         if track.channel == self.next_channel:
498             return 2, track.channel
499         return 1, track.channel
500     return sorted(jobs, key=key)
501
502 def _clear_here(self, track: FastTrack, site: Arr) -> None:
503     if track.status != "cleared" and track.poly is not None:
504         if np.max(np.linalg.norm(track.poly - site, axis=1)) <= 19.999:
505             self._clear(track, site)
506
507 def _harvest(self, site: Arr, excluded: int = 0) -> None:
508     for track in self._order_jobs(self._opportunity_jobs(site, excluded)
509 ):
510         track.opportunities += 1
511         self._sense(track, site)
512         self._clear_here(track, site)
513
514 def _scan_station(self, index: int) -> None:
515     site = self.sites[index]
516     bit = 1 << index
517     mandatory = [t for t in self.tracks.values()
518                 if t.status == "unknown" and not t.station_mask & bit]
519     optional = self._opportunity_jobs(site)
520     required_channels = {t.channel for t in mandatory}
521     for track in self._order_jobs(mandatory + optional):
522         if track.status in {"cleared", "absent_certified"}:

```

```

522         continue
523     if track.channel not in required_channels:
524         track.opportunities += 1
525     body = self._sense(track, site)
526     if track.channel in required_channels and body.get("measure_result") == "no_signal":
527         track.station_mask |= bit
528         self._clear_here(track, site)
529         self._certify()
530
531 def _plan(self) -> tuple[str, int] | None:
532     self._retire_stations()
533     indices = [i for i in self.future if i in self.todo]
534     indices.extend(i for i in sorted(self.todo) if i not in indices)
535     nodes: list[tuple[str, int]] = [("scan", i) for i in indices]
536     points = [self.sites[i] for i in indices]
537     for track in self.tracks.values():
538         if (track.center is None
539             or track.status in {"unknown", "cleared", "absent_certified"}):
540             continue
541         if self.todo and (not self.inline or track.radius > 300.0):
542             continue
543         if self.todo and track.radius > 19.999 and any(
544             self._cached(track, self.sites[i]) is None
545             and not self._known_negative(track, self.sites[i])
546             and self._forecast(track, self.sites[i]) <= 19.5
547             and np.linalg.norm(self.sites[i] - track.center) + 100.0
548             < np.linalg.norm(self.pos - track.center) for i in indices
549             [:3]):
550             continue
551         nodes.append(("service", track.channel))
552         points.append(track.center)
553     if not nodes:
554         return None
555     order = joint_route(np.asarray(points), self.pos, len(indices))
556     self.future = [nodes[i][1] for i in order if nodes[i][0] == "scan"]
557     self.next_channel = next((nodes[i][1] for i in order if nodes[i][0] == "service"), 0)
558     return nodes[order[0]]
559
560 def run(self, enter_body: dict[str, object] | None = None) -> MissionResult:
561     if self._entered:
562         raise ApiError("agent instances cannot enter a second mission")
563     if enter_body is None:
564         enter_body = self.client.enter()

```

```

564     self._entered = True
565     self.virtual_s = float(enter_body["virtual_time_s"])
566     try:
567         while True:
568             action = self._plan()
569             if action is None:
570                 break
571             kind, index = action
572             if kind == "scan":
573                 self.todo.remove(index)
574                 self._scan_station(index)
575             else:
576                 self._localize(self.tracks[index])
577         self._certify()
578         if any(t.status not in {"cleared", "absent_certified"}
579               for t in self.tracks.values()):
580             raise ApiError("mission has unresolved channel certificates")
581         cleared = sum(t.status == "cleared" for t in self.tracks.values())
582         if not 10 <= cleared <= 16:
583             raise ApiError("cleared count violates the stated source-count
584                             bounds")
585         self._exit()
586         return MissionResult(cleared, 20 - cleared, self.measures,
587                             self.clear_calls, self.virtual_s, True)
588     except BaseException:
589         try:
590             self._exit()
591         except Exception:
592             pass
593         raise
594
595 class LocalProbe:
596     def __init__(self, *args, **kwargs) -> None:
597         super().__init__(*args, **kwargs)
598         self.phase = "startup"
599         self.branches: Counter[str] = Counter()
600         self.last_bound: float | None = None
601
602     def _certify(self) -> None:
603         found = sum(bool(t.obs) or t.status == "cleared" for t in self.
604                     tracks.values())
605         if found == 16 and any(t.status == "unknown" for t in self.tracks.
606                               values()):
607             self.branches["found_16_stop"] += 1
608             super()._certify()
609
610     def _scan_station(self, index: int) -> None:

```

```

608     before, self.phase = self.phase, "scan"
609     self.branches["scan_station"] += 1
610     try:
611         super()._scan_station(index)
612     finally:
613         self.phase = before
614
615     def _localize(self, track: FastTrack) -> None:
616         before, self.phase = self.phase, "localize"
617         self.branches["localize"] += 1
618         try:
619             super()._localize(track)
620         finally:
621             self.phase = before
622
623     def _harvest(self, site: Arr, excluded: int = 0) -> None:
624         before, self.phase = self.phase, "harvest"
625         try:
626             super()._harvest(site, excluded)
627         finally:
628             self.phase = before
629
630     def _choose_probe(self, track: FastTrack) -> Arr | None:
631         base = super()._choose_probe(track)
632         self.last_bound = None
633         if track.center is None or track.poly is None or track.radius >
634             450.0:
635             return base
636             center, radius = track.center, track.radius
637             base_distance = math.inf if base is None else float(
638                 np.linalg.norm(base - self.pos) + max(0.0, np.linalg.norm(base -
639                 center) - 20.0))
640             candidates: list[Arr] = []
641             if self.directional:
642                 positives = np.asarray([obs.site for obs in track.obs])
643                 kernel = visible_kernel(track.poly, positives)
644                 if len(kernel):
645                     candidates.extend(kernel)
646                     candidates.append(kernel.mean(axis=0))
647                     close = kernel[int(np.argmin(np.linalg.norm(kernel - center,
648                     axis=1)))]
649                     candidates.append(0.8 * close + 0.2 * kernel.mean(axis=0))
650             else:
651                 if base is not None:
652                     candidates.append(base)
653                 phase = math.atan2(self.pos[1] - center[1], self.pos[0] - center
654                 [0])

```

```

651     angles = phase + np.arange(8) * math.pi / 4.0
652     size = max(25.0, 1.2 * radius + 8.0)
653     candidates.extend(center + size * np.column_stack((np.cos(angles),
654     np.sin(angles))))
655     candidates.append(center.copy())
656     choices = []
657     for site in candidates:
658         if self._cached(track, site) is not None:
659             continue
660         if not self.directional and not self._guaranteed_signal(track,
661         site):
662             continue
663         if self._forecast(track, site) >= 0.85 * radius:
664             continue
665         travel = float(np.linalg.norm(site - self.pos))
666         onward = max(0.0, float(np.linalg.norm(site - center)) - 20.0)
667         if travel + onward > base_distance + 1e-6:
668             continue
669         nominal = (travel + onward + 2 * self._forecast(track, site)) /
670         5.0
671         choices.append((nominal, travel, onward, site))
672     best, value = None, math.inf
673     for _, travel, onward, site in sorted(choices, key=lambda row: row
674     [0])[0]:
675         bound = bearing_radius_bound(track.poly, site, self.err_deg)
676         if bound > 19.5 and bound >= 0.75 * radius:
677             continue
678         score = (travel + onward + 2 * bound) / 5.0
679         score += 6 * max(0.0, math.log2(max(bound, 20.0) / 20.0))
680         if score < value:
681             best, value, self.last_bound = site.copy(), score, bound
682     if best is not None:
683         self.branches["bounded_probe"] += 1
684         return best
685     return base
686
687 def _close_packet(self, track: FastTrack) -> None:
688     if track.center is None or track.radius > 300.0:
689         return
690     before, self.phase = self.phase, "small_packet"
691     self.branches["small_packet"] += 1
692     old = track.radius
693     sites = packet_sites(track.center.copy(), old, self.pos)
694     try:
695         for site in sites:
696             if self._ready_clear(track):
697                 return

```

```

694         self._sense(track, site)
695         if track.status == "cleared" or track.radius < 0.55 * old:
696             return
697         finally:
698             self.phase = before
699
700     def _finish_cover(self, track: FastTrack) -> None:
701         before, self.phase = self.phase, "near_cover"
702         self.branches["near_cover"] += 1
703         try:
704             super()._finish_cover(track)
705         finally:
706             self.phase = before
707
708     class Q3FastAgent(LocalProbe, Q3FastPlanner):
709         pass

```

问题四提交版智能体 robot/q4_agent_fast.py

```

1  from __future__ import annotations
2
3  import math
4
5  import numpy as np
6
7  from robot.q3_agent_fast import FastTrack, LocalProbe, Q3FastBase,
   Q3FastPlanner
8  from robot.client import ApiClient, ApiError
9  from robot.geometry_solver_fast import (
10     Arr, compact_q4_sites, cover_cells, definitely_disjoint,
   direction_certificate,
11     enclosing, packet_localization_bound, packet_sites, path_length,
   route_order,
12 )
13 from robot.q4_agent import import q4_scan_sites
14
15 def q4_ring_sites() -> Arr:
16     inner_r, outer_r = 980.0, 1870.0
17     angle = np.arange(12) * math.pi / 6.0
18     inner = inner_r * np.column_stack((np.cos(angle), np.sin(angle)))
19     outer = outer_r * np.column_stack((
20         np.cos(angle + math.pi / 12.0), np.sin(angle + math.pi / 12.0)))
21     cross_edge = math.sqrt(inner_r**2 + outer_r**2
22                             - 2 * inner_r * outer_r * math.cos(math.pi /
23                             12.0))
24     max_edge = max(inner_r, 2 * inner_r * math.sin(math.pi / 12.0),
25                     2 * outer_r * math.sin(math.pi / 12.0), cross_edge)
26     if max_edge >= 999.0 or outer_r * math.cos(math.pi / 12.0) <= 1800.0:

```

```

26     raise ValueError("direction-complete ring certificate failed")
27 return np.vstack((np.zeros(2), inner, outer[np.r_[11, np.arange(11)]])
28 )
29
30 class Q4FastBase(Q3FastBase):
31     directional = True
32
33     def __init__(self, client: ApiClient, err_deg: float = 1.01,
34                 opportunistic: bool = True, inline: bool = True,
35                 scan: str = "rings") -> None:
36         if scan not in {"rings", "lattice"}:
37             raise ValueError("Q4 scan must be rings or lattice")
38         self.scan = scan
39         super().__init__(client, err_deg, opportunistic, inline)
40
41     def _scan_sites(self) -> Arr:
42         return q4_ring_sites() if self.scan == "rings" else q4_scan_sites()
43
44     def _coarse_packet(self, track: FastTrack) -> None:
45         if track.poly is None:
46             raise ApiError("missing directional region")
47         origin = track.obs[0].site
48         angle = math.radians(track.obs[0].bearing_deg)
49         u = np.array([math.cos(angle), math.sin(angle)])
50         v = np.array([-u[1], u[0]])
51         local = track.poly - origin
52         xs, ys = local @ u, local @ v
53         lo, hi = float(xs.min()) - 60.0, float(xs.max()) + 60.0
54         bottom, top = float(ys.min()) - 160.0, float(ys.max()) + 160.0
55         nx = max(1, int(math.ceil((hi - lo) / 450.0)))
56         ny = max(1, int(math.ceil((top - bottom) / 450.0)))
57         sites = np.array([origin + x * u + y * v
58                         for x in np.linspace(lo, hi, nx + 1)
59                         for y in np.linspace(bottom, top, ny + 1)])
60         for i in route_order(sites, self.pos):
61             if self._ready_clear(track) or track.radius <= 180.0:
62                 return
63             self._sense(track, sites[i])
64             self._harvest(sites[i], excluded=track.channel)
65
66     def _localize(self, track: FastTrack) -> None:
67         for _ in range(3):
68             if self._ready_clear(track):
69                 return
70             site = self._choose_probe(track)
71             if site is None:
72                 break

```

```

72     old = track.radius
73     body = self._sense(track, site)
74     self._harvest(site, excluded=track.channel)
75     if track.status == "cleared":
76         return
77     if body.get("measure_result") == "no_signal" or track.radius > 0.9
* old:
78         break
79     if self._ready_clear(track):
80         return
81     if track.radius > 300.0:
82         self._coarse_packet(track)
83     for _ in range(8):
84         if self._ready_clear(track):
85             return
86         if track.radius > 300.0:
87             break
88         if self._prefer_cover(track):
89             break
90         self._close_packet(track)
91     self._finish_cover(track)
92
93 def _prefer_cover(self, track: FastTrack) -> bool:
94     if track.poly is None or track.center is None or track.radius >
150.0:
95         return False
96     angle = math.radians(track.obs[0].bearing_deg)
97     cells = cover_cells(track.poly, 19.5, angle)
98     centers = np.vstack([enclosing(cell)[0] for cell in cells])
99     order = route_order(centers, self.pos)
100     cover_time = path_length(centers[order], self.pos) / 5.0
101     cover_time += 3.0 * (len(cells) - 1) + 5.0
102     ring_r = 2.0 * track.radius + 40.0
103     approach = abs(float(np.linalg.norm(self.pos - track.center)) -
ring_r)
104     packet_time = (approach + 2.0 * ring_r) / 5.0 + 15.0
105     return cover_time <= packet_time
106
107 def _finish_cover(self, track: FastTrack) -> None:
108     if self._ready_clear(track):
109         return
110     if track.poly is None:
111         raise ApiError("missing directional fallback region")
112     angle = math.radians(track.obs[0].bearing_deg)
113     cells = cover_cells(track.poly, 19.5, angle)
114     centers = np.vstack([enclosing(cell)[0] for cell in cells])
115     for i in route_order(centers, self.pos):

```

```

116     cell = cells[i]
117     if definitely_disjoint(cell, track.poly):
118         continue
119     if any(np.max(np.linalg.norm(cell - old, axis=1)) <= 19.999
120           for old in track.failures):
121         continue
122     if any(np.linalg.norm(centers[i] - old) <= 1e-7 for old in track.
failures):
123         continue
124     if self._clear(track, centers[i], certified=False):
125         return
126     track.status = "geometry_fault"
127     raise ApiError(f"directional coverage exhausted on channel {track.
channel}")
128
129 class Q4FastCertified(Q4FastBase, Q3FastPlanner):
130     def __init__(self, client: ApiClient, err_deg: float = 1.01,
131                 opportunistic: bool = True, inline: bool = True,
132                 scan: str = "compact") -> None:
133         if scan not in {"compact", "rings", "lattice"}:
134             raise ValueError("Q4 scan must be compact, rings or lattice")
135         self.use_compact = scan == "compact"
136         self.certificate_cells = 0
137         self.compact_certified = False
138         super().__init__(client, err_deg, opportunistic, inline,
139                         "rings" if self.use_compact else scan)
140
141     def _scan_sites(self) -> Arr:
142         if not self.use_compact:
143             return super()._scan_sites()
144         sites = compact_q4_sites()
145         ok, self.certificate_cells = direction_certificate(sites)
146         self.compact_certified = ok
147         return sites if ok else q4_ring_sites()
148
149 class Q4FastAgent(LocalProbe, Q4FastCertified):
150     def __init__(self, *args, empty_limit: int = 5, **kwargs) -> None:
151         if not 0 <= empty_limit <= 5:
152             raise ValueError("empty_limit must be in 0..5")
153         super().__init__(*args, **kwargs)
154         self.empty_limit = empty_limit
155         self.empty_calls = 0
156
157     def _clear(self, track: FastTrack, site: Arr, near: bool = False,
158              certified: bool = True) -> bool:
159         if track.status == "cleared":
160             return True

```

```

161     if not certified and self.empty_calls >= self.empty_limit:
162         if track.poly is None or not track.obs:
163             raise ApiError("missing last-cell certificate")
164         angle = math.radians(track.obs[0].bearing_deg)
165         cells = cover_cells(track.poly, 19.5, angle)
166         disks = track.failures + [site]
167         if not all(any(np.max(np.linalg.norm(cell - p, axis=1)) <= 19.999
168                        for p in disks) for cell in cells):
169             raise ApiError("clear would exceed the mission-wide budget")
170     ok = super()._clear(track, site, near, certified)
171     if not ok:
172         self.empty_calls += 1
173         if self.empty_calls > self.empty_limit:
174             track.status = "geometry_fault"
175             raise ApiError("last-cell clear contradicted its coverage
certificate")
176     return ok
177
178 def _cover_fits(self, track: FastTrack) -> bool:
179     if track.poly is None or not track.obs:
180         return False
181     angle = math.radians(track.obs[0].bearing_deg)
182     count = len(cover_cells(track.poly, 19.5, angle))
183     return count - 1 <= self.empty_limit - self.empty_calls
184
185 def _prefer_cover(self, track: FastTrack) -> bool:
186     return self._cover_fits(track) and super()._prefer_cover(track)
187
188 def _coarse_packet(self, track: FastTrack) -> None:
189     before, self.phase = self.phase, "coarse_packet"
190     self.branches["coarse_packet"] += 1
191     try:
192         super()._coarse_packet(track)
193     finally:
194         self.phase = before
195
196 def _finish_cover(self, track: FastTrack) -> None:
197     if self._ready_clear(track):
198         return
199     if self._cover_fits(track):
200         before, self.phase = self.phase, "budgeted_clear_cover"
201         self.branches["budgeted_clear_cover"] += 1
202     try:
203         Q4FastBase._finish_cover(self, track)
204     finally:
205         self.phase = before
206     return

```

```

207     if track.poly is None:
208         raise ApiError("missing directional packet-cover region")
209     before, self.phase = self.phase, "directional_packet_cover"
210     self.branches["directional_packet_cover"] += 1
211     try:
212         angle = math.radians(track.obs[0].bearing_deg)
213         radius = 19.5
214         while packet_localization_bound(radius, self.err_deg) > 19.0:
215             radius /= 2
216             if radius < 0.01:
217                 raise ApiError("no finite packet localization certificate")
218             cells = cover_cells(track.poly, radius, angle)
219             centers = np.vstack([enclosing(cell)[0] for cell in cells])
220             left = set(range(len(cells)))
221             while left:
222                 if self._ready_clear(track):
223                     return
224                 i = min(left, key=lambda j: (float(np.linalg.norm(centers[j] -
self.pos)), j))
225                 left.remove(i)
226                 if definitely_disjoint(cells[i], track.poly):
227                     continue
228                 sites = packet_sites(centers[i], radius, self.pos)
229                 for site in sites:
230                     self._sense(track, site)
231                     if self._ready_clear(track):
232                         return
233                 track.status = "geometry_fault"
234                 raise ApiError("certified directional packet coverage exhausted")
235             finally:
236                 self.phase = before

```

提交版入口 robot/main_fast.py

```

1  from __future__ import annotations
2
3  import argparse
4  import hashlib
5  import json
6  import math
7  import os
8  import sys
9  import time
10 from collections import Counter
11 from pathlib import Path
12 from typing import TextIO
13
14 root = Path(__file__).resolve().parents[1]

```

```

15 sys.path.insert(0, str(root))
16
17 from robot.q3_agent_fast import Q3FastAgent
18 from robot.client import ApiClient
19 from robot.geometry_solver_fast import path_length
20 def redact(value: object, team: str) -> object:
21     if isinstance(value, dict):
22         return {key: "<redacted>" if key in {"robot_id", "team_no"}
23                 else redact(item, team) for key, item in value.items()}
24     if isinstance(value, (list, tuple)):
25         return [redact(item, team) for item in value]
26     if isinstance(value, str) and team:
27         return value.replace(team, "<redacted>")
28     return value
29 from robot.q4_agent_fast import Q4FastAgent
30
31 def instrument(client: ApiClient, agent, file: TextIO | None, team: str)
    -> dict:
32     original = client._post_once
33     seen: set[tuple[str, str]] = set()
34     positions: set[tuple[float, float]] = {(0.0, 0.0)}
35     measured: set[tuple[int, float, float]] = set()
36     state = {"pos": (0.0, 0.0), "channel": 1}
37     stats = {"distance_m": 0.0, "switches": 0, "site_revisits": 0,
38             "duplicate_site_channel": 0, "clear_failures": 0,
39             "phase": {}, "channels": {}, "segments_m": [], "log_errors":
40             0}
41
42 def wrapped(path: str, payload: dict) -> tuple[int, dict]:
43     phase = getattr(agent, "phase", "startup")
44     status, body = original(path, payload)
45     key = (path, str(payload.get("request_id")))
46     unique = status == 200 and body.get("accepted") is True and key not
47     in seen
48     if unique:
49         seen.add(key)
50         if path in {"/measure", "/clear"}:
51             channel = int(payload["channel"])
52             p = payload["position"]
53             pos = (float(p["x"]), float(p["y"]))
54             distance = math.dist(state["pos"], pos)
55             switched = int(path == "/measure" and channel != state["channel"]
56             ])
57             stats["distance_m"] += distance
58             stats["switches"] += switched
59             stats["segments_m"].append(distance)
60             if distance > 1e-7 and pos in positions:

```

```

58         stats["site_revisits"] += 1
59     positions.add(pos)
60     state["pos"] = pos
61     part = stats["phase"].setdefault(phase, Counter())
62     part["distance_m"] += distance
63     part["switches"] += switched
64     part[path.lstrip("/")] += 1
65     track = stats["channels"].setdefault(str(channel), Counter())
66     track[path.lstrip("/")] += 1
67     if path == "/measure":
68         state["channel"] = channel
69         measurement = (channel, *pos)
70         stats["duplicate_site_channel"] += int(measurement in measured
71 )
72         measured.add(measurement)
73         result = str(body.get("measure_result"))
74         part[result] += 1
75         track[result] += 1
76     elif body.get("clear_result") == "no_target_in_range":
77         stats["clear_failures"] += 1
78         part["clear_failures"] += 1
79         track["clear_failures"] += 1
80     if file is not None:
81         record = {"t": time.time_ns() // 1_000_000, "variant": "fast",
82                 "phase": phase, "path": path, "request": payload,
83                 "status": status, "response": body, "unique_accepted":
84 unique}
85         try:
86             file.write(json.dumps(redact(record, team), ensure_ascii=False)
87 + "\n")
88             file.flush()
89         except OSError:
90             stats["log_errors"] += 1
91     return status, body
92
93 client._post_once = wrapped
94 return stats
95
96 def main(argv: list[str] | None = None) -> int:
97     parser = argparse.ArgumentParser(description="Q3/Q4 fast")
98     parser.add_argument("--team", default=os.environ.get("CUMCM_TEAM_NO",
99 ""))
100     parser.add_argument("--problem", type=int, choices=(3, 4), default=3)
101     parser.add_argument("--base-url", default="http://127.0.0.1:2026")
102     parser.add_argument("--wait-s", type=float, default=300.0)
103     parser.add_argument("--q4-scan", choices=("compact", "rings", "lattice
104 "), default="compact")

```

```

100 parser.add_argument("--empty-limit", type=int, choices=range(6),
101                     default=5)
102 parser.add_argument("--no-opportunistic", action="store_true")
103 parser.add_argument("--no-inline", action="store_true")
104 parser.add_argument("--log-dir", help="optional directory for redacted
105     action logs")
106 args = parser.parse_args(argv)
107 if not args.team or not math.isfinite(args.wait_s) or args.wait_s <=
108     0:
109     print("provide --team or CUMCM_TEAM_NO; --wait-s must be a finite
110         positive number")
111     return 2
112 file: TextIO | None = None
113 try:
114     client = ApiClient(args.base_url, args.team)
115     options = {"opportunistic": not args.no_opportunistic, "inline": not
116         args.no_inline}
117     agent = (Q3FastAgent(client, **options) if args.problem == 3 else
118         Q4FastAgent(client, scan=args.q4_scan, empty_limit=args.
119     empty_limit, **options))
120     hashes = {p.name: hashlib.sha256(p.read_bytes()).hexdigest()
121         for name in ("*_fast*.py",)
122         for p in sorted((root / "robot").glob(name))}
123     if args.log_dir:
124         folder = Path(args.log_dir).resolve()
125         folder.mkdir(parents=True, exist_ok=True)
126         file = (folder / f"q{args.problem}_fast_{time.time_ns()}.jsonl").
127         open("x", encoding="utf-8")
128         file.write(json.dumps({"variant": "fast", "source_sha256": hashes
129     }) + "\n")
130     stats = instrument(client, agent, file, args.team)
131     print(f"Q{args.problem} fast ready, scan sites {len(agent.sites)},
132     waiting for the official test window")
133     enter = client.enter_when_open(wait_s=args.wait_s)
134     started = time.perf_counter()
135     result = agent.run(enter_body=enter)
136     lengths = stats.pop("segments_m")
137     bins = [0, 20, 100, 300, 1000, math.inf]
138     stats["segment_bins"] = {f"({lo},{hi})": sum(lo < x <= hi for x in
139     lengths)
140         for lo, hi in zip(bins, bins[1:])}
141     stats["stationary_actions"] = sum(x <= 1e-7 for x in lengths)
142     stats["max_segment_m"] = max(lengths, default=0.0)
143     accounted = (stats["distance_m"] / 5 + 5 * result.measures + stats["
144     switches"]
145         + 5 * result.cleared + 3 * stats["clear_failures"])
146     out = {"problem": args.problem, "variant": "fast", "source_sha256":

```

```

hashes,
136     "cleared": result.cleared, "absent_certified": result.
absent_certified,
137     "channel_certificates": {str(k): t.status for k, t in agent.
tracks.items()},
138     "measures": result.measures, "clear_calls": result.
clear_calls,
139     "branches": dict(agent.branches), "actions": stats,
140     "inferred_negative_count": agent.inferred_negative_count,
141     "scan_site_count": len(agent.sites),
142     "scan_seed_path_m": path_length(agent.sites, agent.sites[0]),
143     "virtual_time_s": result.virtual_time_s,
144     "time_account_residual_s": result.virtual_time_s - float(
enter["virtual_time_s"]) - accounted,
145     "mean_virtual_per_cleared_s": result.virtual_time_s / result.
cleared,
146     "program_s": time.perf_counter() - started, "complete":
result.complete}
147     text = json.dumps(redact(out, args.team), ensure_ascii=False)
148     print(text)
149     if file is not None:
150         try:
151             file.write(json.dumps({"summary": redact(out, args.team)},
ensure_ascii=False) + "\n")
152         except OSError:
153             print("mission complete, but writing the log summary failed")
154     return 0
155 except KeyboardInterrupt:
156     print("program interrupted")
157     return 130
158 except Exception as exc:
159     print(redact(f"run failed: {type(exc).__name__}: {exc}", args.team))
160     return 1
161 finally:
162     if file is not None:
163         try:
164             file.close()
165         except OSError:
166             pass
167
168 if __name__ == "__main__":
169     raise SystemExit(main())

```